

國立中央大學

114 學年度 第一學期

軟體工程 期末書面報告

E_F 測試計畫與測試報告

寵物領養平台

組別：第十七組

組員：

114423001 陳柏甫

114423007 潘維晟

114423042 劉和媛

114423043 李嶧

目錄

1 測試計畫.....	6
2 單元測試計畫.....	7
2.1 Admin Service 單元測試計畫.....	8
2.2 Animal Service 單元測試計畫.....	17
2.3 Application Service 單元測試計畫.....	37
2.4 Attachment Service 單元測試計畫.....	53
2.5 Audit Service 單元測試計畫.....	58
2.6 Auth Service 單元測試計畫.....	63
2.7 Email Service 單元測試計畫.....	73
2.8 Job Service 單元測試計畫.....	80
2.9 Medical Record Service 單元測試計畫.....	87
2.10 Notification Service 單元測試計畫.....	100
2.11 Permission Service 單元測試計畫.....	109
2.12 Shelter Service 單元測試計畫.....	120
2.13 User Service 單元測試.....	139
2.14 單元測試結果.....	150
3 整合測試.....	151
3.1 Animal 整合測試.....	153
3.1.2 TestAnimalListFilters - 動物列表進階篩選測試類.....	156
3.1.3 TestAnimalCreate - 動物創建測試類.....	157
3.1.4 TestAnimalDetail - 動物詳情測試類.....	158
3.1.5 TestAnimalGetDetail - 動物詳情擴展測試類.....	159
3.1.6 TestAnimalUpdate - 動物更新測試類.....	160

3.1.7 TestAnimalDelete - 動物刪除測試類	162
3.1.8 TestAnimalImages - 動物圖片管理測試類	163
3.1.9 TestAnimalImagePermissions - 動物圖片權限測試類	164
3.1.10 TestAnimalStatusManagement - 動物狀態管理測試類	167
3.1.11 TestAnimalStatusPermissions - 動物狀態權限測試類	171
3.1.12 TestAnimalEdgeCases - 動物管理邊界案例測試類	173
3.2 Application 整合測試	174
3.2.1 TestApplicationCreate - 創建領養申請測試類	174
3.2.2 TestApplicationList - 申請列表測試類	177
3.2.3 TestApplicationReview - 申請審核測試類	179
3.2.4 TestApplicationDetail - 申請詳情測試類	181
3.3 Auth 整合測試	182
3.3.1 TestAuthRegistration - 用戶註冊測試類	183
3.3.2 TestAuthLogin - 用戶登入測試類	184
3.3.4 TestAuthMe - 當前用戶資訊測試類	186
3.4 Medical Record 整合測試	187
3.4.1 TestMedicalRecordCreate - 創建醫療記錄測試類	188
3.4.2 TestMedicalRecordList - 醫療記錄列表測試類	189
3.4.3 TestMedicalRecordUpdate - 醫療記錄更新測試類	190
3.5 Notification 整合測試	191
3.5.1 TestNotificationList - 通知列表測試類	192
3.5.2 TestNotificationMarkRead - 標記通知已讀測試類	193
3.5.3 TestNotificationDelete - 刪除通知測試類	195
3.5.4 TestNotificationCreation - 通知創建測試類	196

3.5.5 TestNotificationCount - 通知計數測試類.....	196
3.6 Shelter 整合測試	197
3.6.1 TestShelterCreate - 收容所創建測試類.....	197
3.6.2 TestShelterList - 收容所列表測試類	198
3.6.3 TestShelterDetail - 收容所詳情測試類.....	199
3.6.4 TestShelterUpdate - 收容所更新測試類.....	200
3.6.5 TestShelterVerification - 收容所驗證測試類	201
3.7 User 整合測試	203
3.7.1 TestUserProfile - 用戶個人資料測試類	203
3.7.2 TestUserUpdate - 用戶更新測試類.....	203
3.8 整合測試結果.....	209
3.8.1 Animal.....	209
3.8.4 Medical	210
3.8.5 Notification	210
3.8.6 Shelter	211
3.8.7 User.....	211
3.8.8 覆蓋率.....	211
4. E2E 測試.....	212
4.1 執行摘要.....	212
4.2 測試目標與範圍.....	213
4.3 測試流程.....	213
4.4 測試執行總結.....	216
4.5 測試覆蓋分析.....	218
4.6 附錄.....	220
4.7 測試結果報告.....	221

5. 壓力測試.....	222
5.1 測試環境.....	222
5.2 登入.....	223
5.3 建立動物資料.....	229
5.4 檔案上傳.....	235

1 測試計畫

整體測試目標

本專案測試計畫旨在通過多層次測試策略，確保「貓狗領養平台」系統的品質、穩定性與可維護性。測試計畫涵蓋單元測試、整合測、E2E 測試與壓力測試四個層面。

測試目標

- 功能與邏輯：確保使用案例的行為正確執行
- 限制驗證：驗證系統能正確處理邊界條件
- 效能承載：透過壓力測試量化系統在高負載下的錯誤百分比和吞吐量

測試範疇

- 功能範圍：核心業務邏輯 (service layer)
- 流程範圍：涵蓋所有使用者(一般會員、收容所與系統管理員)在完整領養流程中的互動
- 效能範圍：系統在高負載下對「登入」、「建立動物資料」及「檔案上傳」等關鍵操作的承載能力

測試內容

- 單元測試：驗證基礎邏輯 (共 353 個測試)
- 整合測試：驗證元件協作(共 88 個測試)
- 端對端測試 (E2E)：驗證跨角色的完整操作流程
- 壓力測試：模擬 100 至 300 人操作，測量「登入」等操作的平均反應時間、最大值、錯誤率和處理量

測試類型

- 單元測試：灰箱測試
- 整合測試：灰箱測試
- E2E 測試：黑箱測試
- 壓力測試：黑箱測

2 單元測試計畫

單元測試採用服務層優先（Service-Layer-First）測試策略，聚焦於 `app/services/` 目錄下的業務邏輯層，確保核心功能的正確性與穩定性。（共有 353 測試）

測試聚焦點：Service 層

為何聚焦於 Service 層？

- 業務邏輯核心：Service 層包含系統的主要業務邏輯，是功能實現的核心。
- 獨立性高：Service 層相對獨立於框架和基礎設施，更容易進行單元測試。
- 投資報酬率最高：測試 Service 層能以最少的測試代碼覆蓋最多的業務場景。
- 易於維護：純粹的業務邏輯測試不受框架變更影響，長期維護成本低。

測試隔離策略：

- 使用 Mock 技術隔離外部依賴（資料庫、MinIO、Redis、SMTP）。
- 透過 Fixture 提供可重複使用的測試資料和環境設置。
- 每個測試獨立運行，互不干擾，確保**測試結果**的可靠性。

覆蓋率測量方法：

使用 `pytest-cov` 工具進行代碼覆蓋率分析，具體實施方式：

`pytest tests/unit --cov=app/services --cov-report=term --cov-report=html`

參數說明：

`--cov=app/services`：指定覆蓋率分析範圍為 `app/services/` 目錄

`--cov-report=term`：在終端輸出覆蓋率摘要表格

`--cov-report=html`：生成 HTML 詳細報告（存放於 `htmlcov/` 目錄）

覆蓋率指標：

語句覆蓋率（Statement Coverage）：測量有多少程式碼行被執行過。

2.1 Admin Service 單元測試計劃

Admin Service 的所有單元測試案例，共計 23 個測試案例。

測試編號：UT-ADMINSERVICE-001

測試目標：TC-01: 返回完整的系統統計資料

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AdminService.get_system_statistics() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-ADMINSERVICE-002

測試目標：TC-02: 分頁列出用戶

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AdminService.list_all_users() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 執行 AdminService.list_all_users()
- 驗證結果值正確

測試結果：通過

測試編號：UT-ADMINSERVICE-003

測試目標：TC-03: 依角色篩選用戶

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AdminService.list_all_users() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-ADMINSERVICE-004

測試目標：TC-04: 關鍵字搜尋用戶

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AdminService.list_all_users() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-ADMINSERVICE-005

測試目標：TC-05: 無效角色拋出錯誤

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AdminService.list_all_users() 方法
3. 確認拋出預期的異常

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-ADMINSERVICE-006

測試目標：TC-06: 成功封禁用戶

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AdminService.ban_user() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-ADMINSERVICE-007

測試目標：TC-07: 用戶不存在拋出錯誤

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AdminService.ban_user() 方法
3. 確認拋出預期的異常

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-ADMINSERVICE-008

測試目標：TC-08: 不能封禁管理員

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AdminService.ban_user() 方法
3. 確認拋出預期的異常

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-ADMINSERVICE-009

測試目標：TC-09: 成功解除封禁

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AdminService.unban_user() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 執行 AdminService.unban_user()
- 驗證結果值正確

測試結果：通過

測試編號：UT-ADMINSERVICE-010

測試目標：TC-10: 用戶不存在拋出錯誤

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AdminService.unban_user() 方法
3. 確認拋出預期的異常

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-ADMINSERVICE-011

測試目標： TC-11: 分頁列出動物

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AdminService.list_all_animals() 方法
3. 驗證結果符合預期

測試角色： 平台管理員

測試流程：

- 執行 AdminService.list_all_animals()
- 驗證結果值正確

測試結果： 通過

測試編號：UT-ADMINSERVICE-012

測試目標： TC-12: 依狀態篩選動物

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AdminService.list_all_animals() 方法
3. 驗證結果符合預期

測試角色： 平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-ADMINSERVICE-013

測試目標： TC-13: 包含已刪除動物

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 AdminService.list_all_animals() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-ADMINSERVICE-014

測試目標：TC-14: 分頁列出申請

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 AdminService.list_all_applications() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 執行 AdminService.list_all_applications()
- 驗證結果值正確

測試結果：通過

測試編號：UT-ADMINSERVICE-015

測試目標：TC-15: 依狀態篩選申請

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 AdminService.list_all_applications() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-ADMINSERVICE-016

測試目標：TC-16: 基本審計日誌列表

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AdminService.list_audit_logs() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 執行 AdminService.list_audit_logs()
- 驗證結果值正確

測試結果：通過

測試編號：UT-ADMINSERVICE-017

測試目標：TC-17: 依操作者篩選

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AdminService.list_audit_logs() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-ADMINSERVICE-018

測試目標：TC-18: 依操作類型篩選

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AdminService.list_audit_logs() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-ADMINSERVICE-019

測試目標： TC-19: 依日期範圍篩選

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AdminService.list_audit_logs() 方法
3. 驗證結果符合預期

測試角色： 平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-ADMINSERVICE-020

測試目標： TC-20: 無效開始日期格式

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AdminService.list_audit_logs() 方法
3. 確認拋出預期的異常

測試角色： 平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-ADMINSERVICE-021

測試目標：TC-21: 無效結束日期格式

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AdminService.list_audit_logs() 方法
3. 確認拋出預期的異常

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-ADMINSERVICE-022

測試目標：TC-22: 審計日誌包含操作者資訊

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AdminService.list_audit_logs() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 執行 AdminService.list_audit_logs()
- 驗證結果值正確

測試結果：通過

測試編號：UT-ADMINSERVICE-023

測試目標：TC-23: 取得審核員列表

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AdminService.get_reviewers() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 執行 AdminService.get_reviewers()
- 驗證結果值正確

測試結果：通過

2.2 Animal Service 單元測試計劃

Animal Service 的所有單元測試案例，共計 60 個測試案例。

測試編號：UT-ANIMAL-001

測試目標：驗證收容所成員創建動物歸屬收容所

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 收容所成員 (UserRole.SHELTER_MEMBER) 提供動物資料
2. 呼叫 AnimalService.create_animal(user, data)
3. 驗證動物的 shelter_id 等於用戶的 primary_shelter_id

測試角色：收容所成員

測試流程：

- Mock 收容所成員用戶對象 (role=SHELTER_MEMBER, primary_shelter_id=10)
- 提供動物資料 {'name': '小白', 'species': 'DOG'}
- 執行創建操作
- 驗證 animal.shelter_id == 10 且 animal.owner_id is None

測試結果：通過

測試編號：UT-ANIMAL-002

測試目標：驗證一般會員創建個人動物

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 一般會員 (UserRole.GENERAL_MEMBER) 提供動物資料
2. 執行創建操作
3. 驗證動物的 owner_id 等於用戶 ID

測試角色：一般會員

測試流程：

- Mock 一般會員用戶對象 (user_id=2, role=GENERAL_MEMBER, primary_shelter_id=None)
- 提供動物資料 {'name': '小黑', 'species': 'CAT'}
- 執行創建操作
- 驗證 animal.owner_id == 2 且 animal.shelter_id is None

測試結果：通過

測試編號：UT-ANIMAL-003

測試目標：驗證管理員可指定收容所

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 管理員 (UserRole.ADMIN) 提供動物資料及指定的 shelter_id
2. 執行創建操作
3. 驗證動物歸屬指定的收容所

測試角色：平台管理員

測試流程：

- Mock 管理員用戶對象 (role=ADMIN)
- 提供動物資料 {'name': '小花', 'species': 'DOG', 'shelter_id': 20}
- 執行創建操作
- 驗證 animal.shelter_id == 20

測試結果：通過

測試編號：UT-ANIMAL-004

測試目標：驗證防護測試 (同時指定 owner 和 shelter)

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 測試不能同時設定 owner_id 和 shelter_id

測試角色：平台

測試流程：

- 驗證資料完整性約束

測試結果：通過

測試編號：UT-ANIMAL-005

測試目標：驗證擁有者可更新自己的動物

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 擁有者提供更新資料
2. 呼叫 AnimalService.update_animal(animal_id, user, data)
3. 驗證資料更新成功

測試角色：動物擁有者

測試流程：

- Mock 動物擁有者 (user_id=1)
- Mock 動物對象 (animal_id=100, owner_id=1)
- Mock permission_service.can_manage_animal 返回 True
- 提供更新資料 {'name': '新名字'}
- 驗證 animal.name == '新名字'

測試結果：通過

測試編號：UT-ANIMAL-006

測試目標：驗證已刪除動物拋出 NotFoundError

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 查詢已刪除的動物
2. 驗證拋出 NotFoundError

測試角色：使用者

測試流程：

- Mock Animal.query.filter_by(deleted_at=None).first() 返回 None
- 呼叫 update_animal(999, user, data)
- 預期拋出 NotFoundError

測試結果：通過

測試編號：UT-ANIMAL-007

測試目標：驗證無權限用戶拋出 PermissionDeniedError

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 非擁有者嘗試更新動物
2. 驗證拋出 PermissionDeniedError

測試角色：非擁有者

測試流程：

- Mock permission_service.can_manage_animal 返回 False
- 呼叫 update_animal(100, user, data)
- 預期拋出 PermissionDeniedError

測試結果：通過

測試編號：UT-ANIMAL-008

測試目標：驗證更新多個欄位

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 同時更新動物的多個屬性 (name, breed, color, description)
2. 驗證所有欄位都正確更新

測試角色：動物擁有者

測試流程：

- 提供多欄位更新資料
- 執行更新操作
- 逐一驗證各欄位的新值

測試結果：通過

測試編號：UT-ANIMAL-009

測試目標：驗證更新動物出生日期 (dob)

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 更新動物的 dob 欄位
2. 驗證日期正確設定

測試角色：動物擁有者

測試流程：

- 提供 {'dob': '2020-01-15'}
- 執行更新
- 驗證 animal.dob 不為 None

測試結果：通過

測試編號：UT-ANIMAL-010

測試目標：驗證更新醫療摘要 (medical_summary)

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 更新動物的 medical_summary 欄位
2. 驗證內容正確設定

測試角色：動物擁有者

測試流程：

- 提供 {'medical_summary': '健康狀況良好'}
- 執行更新
- 驗證 animal.medical_summary == '健康狀況良好'

測試結果：通過

測試編號：UT-ANIMAL-011

測試目標：驗證更新動物狀態欄位

測試方法：狀態轉換測試 (State Transition Testing)

測試步驟：

1. 直接更新 status 欄位
2. 驗證狀態變更

測試角色：動物擁有者

測試流程：

- 提供 {'status': 'PUBLISHED'}
- 驗證 animal.status == AnimalStatus.PUBLISHED

測試結果：通過

測試編號：UT-ANIMAL-012

測試目標：驗證軟刪除設定 deleted_at 時間戳

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 擁有者刪除動物
2. 驗證 deleted_at 欄位被設定

測試角色：動物擁有者

測試流程：

- Mock animal.deleted_at = None
- Mock permission_service.can_manage_animal 返回 True
- 呼叫 delete_animal(100, user)
- 驗證 animal.deleted_at is not None

測試結果：通過

測試編號：UT-ANIMAL-013

測試目標：驗證管理員可以刪除任何動物

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 管理員刪除非自己的動物
2. 驗證刪除成功（即使 can_manage_animal 返回 False）

測試角色：平台管理員

測試流程：

- Mock 管理員（role=ADMIN）
- Mock animal.owner_id = 999（不是管理員的）
- Mock permission_service.can_manage_animal 返回 False
- 執行刪除操作
- 驗證 animal.deleted_at is not None

測試結果：通過

測試編號：UT-ANIMAL-014

測試目標：驗證取得動物基本資料

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 提供 animal_id
2. 呼叫 AnimalService.get_animal(animal_id)
3. 驗證返回動物對象

測試角色：使用者

測試流程：

- Mock Animal.query.filter_by(animal_id=1, deleted_at=None).first() 返回動物
- 呼叫 get_animal(1)
- 驗證返回的動物資料

測試結果：通過

測試編號：UT-ANIMAL-015

測試目標：驗證動物不存在拋出 `NotFoundError`

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 查詢不存在的 `animal_id`
2. 驗證拋出 `NotFoundError`

測試角色：使用者

測試流程：

- Mock query 返回 `None`
- 呼叫 `get_animal(999)`
- 預期拋出 `NotFoundError`

測試結果：通過

測試編號：UT-ANIMAL-016

測試目標：驗證排除已刪除動物

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 查詢時自動過濾 `deleted_at != None` 的動物
2. 驗證 `filter_by` 呼叫包含 `deleted_at=None`

測試角色：使用者

測試流程：

- 驗證查詢條件包含 `deleted_at=None`
- 確保已刪除動物不被返回

測試結果：通過

測試編號：UT-ANIMAL-017

測試目標：驗證訪客只看已發布動物

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 未登入用戶查詢動物列表
2. 驗證只返回 `status=PUBLISHED` 的動物

測試角色：訪客（未登入）

測試流程：

- 不提供 `current_user_id`
- 執行 `list_animals({'page': 1, 'per_page': 20})`
- 驗證查詢條件包含 `status=PUBLISHED`
- 驗證返回分頁結果

測試結果：通過

測試編號：UT-ANIMAL-018

測試目標：驗證依物種篩選

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 提供 species 參數
2. 驗證返回符合物種的動物

測試角色：使用者

測試流程：

- 提供 {'species': 'DOG', 'page': 1}
- 驗證查詢條件包含 species=DOG
- 驗證返回結果

測試結果：通過

測試編號：UT-ANIMAL-019

測試目標：驗證分頁功能正確運作

測試方法：邊界值分析 (Boundary Value Analysis)

測試步驟：

1. 提供 page 和 per_page 參數
2. 驗證分頁資訊正確

測試角色：使用者

測試流程：

- 提供 {'page': 2, 'per_page': 20}
- Mock pagination 返回 total=100, pages=5, page=2
- 驗證返回的 page 和 total 正確

測試結果：通過

測試編號：UT-ANIMAL-020

測試目標：驗證關鍵字搜尋功能

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 提供 q (關鍵字) 參數
2. 驗證搜尋 name 或 description 欄位

測試角色：使用者

測試流程：

- 提供 {'q': '小白', 'page': 1}
- 驗證查詢使用 filter 進行模糊搜尋
- 驗證返回匹配的動物

測試結果：通過

測試編號：UT-ANIMAL-021

測試目標：驗證來源類型篩選 - 收容所

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 提供 source_type='shelter'
2. 驗證只返回 shelter_id IS NOT NULL 的動物

測試角色：使用者

測試流程：

- 提供 {'source_type': 'shelter', 'page': 1}
- 驗證篩選條件
- 驗證返回收容所動物

測試結果：通過

測試編號：UT-ANIMAL-022

測試目標：驗證來源類型篩選 - 個人

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 提供 source_type='personal'
2. 驗證只返回 owner_id IS NOT NULL 的動物

測試角色：使用者

測試流程：

- 提供 {'source_type': 'personal', 'page': 1}
- 驗證篩選條件
- 驗證返回個人動物

測試結果：通過

測試編號：UT-ANIMAL-023

測試目標：驗證地區篩選

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 提供 region 參數
2. 驗證返回該地區的動物

測試角色：使用者

測試流程：

- 提供 {'region': '台北', 'page': 1}
- 驗證篩選邏輯
- 驗證返回結果

測試結果：通過

測試編號：UT-ANIMAL-024

測試目標：驗證年齡範圍篩選（MySQL）

測試方法：邊界值分析 (Boundary Value Analysis)

測試步驟：

1. 提供 min_age 和 max_age 參數
2. 驗證使用 TIMESTAMPDIFF 計算年齡（MySQL）

測試角色：使用者

測試流程：

- Mock db.engine.name = 'mysql'
- 提供 {'min_age': 6, 'max_age': 24, 'page': 1}
- 驗證年齡計算邏輯（月份）
- 驗證返回符合年齡範圍的動物

測試結果：通過

測試編號：UT-ANIMAL-025

測試目標：驗證年齡範圍篩選（SQLite）

測試方法：邊界值分析 (Boundary Value Analysis)

測試步驟：

1. 提供 min_age 參數
2. 驗證 SQLite 使用替代的日期計算方法

測試角色：使用者

測試流程：

- Mock db.engine.name = 'sqlite'
- 提供 {'min_age': 6, 'page': 1}
- 驗證使用 SQLite 相容的日期函數
- 驗證返回結果

測試結果：通過

測試編號：UT-ANIMAL-026

測試目標：驗證 owner_id 篩選（查詢自己）

測試方法：決策表測試 (Decision Table Testing)

測試步驟：

1. 用戶查詢自己的動物（owner_id=current_user_id）
2. 驗證可以看到自己所有狀態的動物

測試角色：一般會員

測試流程：

- Mock user (role=GENERAL_MEMBER)

- 提供 {'owner_id': 5, 'page': 1}, current_user_id=5
- 驗證不限制 status
- 驗證返回所有自己的動物

測試結果：通過

測試編號：UT-ANIMAL-027

測試目標：驗證收容所成員查詢 owner_id

測試方法：決策表測試 (Decision Table Testing)

測試步驟：

1. 收容所成員查詢特定 owner_id
2. 驗證權限處理

測試角色：收容所成員

測試流程：

- Mock user (role=SHELTER_MEMBER, primary_shelter_id=10)
- 提供 {'owner_id': 5, 'page': 1}, current_user_id=5
- 驗證查詢邏輯
- 驗證返回結果

測試結果：通過

測試編號：UT-ANIMAL-028

測試目標：驗證訪客查詢 owner_id（只看已發布）

測試方法：決策表測試 (Decision Table Testing)

測試步驟：

1. 未登入用戶查詢特定 owner_id 的動物
2. 驗證只返回 status=PUBLISHED 的動物

測試角色：訪客

測試流程：

- 提供 {'owner_id': 5, 'page': 1}, current_user_id=None
- 驗證強制 status=PUBLISHED
- 驗證返回公開動物

測試結果：通過

測試編號：UT-ANIMAL-029

測試目標：驗證查詢其他用戶的動物

測試方法：決策表測試 (Decision Table Testing)

測試步驟：

1. 用戶 A 查詢用戶 B 的動物
2. 驗證只能看到已發布的

測試角色：使用者

測試流程：

- 提供 {'owner_id': 5, 'page': 1}, current_user_id=10
- 驗證限制 status=PUBLISHED
- 驗證返回結果

測試結果： 通過

測試編號： UT-ANIMAL-030

測試目標：驗證 created_by 篩選

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 提供 created_by 參數
2. 驗證返回該用戶創建的動物

測試角色：使用者

測試流程：

- 提供 {'created_by': 10, 'page': 1}
- 驗證篩選條件
- 驗證返回結果

測試結果： 通過

測試編號： UT-ANIMAL-031

測試目標：驗證依收容所 ID 篩選

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 提供 shelter_id 參數
2. 驗證返回該收容所的動物

測試角色：使用者

測試流程：

- 提供 {'shelter_id': 10, 'page': 1}
- 驗證篩選條件
- 驗證返回結果

測試結果： 通過

測試編號： UT-ANIMAL-032

測試目標：驗證依性別篩選

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 提供 sex 參數
2. 驗證返回符合性別的動物

測試角色：使用者

測試流程：

- 提供 {'sex': 'MALE', 'page': 1}
- 驗證篩選條件
- 驗證返回雄性動物

測試結果： 通過

測試編號： UT-ANIMAL-033

測試目標：驗證提交審核功能

測試方法：狀態轉換測試 (State Transition Testing)

測試步驟：

1. 擁有者將動物從 DRAFT 提交審核
2. 呼叫 AnimalService.submit_for_review(animal_id, user)
3. 驗證狀態變更為 SUBMITTED

測試角色：動物擁有者

測試流程：

- Mock animal.status = AnimalStatus.DRAFT
- Mock permission_service.can_submit_animal_for_review 返回 True
- 執行 submit_for_review(100, user)
- 驗證 animal.status == AnimalStatus.SUBMITTED

測試結果： 通過

測試編號： UT-ANIMAL-034

測試目標：驗證發布動物功能

測試方法：狀態轉換測試 (State Transition Testing)

測試步驟：

1. 管理員將 SUBMITTED 動物發布
2. 呼叫 AnimalService.publish_animal(animal_id, admin)
3. 驗證狀態變更為 PUBLISHED

測試角色：平台管理員

測試流程：

- Mock animal.status = AnimalStatus.SUBMITTED
- Mock permission_service.can_publish_animal 返回 True
- 執行 publish_animal(100, admin)
- 驗證 animal.status == AnimalStatus.PUBLISHED

測試結果： 通過

測試編號： UT-ANIMAL-035

測試目標：驗證下架動物功能

測試方法：狀態轉換測試 (State Transition Testing)

測試步驟：

1. 擁有者下架 PUBLISHED 動物
2. 呼叫 AnimalService.retire_animal(animal_id, user)
3. 驗證狀態變更為 RETIRED

測試角色：動物擁有者

測試流程：

- Mock animal.status = AnimalStatus.PUBLISHED
- Mock permission_service.can_manage_animal 返回 True
- 執行 retire_animal(100, user)
- 驗證 animal.status == AnimalStatus.RETIRED

測試結果：通過

測試編號：UT-ANIMAL-036

測試目標：驗證拒絕動物並記錄原因

測試方法：狀態轉換測試 (State Transition Testing)

測試步驟：

1. 管理員拒絕 SUBMITTED 動物
2. 提供拒絕原因
3. 驗證狀態變回 DRAFT 且記錄原因

測試角色：平台管理員

測試流程：

- Mock animal.status = AnimalStatus.SUBMITTED
- Mock permission_service.can_reject_animal 返回 True
- 執行 reject_animal(100, admin, '資料不完整')
- 驗證 animal.status == AnimalStatus.DRAFT
- 驗證 animal.rejection_reason == '資料不完整'

測試結果：通過

測試編號：UT-ANIMAL-037

測試目標：驗證管理員可下架任何動物

測試方法：狀態轉換測試 (State Transition Testing)

測試步驟：

1. 管理員下架非自己的動物
2. 驗證下架成功（即使 can_manage_animal 返回 False）

測試角色：平台管理員

測試流程：

- Mock admin (role=ADMIN)
- Mock animal.owner_id = 999

- Mock `permission_service.can_manage_animal` 返回 `False`
- 執行 `retire_animal(100, admin)`
- 驗證 `animal.status == AnimalStatus.RETIRED`

測試結果：通過

測試編號：UT-ANIMAL-038

測試目標：驗證提交審核權限不足拋出錯誤

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 無權限用戶嘗試提交審核
2. 驗證拋出 `PermissionDeniedError`

測試角色：非擁有者

測試流程：

- Mock `permission_service.can_submit_animal_for_review` 返回 `False`
- 執行 `submit_for_review(100, user)`
- 預期拋出 `PermissionDeniedError`

測試結果：通過

測試編號：UT-ANIMAL-039

測試目標：驗證發布動物權限不足拋出錯誤

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 非管理員嘗試發布動物
2. 驗證拋出 `PermissionDeniedError`

測試角色：非管理員

測試流程：

- Mock `permission_service.can_publish_animal` 返回 `False`
- 執行 `publish_animal(100, user)`
- 預期拋出 `PermissionDeniedError`

測試結果：通過

測試編號：UT-ANIMAL-040

測試目標：驗證拒絕動物權限不足拋出錯誤

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 非管理員嘗試拒絕動物
2. 驗證拋出 `PermissionDeniedError`

測試角色：非管理員

測試流程：

- Mock `permission_service.can_reject_animal` 返回 `False`
- 執行 `reject_animal(100, user, '理由')`
- 預期拋出 `PermissionDeniedError`

測試結果：通過

測試編號：UT-ANIMAL-041

測試目標：驗證提交審核時狀態錯誤拋出 `ValidationError`

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 嘗試提交非 `DRAFT` 狀態的動物
2. 驗證拋出 `ValidationError`

測試角色：動物擁有者

測試流程：

- Mock `animal.status = AnimalStatus.PUBLISHED`
- 執行 `submit_for_review(100, user)`
- 預期拋出 `ValidationError('只能提交草稿狀態的動物')`

測試結果：通過

測試編號：UT-ANIMAL-042

測試目標：驗證發布已發布動物拋出 `ValidationError`

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 嘗試發布已經是 `PUBLISHED` 的動物
2. 驗證拋出 `ValidationError`

測試角色：平台管理員

測試流程：

- Mock `animal.status = AnimalStatus.PUBLISHED`
- 執行 `publish_animal(100, admin)`
- 預期拋出 `ValidationError('動物已經是發布狀態')`

測試結果：通過

測試編號：UT-ANIMAL-043

測試目標：驗證下架已下架動物拋出 `ValidationError`

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 嘗試下架已經是 `RETIRED` 的動物
2. 驗證拋出 `ValidationError`

測試角色：動物擁有者

測試流程：

- Mock animal.status = AnimalStatus.RETIRED
- 執行 retire_animal(100, user)
- 預期拋出 ValidationError('動物已經是下架狀態')

測試結果：通過

測試編號：UT-ANIMAL-044

測試目標：驗證拒絕非待審核動物拋出 ValidationError

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 嘗試拒絕非 SUBMITTED 狀態的動物
2. 驗證拋出 ValidationError

測試角色：平台管理員

測試流程：

- Mock animal.status = AnimalStatus.PUBLISHED
- 執行 reject_animal(100, admin, '理由')
- 預期拋出 ValidationError('只能拒絕待審核狀態的動物')

測試結果：通過

測試編號：UT-ANIMAL-045

測試目標：驗證拒絕動物未提供原因拋出 ValidationError

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 拒絕動物但提供空白原因
2. 驗證拋出 ValidationError

測試角色：平台管理員

測試流程：

- Mock animal.status = AnimalStatus.SUBMITTED
- 執行 reject_animal(100, admin, '') # 空白字串
- 預期拋出 ValidationError('請提供拒絕原因')

測試結果：通過

測試編號：UT-ANIMAL-046

測試目標：驗證新增圖片功能

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 擁有者上傳動物圖片
2. 呼叫 AnimalService.add_image(animal_id, user, key, url)
3. 驗證創建 AnimalImage 記錄

測試角色：動物擁有者

測試流程：

- Mock `permission_service.can_manage_animal` 返回 True
- Mock 圖片數量查詢返回 0
- 執行 `add_image(100, user, 'key', 'url')`
- 驗證 `AnimalImage` 類別被呼叫

測試結果：通過

測試編號：UT-ANIMAL-047

測試目標：驗證刪除圖片功能

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 擁有者刪除動物圖片
2. 呼叫 `AnimalService.delete_image(animal_id, image_id, user)`
3. 驗證圖片記錄被刪除

測試角色：動物擁有者

測試流程：

- Mock `permission_service.can_manage_animal` 返回 True
- Mock `AnimalImage.query` 返回圖片對象
- 執行 `delete_image(100, 50, user)`
- 驗證 `db.session.delete` 被呼叫

測試結果：通過

測試編號：UT-ANIMAL-048

測試目標：驗證重新排序圖片功能

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 擁有者調整圖片順序
2. 呼叫 `AnimalService.reorder_images(animal_id, user, orders)`
3. 驗證圖片 order 欄位更新

測試角色：動物擁有者

測試流程：

- Mock `permission_service.can_manage_animal` 返回 True
- Mock 兩張圖片對象 (`image1.order=1, image2.order=2`)
- 提供新順序 `[{'image_id': 2, 'order': 1}, {'image_id': 1, 'order': 2}]`
- 執行 `reorder_images(100, user, orders)`
- 驗證 `image1.order == 2` 且 `image2.order == 1`

測試結果：通過

測試編號：UT-ANIMAL-049

測試目標：驗證新增圖片權限不足拋出錯誤

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 無權限用戶嘗試新增圖片
2. 驗證拋出 `PermissionDeniedError`

測試角色：非擁有者

測試流程：

- Mock `permission_service.can_manage_animal` 返回 `False`
- 執行 `add_image(100, user, 'key', 'url')`
- 預期拋出 `PermissionDeniedError`

測試結果：通過

測試編號：UT-ANIMAL-050

測試目標：驗證刪除不存在的圖片拋出 `NotFoundError`

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 嘗試刪除不存在的圖片 ID
2. 驗證拋出 `NotFoundError`

測試角色：動物擁有者

測試流程：

- Mock `permission_service.can_manage_animal` 返回 `True`
- Mock `AnimalImage.query` 返回 `None`
- 執行 `delete_image(100, 50, user)`
- 預期拋出 `NotFoundError`

測試結果：通過

測試編號：UT-ANIMAL-051

測試目標：驗證新增圖片時動物不存在拋出 `NotFoundError`

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 為不存在的動物新增圖片
2. 驗證拋出 `NotFoundError`

測試角色：使用者

測試流程：

- Mock `Animal.query` 返回 `None`
- 執行 `add_image(999, user, 'key', 'url')`
- 預期拋出 `NotFoundError('動物不存在')`

測試結果：通過

測試編號：UT-ANIMAL-052

測試目標：驗證刪除圖片時動物不存在拋出 NotFoundError

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 為不存在的動物刪除圖片
2. 驗證拋出 NotFoundError

測試角色：使用者

測試流程：

- Mock Animal.query 返回 None
- 執行 delete_image(999, 50, user)
- 預期拋出 NotFoundError('動物不存在')

測試結果：通過

測試編號：UT-ANIMAL-053

測試目標：驗證重新排序圖片時動物不存在拋出 NotFoundError

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 為不存在的動物排序圖片
2. 驗證拋出 NotFoundError

測試角色：使用者

測試流程：

- Mock Animal.query 返回 None
- 執行 reorder_images(999, user, [])
- 預期拋出 NotFoundError('動物不存在')

測試結果：通過

測試編號：UT-ANIMAL-054

測試目標：驗證重新排序圖片權限不足拋出錯誤

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 無權限用戶嘗試排序圖片
2. 驗證拋出 PermissionDeniedError

測試角色：非擁有者

測試流程：

- Mock permission_service.can_manage_animal 返回 False
- 執行 reorder_images(100, user, [])
- 預期拋出 PermissionDeniedError

測試結果：通過

測試編號：UT-ANIMAL-055

測試目標：驗證無效物種值拋出 ValidationError

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 查詢時提供無效的 species 值
2. 驗證拋出 ValidationError

測試角色：使用者

測試流程：

- 提供 {'species': 'INVALID_SPECIES', 'page': 1}
- 預期拋出 ValidationError(match='無效的物種值')

測試結果：通過

測試編號：UT-ANIMAL-056

測試目標：驗證無效性別值拋出 ValidationError

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 查詢時提供無效的 sex 值
2. 驗證拋出 ValidationError

測試角色：使用者

測試流程：

- 提供 {'sex': 'INVALID_SEX', 'page': 1}
- 預期拋出 ValidationError(match='無效的性別值')

測試結果：通過

測試編號：UT-ANIMAL-057

測試目標：驗證無效狀態值拋出 ValidationError

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 查詢時提供無效的 status 值
2. 驗證拋出 ValidationError

測試角色：使用者

測試流程：

- 提供 {'status': 'INVALID_STATUS', 'page': 1}
- 預期拋出 ValidationError(match='無效的狀態值')

測試結果：通過

測試編號：UT-ANIMAL-058

測試目標：驗證更新動物時無效物種拋出錯誤

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 更新動物時提供無效的 species
2. 驗證拋出 ValidationError 或 ValueError

測試角色：動物擁有者

測試流程：

- Mock permission_service.can_manage_animal 返回 True
- 提供 {'species': 'INVALID'}
- 預期拋出 ValidationError 或 ValueError

測試結果：通過

測試編號：UT-ANIMAL-059

測試目標：驗證更新動物時無效性別拋出錯誤

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 更新動物時提供無效的 sex
2. 驗證拋出 ValidationError 或 ValueError

測試角色：動物擁有者

測試流程：

- Mock permission_service.can_manage_animal 返回 True
- 提供 {'sex': 'INVALID'}
- 預期拋出 ValidationError 或 ValueError

測試結果：通過

測試編號：UT-ANIMAL-060

測試目標：驗證更新動物時無效狀態拋出錯誤

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 更新動物時提供無效的 status
2. 驗證拋出 ValidationError 或 ValueError

測試角色：動物擁有者

測試流程：

- Mock permission_service.can_manage_animal 返回 True
- 提供 {'status': 'INVALID'}
- 預期拋出 ValidationError 或 ValueError

測試結果：通過

2.3 Application Service 單元測試計畫

Application Service 的所有單元測試案例，共計 39 個測試案例。

測試編號：UT-APPLICATIONSERVICE-001

測試目標：TC-01: 管理員可以看到所有申請

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ApplicationService.list_applications() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 執行 ApplicationService.list_applications()
- 驗證結果值正確

測試結果：通過

測試編號：UT-APPLICATIONSERVICE-002

測試目標：TC-02: 一般會員審核模式 (看自己動物的申請)

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ApplicationService.list_applications() 方法
3. 驗證結果符合預期

測試角色：收容所成員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-APPLICATIONSERVICE-003

測試目標：TC-03: 一般會員我的申請模式

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ApplicationService.list_applications() 方法
3. 驗證結果符合預期

測試角色：收容所成員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-APPLICATIONSERVICE-004

測試目標：TC-04: 收容所成員包含收容所動物

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 ApplicationService.list_applications() 方法
3. 驗證結果符合預期

測試角色：收容所成員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-APPLICATIONSERVICE-005

測試目標：TC-05: 依狀態篩選

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 ApplicationService.list_applications() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-APPLICATIONSERVICE-006

測試目標：TC-06: 無效狀態拋出錯誤

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 ApplicationService.list_applications() 方法
3. 確認拋出預期的異常

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-APPLICATIONSERVICE-007

測試目標：TC-07: 非管理員不能查詢其他用戶的申請

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 ApplicationService.list_applications() 方法
3. 確認拋出預期的異常

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-APPLICATIONSERVICE-008

測試目標：TC-08: 管理員可以查詢指定申請人

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 ApplicationService.list_applications() 方法

3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-APPLICATIONSERVICE-009

測試目標：TC-09: 成功創建申請

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 提供測試數據
3. 呼叫 ApplicationService.create_application() 方法
4. 驗證結果符合預期

測試角色：一般會員

測試流程：

- 提供測試數據
- 執行 ApplicationService.create_application()
- 驗證結果值正確

測試結果：通過

測試編號：UT-APPLICATIONSERVICE-010

測試目標：TC-10: 非一般會員不能申請

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ApplicationService.create_application() 方法
3. 確認拋出預期的異常

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-APPLICATIONSERVICE-011

測試目標： TC-11: 缺少 animal_id

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ApplicationService.create_application() 方法
3. 確認拋出預期的異常

測試角色： 一般會員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-APPLICATIONSERVICE-012

測試目標： TC-12: 動物不存在

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ApplicationService.create_application() 方法
3. 確認拋出預期的異常

測試角色： 一般會員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-APPLICATIONSERVICE-013

測試目標： TC-13: 不能申請已領養動物

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 ApplicationService.create_application() 方法
3. 確認拋出預期的異常

測試角色：一般會員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-APPLICATIONSERVICE-014

測試目標：TC-14: 不能申請未發布動物

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 ApplicationService.create_application() 方法
3. 確認拋出預期的異常

測試角色：一般會員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-APPLICATIONSERVICE-015

測試目標：TC-15: 不能申請自己的動物

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 ApplicationService.create_application() 方法
3. 確認拋出預期的異常

測試角色：一般會員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法

- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-APPLICATIONSERVICE-016

測試目標： TC-16: 重複申請拋出衝突錯誤

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ApplicationService.create_application() 方法
3. 確認拋出預期的異常

測試角色： 一般會員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-APPLICATIONSERVICE-017

測試目標： TC-17: 冪等性鍵值返回已存在的申請

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ApplicationService.create_application() 方法
3. 驗證結果符合預期

測試角色： 一般會員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-APPLICATIONSERVICE-018

測試目標： TC-18: 成功批准申請

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 ApplicationService.review_application() 方法
3. 驗證結果符合預期

測試角色：一般會員

測試流程：

- 執行 ApplicationService.review_application()
- 驗證結果值正確

測試結果：通過

測試編號：UT-APPLICATIONSERVICE-019

測試目標：TC-19: 成功拒絕申請

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 ApplicationService.review_application() 方法
3. 驗證結果符合預期

測試角色：一般會員

測試流程：

- 執行 ApplicationService.review_application()
- 驗證結果值正確

測試結果：通過

測試編號：UT-APPLICATIONSERVICE-020

測試目標：TC-20: 申請不存在

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 ApplicationService.review_application() 方法
3. 確認拋出預期的異常

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-APPLICATIONSERVICE-021

測試目標： TC-21: 無權限審核

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ApplicationService.review_application() 方法
3. 確認拋出預期的異常

測試角色： 一般會員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-APPLICATIONSERVICE-022

測試目標： TC-22: 管理員不能審核申請

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ApplicationService.review_application() 方法
3. 確認拋出預期的異常

測試角色： 平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-APPLICATIONSERVICE-023

測試目標： TC-23: 無效操作

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 `ApplicationService.review_application()` 方法
3. 確認拋出預期的異常

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-APPLICATIONSERVICE-024

測試目標：TC-24: 不能審核非待審核申請

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 `ApplicationService.review_application()` 方法
3. 確認拋出預期的異常

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-APPLICATIONSERVICE-025

測試目標：TC-25: 樂觀鎖衝突

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 `ApplicationService.review_application()` 方法
3. 確認拋出預期的異常

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法

- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-APPLICATIONSERVICE-026

測試目標： TC-26: 成功撤回申請

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ApplicationService.withdraw_application() 方法
3. 驗證結果符合預期

測試角色： 一般用戶

測試流程：

- 執行 ApplicationService.withdraw_application()
- 驗證結果值正確

測試結果： 通過

測試編號： UT-APPLICATIONSERVICE-027

測試目標： TC-27: 不能撤回其他人的申請

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ApplicationService.withdraw_application() 方法
3. 確認拋出預期的異常

測試角色： 一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-APPLICATIONSERVICE-028

測試目標： TC-28: 不能撤回已批准申請

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 ApplicationService.withdraw_application() 方法
3. 確認拋出預期的異常

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-APPLICATIONSERVICE-029

測試目標：TC-29: 管理員可以指派申請

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 ApplicationService.assign_application() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 執行 ApplicationService.assign_application()
- 驗證結果值正確

測試結果：通過

測試編號：UT-APPLICATIONSERVICE-030

測試目標：TC-30: 非管理員不能指派

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 ApplicationService.assign_application() 方法
3. 確認拋出預期的異常

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-APPLICATIONSERVICE-031

測試目標： TC-31: 指派給無效受理人

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ApplicationService.assign_application() 方法
3. 確認拋出預期的異常

測試角色： 平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-APPLICATIONSERVICE-032

測試目標： TC-32: 管理員查詢指定申請人的申請

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ApplicationService.list_applications() 方法
3. 驗證結果符合預期

測試角色： 平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-APPLICATIONSERVICE-033

測試目標： TC-33: 用 animal_id 過濾申請

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 ApplicationService.list_applications() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-APPLICATIONSERVICE-034

測試目標：TC-34: 動物有待審核申請時拋出衝突錯誤

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 ApplicationService.create_application() 方法
3. 確認拋出預期的異常

測試角色：一般會員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-APPLICATIONSERVICE-035

測試目標：TC-35: 通知失敗不影響申請創建

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 ApplicationService.create_application() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色：一般會員

測試流程：

- 執行 ApplicationService.create_application()

- 驗證結果值正確

測試結果：通過

測試編號：UT-APPLICATIONSERVICE-036

測試目標：TC-36: 通知失敗不影響審核

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 ApplicationService.review_application() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色：收容所成員

測試流程：

- 執行 ApplicationService.review_application()
- 驗證結果值正確

測試結果：通過

測試編號：UT-APPLICATIONSERVICE-037

測試目標：TC-37: 通知失敗不影響指派

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 ApplicationService.assign_application() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色：平台管理員

測試流程：

- 執行 ApplicationService.assign_application()
- 驗證結果值正確

測試結果：通過

測試編號：UT-APPLICATIONSERVICE-038

測試目標：TC-38: 審核通過且發送 email 通知（owner 類型）

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 ApplicationService.review_application() 方法
3. 驗證結果符合預期

測試角色：收容所成員

測試流程：

- 執行 ApplicationService.review_application()
- 驗證結果值正確

測試結果： 通過

測試編號： UT-APPLICATIONSERVICE-039

測試目標： TC-39: 審核通過且發送 email 通知（shelter 類型）

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 ApplicationService.review_application() 方法
3. 驗證結果符合預期

測試角色：收容所成員

測試流程：

- 執行 ApplicationService.review_application()
- 驗證結果值正確

測試結果： 通過

2.4 Attachment Service 單元測試計畫

AttachmentService 的所有單元測試案例，共計 11 個測試案例。

測試編號： UT-ATTACHMENTSERVICE-001

測試目標： TC-01: 成功創建附件

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 提供測試數據
3. 呼叫 AttachmentService.create_attachment() 方法

4. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 提供測試數據
- 執行 AttachmentService.create_attachment()

測試結果：通過

測試編號：UT-ATTACHMENTSERVICE-002

測試目標：TC-02: 缺少必填欄位

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 提供測試數據
3. 呼叫 AttachmentService.create_attachment() 方法
4. 確認拋出預期的異常

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-ATTACHMENTSERVICE-003

測試目標：TC-03: 檔案不存在於 MinIO

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 提供測試數據
3. 呼叫 AttachmentService.create_attachment() 方法
4. 確認拋出預期的異常

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-ATTACHMENTSERVICE-004

測試目標：TC-04: 創建不關聯實體的附件

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 提供測試數據
3. 呼叫 AttachmentService.create_attachment() 方法
4. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 提供測試數據
- 執行 AttachmentService.create_attachment()

測試結果：通過

測試編號：UT-ATTACHMENTSERVICE-005

測試目標：TC-05: 成功取得附件

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 AttachmentService.get_attachment() 方法
3. 驗證結果符合預期

測試角色：系統

測試流程：

- 執行 AttachmentService.get_attachment()
- 驗證結果值正確

測試結果：通過

測試編號：UT-ATTACHMENTSERVICE-006

測試目標：TC-06: 附件不存在

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）

2. 呼叫 AttachmentService.get_attachment() 方法
3. 確認拋出預期的異常

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-ATTACHMENTSERVICE-007

測試目標：TC-07: 上傳者刪除自己的附件

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AttachmentService.delete_attachment() 方法
3. 驗證結果符合預期

測試角色：一般會員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-ATTACHMENTSERVICE-008

測試目標：TC-08: 管理員刪除附件

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AttachmentService.delete_attachment() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-ATTACHMENTSERVICE-009

測試目標： TC-09: 非上傳者且非管理員不能刪除

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AttachmentService.delete_attachment() 方法
3. 確認拋出預期的異常

測試角色： 平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-ATTACHMENTSERVICE-010

測試目標： TC-10: 刪除不存在的附件

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AttachmentService.delete_attachment() 方法
3. 確認拋出預期的異常

測試角色： 一般會員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-ATTACHMENTSERVICE-011

測試目標： TC-11: 生成公開 URL

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 呼叫 AttachmentService.generate_public_url() 方法
2. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試數據
- 執行被測試的服務方法
- 驗證返回結果或異常

測試結果：通過

2.5 Audit Service 單元測試計畫

Audit Service 的所有單元測試案例，共計 13 個測試案例。

測試編號：UT-AUDITSERVICE-001

測試目標：TC-01: 完整參數記錄審計日誌

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AuditService.log() 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 執行 AuditService.log()
- 驗證結果值正確

測試結果：通過

測試編號：UT-AUDITSERVICE-002

測試目標：TC-02: 不立即提交的日誌記錄

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AuditService.log() 方法
3. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-AUDITSERVICE-003

測試目標： TC-03: 最小參數記錄（系統操作）

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 AuditService.log() 方法
3. 驗證結果符合預期

測試角色： 系統

測試流程：

- 執行 AuditService.log()
- 驗證結果值正確

測試結果： 通過

測試編號：UT-AUDITSERVICE-004

測試目標： TC-04: 異常處理不影響主流程

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 AuditService.log() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色： 系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-AUDITSERVICE-005

測試目標：TC-05: 記錄成功登入

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AuditService.log_login() 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 執行 AuditService.log_login()
- 驗證結果值正確

測試結果：通過

測試編號：UT-AUDITSERVICE-006

測試目標：TC-06: 記錄失敗登入

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AuditService.log_login() 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 執行 AuditService.log_login()
- 驗證結果值正確

測試結果：通過

測試編號：UT-AUDITSERVICE-007

測試目標：TC-07: 記錄申請審核

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AuditService.log_application_review() 方法
3. 驗證結果符合預期

測試角色：系統

測試流程：

- 執行 AuditService.log_application_review()

- 驗證結果值正確

測試結果：通過

測試編號：UT-AUDITSERVICE-008

測試目標：TC-08: 記錄動物發布

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 AuditService.log_animal_publish() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 執行 AuditService.log_animal_publish()
- 驗證結果值正確

測試結果：通過

測試編號：UT-AUDITSERVICE-009

測試目標：TC-09: 記錄收容所驗證

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 AuditService.log_shelter_verify() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 執行 AuditService.log_shelter_verify()
- 驗證結果值正確

測試結果：通過

測試編號：UT-AUDITSERVICE-010

測試目標：TC-10: 記錄收容所取消驗證

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）

2. 呼叫 `AuditService.log_shelter_verify()` 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 執行 `AuditService.log_shelter_verify()`
- 驗證結果值正確

測試結果：通過

測試編號：UT-AUDITSERVICE-011

測試目標：TC-11: 記錄用戶封禁

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 `AuditService.log_user_ban()` 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 執行 `AuditService.log_user_ban()`
- 驗證結果值正確

測試結果：通過

測試編號：UT-AUDITSERVICE-012

測試目標：TC-12: 記錄資料匯出

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 `AuditService.log_data_export()` 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 執行 `AuditService.log_data_export()`
- 驗證結果值正確

測試結果：通過

測試編號：UT-AUDITSERVICE-013

測試目標：TC-13: 記錄資料刪除

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AuditService.log_data_deletion() 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 執行 AuditService.log_data_deletion()
- 驗證結果值正確

測試結果：通過

2.6 Auth Service 單元測試計畫

Auth Service 的所有單元測試案例，共計 25 個測試案例。

測試編號：UT-AUTHSERVICE-001

測試目標：測試成功創建待註冊記錄

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AuthService.register_pending() 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-AUTHSERVICE-002

測試目標：測試 Email 已存在

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 AuthService.register_pending() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-AUTHSERVICE-003

測試目標：測試成功驗證註冊

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 AuthService.verify_registration() 方法
3. 驗證結果符合預期

測試角色：一般會員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-AUTHSERVICE-004

測試目標：測試找不到待註冊記錄

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 呼叫 AuthService.verify_registration() 方法
2. 驗證結果符合預期
3. 確認拋出預期的異常

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-AUTHSERVICE-005

測試目標： 測試驗證碼已過期

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AuthService.verify_registration() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色： 一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-AUTHSERVICE-006

測試目標： 測試驗證時 email 已被註冊

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AuthService.verify_registration() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色： 一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-AUTHSERVICE-007

測試目標：測試驗證碼錯誤達到最大次數

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 AuthService.verify_registration() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-AUTHSERVICE-008

測試目標：測試驗證註冊時指定角色

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 AuthService.verify_registration() 方法
3. 驗證結果符合預期

測試角色：收容所成員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-AUTHSERVICE-009

測試目標：測試成功重新發送驗證碼

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）

2. 呼叫 AuthService.resend_registration_code() 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-AUTHSERVICE-010

測試目標：測試重發次數超過限制

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AuthService.resend_registration_code() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-AUTHSERVICE-011

測試目標：測試成功登入

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AuthService.login() 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法

- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-AUTHSERVICE-012

測試目標： 測試用戶不存在

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AuthService.login() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色： 一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-AUTHSERVICE-013

測試目標： 測試 Email 未驗證

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AuthService.login() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色： 一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-AUTHSERVICE-014

測試目標：測試密碼錯誤時增加失敗次數

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AuthService.login() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-AUTHSERVICE-015

測試目標：測試失敗 5 次後鎖定帳號

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AuthService.login() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-AUTHSERVICE-016

測試目標：測試成功驗證 Email

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AuthService.verify_email() 方法

3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-AUTHSERVICE-017

測試目標：測試無效 token

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 呼叫 AuthService.verify_email() 方法
2. 驗證結果符合預期
3. 確認拋出預期的異常

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-AUTHSERVICE-018

測試目標：測試已驗證的用戶

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AuthService.verify_email() 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-AUTHSERVICE-019

測試目標：測試成功發送驗證郵件

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 AuthService.send_verification_email() 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-AUTHSERVICE-020

測試目標：測試用戶已驗證

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 AuthService.send_verification_email() 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-AUTHSERVICE-021

測試目標：測試成功請求密碼重置

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）

2. 呼叫 AuthService.request_password_reset() 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-AUTHSERVICE-022

測試目標：測試用戶不存在 - 為安全考量總是返回 True

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AuthService.request_password_reset() 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-AUTHSERVICE-023

測試目標：測試成功重置密碼

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 AuthService.reset_password() 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-AUTHSERVICE-024

測試目標： 測試無效 token

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 呼叫 AuthService.reset_password() 方法
2. 驗證結果符合預期
3. 確認拋出預期的異常

測試角色： 系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-AUTHSERVICE-025

測試目標： 測試密碼過短

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 呼叫 AuthService.reset_password() 方法
2. 驗證結果符合預期
3. 確認拋出預期的異常

測試角色： 系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

2.7 Email Service 單元測試計畫

Email Service 的所有單元測試案例，共計 17 個測試案例。

測試編號：UT-EMAILSERVICE-001

測試目標：測試成功透過 SMTP 發送郵件

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-EMAILSERVICE-002

測試目標：測試無 SMTP 設定時直接返回 True

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試數據
- 執行被測試的服務方法
- 驗證返回結果或異常

測試結果：通過

測試編號：UT-EMAILSERVICE-003

測試目標：測試使用 SSL 連接

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據

- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-EMAILSERVICE-004

測試目標：測試 SMTP 失敗時回退

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 驗證結果符合預期
2. 確認拋出預期的異常

測試角色：一般用戶

測試流程：

- 準備測試數據
- 執行被測試的服務方法
- 驗證返回結果或異常

測試結果： 通過

測試編號：UT-EMAILSERVICE-005

測試目標：測試發送 HTML 郵件

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-EMAILSERVICE-006

測試目標：測試 SMTP 登入失敗

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 驗證結果符合預期
3. 確認拋出預期的異常

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-EMAILSERVICE-007

測試目標：測試發送驗證郵件

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-EMAILSERVICE-008

測試目標：測試無 FRONTEND_URL 時使用 localhost 回退

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-EMAILSERVICE-009

測試目標：測試發送密碼重置郵件

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-EMAILSERVICE-010

測試目標：測試無 FRONTEND_URL 時使用 localhost 回退

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-EMAILSERVICE-011

測試目標：測試發送註冊驗證碼郵件

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-EMAILSERVICE-012

測試目標：測試簡單郵件發送

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-EMAILSERVICE-013

測試目標：測試包含動物資訊的郵件

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-EMAILSERVICE-014

測試目標：測試包含機構聯絡資訊的郵件

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-EMAILSERVICE-015

測試目標： 測試包含個人聯絡資訊的郵件

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 驗證結果符合預期

測試角色： 一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-EMAILSERVICE-016

測試目標： 測試使用模板成功發送郵件

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備查詢條件
2. 執行查詢操作
3. 驗證返回結果集
4. 確認數據正確過濾

測試角色： 系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-EMAILSERVICE-017

測試目標： 測試模板不存在時回退到簡單格式

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)

2. 驗證結果符合預期
3. 確認拋出預期的異常

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

2.8 Job Service 單元測試計畫

Job Service 的所有單元測試案例，共計 18 個測試案例。

測試編號：UT-JOBSERVICE-001

測試目標：TC-01: 管理員可以查看所有任務

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 JobService.list_jobs() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 執行 JobService.list_jobs()
- 驗證結果值正確

測試結果：通過

測試編號：UT-JOBSERVICE-002

測試目標：TC-02: 普通用戶只能看到自己的任務

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 JobService.list_jobs() 方法
3. 驗證結果符合預期

測試角色：一般會員

測試流程：

- 執行 `JobService.list_jobs()`
- 驗證結果值正確

測試結果：通過

測試編號：UT-JOBSERVICE-003

測試目標：TC-03: 按類型過濾任務

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 `JobService.list_jobs()` 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 執行 `JobService.list_jobs()`
- 驗證結果值正確

測試結果：通過

測試編號：UT-JOBSERVICE-004

測試目標：TC-04: 無效的狀態過濾

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 `JobService.list_jobs()` 方法
3. 確認拋出預期的異常

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-JOBSERVICE-005

測試目標：TC-04a: 管理員可以按創建者過濾

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 JobService.list_jobs() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 執行 JobService.list_jobs()
- 驗證結果值正確

測試結果：通過

測試編號：UT-JOBSERVICE-006

測試目標：TC-05: 管理員可以查看任何任務

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 JobService.get_job() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 執行 JobService.get_job()
- 驗證結果值正確

測試結果：通過

測試編號：UT-JOBSERVICE-007

測試目標：TC-06: 用戶可以查看自己的任務

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 JobService.get_job() 方法
3. 驗證結果符合預期

測試角色：一般會員

測試流程：

- 準備測試環境和測試數據

- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-JOBSERVICE-008

測試目標： TC-07: 用戶不能查看他人的任務

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 JobService.get_job() 方法
3. 確認拋出預期的異常

測試角色： 一般會員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-JOBSERVICE-009

測試目標： TC-08: 任務不存在

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 JobService.get_job() 方法
3. 確認拋出預期的異常

測試角色： 一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-JOBSERVICE-010

測試目標： TC-09: 成功重試失敗的任務

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 JobService.retry_job() 方法
3. 驗證結果符合預期

測試角色：一般會員

測試流程：

- 執行 JobService.retry_job()
- 驗證結果值正確

測試結果：通過

測試編號：UT-JOBSERVICE-011

測試目標：TC-10: 不能重試非失敗的任務

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 JobService.retry_job() 方法
3. 確認拋出預期的異常

測試角色：一般會員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-JOBSERVICE-012

測試目標：TC-11: 成功取消待處理的任務

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 JobService.cancel_job() 方法
3. 驗證結果符合預期

測試角色：一般會員

測試流程：

- 執行 JobService.cancel_job()

- 驗證結果值正確

測試結果：通過

測試編號：UT-JOBSERVICE-013

測試目標：TC-12: 不能取消已完成的任務

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 JobService.cancel_job() 方法
3. 確認拋出預期的異常

測試角色：一般會員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-JOBSERVICE-014

測試目標：TC-13: 管理員成功核准任務

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 JobService.approve_job() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 執行 JobService.approve_job()
- 驗證結果值正確

測試結果：通過

測試編號：UT-JOBSERVICE-015

測試目標：TC-14: 非管理員不能核准任務

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 JobService.approve_job() 方法
3. 確認拋出預期的異常

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-JOBSERVICE-016

測試目標：TC-15: 不能核准非待處理的任務

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 JobService.approve_job() 方法
3. 確認拋出預期的異常

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-JOBSERVICE-017

測試目標：TC-16: 管理員成功拒絕任務

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 JobService.reject_job() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 執行 JobService.reject_job()
- 驗證結果值正確

測試結果： 通過

測試編號： UT-JOBSERVICE-018

測試目標： TC-17: 非管理員不能拒絕任務

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 JobService.reject_job() 方法
3. 確認拋出預期的異常

測試角色： 平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

2.9 Medical Record Service 單元測試計畫

Medical Record Service 的所有單元測試案例，共計 31 個測試案例。

測試編號： UT-MEDICALRECORDSERVICE-001

測試目標： 測試管理員可以看到所有動物

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 MedicalRecordService.list_animals_for_medical_records() 方法
3. 驗證結果符合預期

測試角色： 平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-MEDICALRECORDSERVICE-002

測試目標： 測試收容所成員可以看到自己的和收容所的動物

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 MedicalRecordService.list_animals_for_medical_records() 方法
3. 驗證結果符合預期

測試角色： 收容所成員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-MEDICALRECORDSERVICE-003

測試目標： 測試一般用戶只能看到自己的動物

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 MedicalRecordService.list_animals_for_medical_records() 方法
3. 驗證結果符合預期

測試角色： 收容所成員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-MEDICALRECORDSERVICE-004

測試目標： 測試管理員可以創建醫療記錄

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 MedicalRecordService.create_medical_record() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-MEDICALRECORDSERVICE-005

測試目標：測試動物不存在時拋出異常

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 呼叫 MedicalRecordService.create_medical_record() 方法
2. 驗證結果符合預期
3. 確認拋出預期的異常

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-MEDICALRECORDSERVICE-006

測試目標：測試用戶不存在時拋出異常

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 MedicalRecordService.create_medical_record() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據

- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-MEDICALRECORDSERVICE-007

測試目標： 測試無權限用戶創建醫療記錄時拋出異常

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 MedicalRecordService.create_medical_record() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色： 收容所成員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-MEDICALRECORDSERVICE-008

測試目標： 測試創建者可以在 24 小時內更新

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 MedicalRecordService.update_medical_record() 方法
3. 驗證結果符合預期

測試角色： 收容所成員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-MEDICALRECORDSERVICE-009

測試目標：測試記錄不存在時拋出異常

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 呼叫 MedicalRecordService.update_medical_record() 方法
2. 驗證結果符合預期
3. 確認拋出預期的異常

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-MEDICALRECORDSERVICE-010

測試目標：測試管理員不能直接編輯醫療記錄

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 MedicalRecordService.update_medical_record() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-MEDICALRECORDSERVICE-011

測試目標：測試超過 24 小時後無法更新

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 MedicalRecordService.update_medical_record() 方法
3. 驗證結果符合預期

4. 確認拋出預期的異常

測試角色：收容所成員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-MEDICALRECORDSERVICE-012

測試目標：測試管理員可以驗證醫療記錄

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 MedicalRecordService.verify_medical_record() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-MEDICALRECORDSERVICE-013

測試目標：測試非管理員無法驗證醫療記錄

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 MedicalRecordService.verify_medical_record() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-MEDICALRECORDSERVICE-014

測試目標：測試解析 YYYY-MM-DD 格式

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 呼叫 MedicalRecordService.parse_date() 方法
2. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-MEDICALRECORDSERVICE-015

測試目標：測試解析 YYYY/MM/DD 格式

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 呼叫 MedicalRecordService.parse_date() 方法
2. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-MEDICALRECORDSERVICE-016

測試目標：測試解析 DD/MM/YYYY 格式

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 呼叫 MedicalRecordService.parse_date() 方法
2. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-MEDICALRECORDSERVICE-017

測試目標：測試解析 DD-MM-YYYY 格式

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 呼叫 MedicalRecordService.parse_date() 方法
2. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-MEDICALRECORDSERVICE-018

測試目標：測試解析帶有空白的日期

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 呼叫 MedicalRecordService.parse_date() 方法
2. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-MEDICALRECORDSERVICE-019

測試目標：測試無效日期格式拋出異常

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 呼叫 MedicalRecordService.parse_date() 方法
2. 驗證結果符合預期
3. 確認拋出預期的異常

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-MEDICALRECORDSERVICE-020

測試目標：測試成功列出動物的醫療記錄

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 MedicalRecordService.list_animal_medical_records() 方法
3. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-MEDICALRECORDSERVICE-021

測試目標：測試動物不存在時拋出異常

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 呼叫 MedicalRecordService.list_animal_medical_records() 方法
2. 驗證結果符合預期
3. 確認拋出預期的異常

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-MEDICALRECORDSERVICE-022

測試目標：測試成功獲取醫療記錄

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 MedicalRecordService.get_medical_record() 方法
3. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-MEDICALRECORDSERVICE-023

測試目標：測試記錄不存在時拋出異常

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 呼叫 MedicalRecordService.get_medical_record() 方法
2. 驗證結果符合預期
3. 確認拋出預期的異常

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-MEDICALRECORDSERVICE-024

測試目標：測試按名稱過濾

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 MedicalRecordService.list_animals_for_medical_records() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-MEDICALRECORDSERVICE-025

測試目標：測試按品種過濾

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 MedicalRecordService.list_animals_for_medical_records() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-MEDICALRECORDSERVICE-026

測試目標：測試按物種過濾

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 MedicalRecordService.list_animals_for_medical_records() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-MEDICALRECORDSERVICE-027

測試目標：測試按年齡範圍過濾

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 MedicalRecordService.list_animals_for_medical_records() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-MEDICALRECORDSERVICE-028

測試目標：測試過濾已領養動物

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備要刪除的對象
2. 執行刪除操作
3. 驗證刪除標記或實際刪除
4. 確認數據庫操作

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-MEDICALRECORDSERVICE-029

測試目標：測試過濾未領養動物

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備要刪除的對象
2. 執行刪除操作
3. 驗證刪除標記或實際刪除
4. 確認數據庫操作

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-MEDICALRECORDSERVICE-030

測試目標：測試創建醫療紀錄並添加附件

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 MedicalRecordService.create_medical_record() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-MEDICALRECORDSERVICE-031

測試目標：測試更新醫療紀錄並添加新附件

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）

2. 呼叫 MedicalRecordService.update_medical_record() 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

2.10 Notification Service 單元測試計劃

Notification Service 的所有單元測試案例，共計 21 個測試案例。

測試編號：UT-NOTIFICATIONSERVICE-001

測試目標：Test successfully creating notification

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 NotificationService.create() 方法
3. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-NOTIFICATIONSERVICE-002

測試目標：Test creating notification without commit

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)

2. 呼叫 NotificationService.create() 方法
3. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-NOTIFICATIONSERVICE-003

測試目標：Test notification when animal owner exists

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 NotificationService.notify_application_submitted() 方法
3. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-NOTIFICATIONSERVICE-004

測試目標：Test notification when animal belongs to shelter

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 NotificationService.notify_application_submitted() 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法

- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-NOTIFICATIONSERVICE-005

測試目標：Test no notification when animal doesn't exist

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 NotificationService.notify_application_submitted() 方法
3. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-NOTIFICATIONSERVICE-006

測試目標：Test notification when application is approved

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 NotificationService.notify_application_reviewed() 方法
3. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-NOTIFICATIONSERVICE-007

測試目標：Test listing all notifications

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 NotificationService.list_notifications() 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-NOTIFICATIONSERVICE-008

測試目標：Test listing unread notifications only

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 NotificationService.list_notifications() 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-NOTIFICATIONSERVICE-009

測試目標：Test getting unread count

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 NotificationService.get_unread_count() 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-NOTIFICATIONSERVICE-010

測試目標： Test marking notification as read

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 NotificationService.mark_as_read() 方法
3. 驗證結果符合預期

測試角色： 一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-NOTIFICATIONSERVICE-011

測試目標： Test marking non-existent notification

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 呼叫 NotificationService.mark_as_read() 方法
2. 驗證結果符合預期
3. 確認拋出預期的異常

測試角色： 一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-NOTIFICATIONSERVICE-012

測試目標：Test permission denied when marking others notification

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 呼叫 NotificationService.mark_as_read() 方法
2. 驗證結果符合預期
3. 確認拋出預期的異常

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-NOTIFICATIONSERVICE-013

測試目標：Test marking all notifications as read

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 NotificationService.mark_all_as_read() 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-NOTIFICATIONSERVICE-014

測試目標：Test successfully deleting notification

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)

2. 呼叫 NotificationService.delete_notification() 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-NOTIFICATIONSERVICE-015

測試目標：Test deleting non-existent notification

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 呼叫 NotificationService.delete_notification() 方法
2. 驗證結果符合預期
3. 確認拋出預期的異常

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-NOTIFICATIONSERVICE-016

測試目標：Test notifying animal owner of status change

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 NotificationService.notify_animal_status_changed() 方法
3. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法

- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-NOTIFICATIONSERVICE-017

測試目標：Test notifying shelter of animal status change

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 NotificationService.notify_animal_status_changed() 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-NOTIFICATIONSERVICE-018

測試目標：Test notifying job creator of completion

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 NotificationService.notify_job_completed() 方法
3. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-NOTIFICATIONSERVICE-019

測試目標：Test no notification when job has no creator

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 NotificationService.notify_job_completed() 方法
3. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-NOTIFICATIONSERVICE-020

測試目標：Test sending system notification

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 NotificationService.notify_system() 方法
3. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-NOTIFICATIONSERVICE-021

測試目標：Test notifying applicant that application is under review

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 NotificationService.notify_application_under_review() 方法
3. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

2.11 Permission Service 單元測試計劃

Permission Service 的所有單元測試案例，共計 26 個測試案例。

測試編號： UT-PERMISSIONSERVICE-001

測試目標： TC-01: 擁有者可以管理個人送養動物

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 PermissionService.can_manage_animal() 方法
3. 驗證結果符合預期

測試角色： 收容所成員

測試流程：

- 準備測試數據
- 執行被測試的服務方法
- 驗證返回結果或異常

測試結果： 通過

測試編號： UT-PERMISSIONSERVICE-002

測試目標： TC-02: 非擁有者不能管理個人動物

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 PermissionService.can_manage_animal() 方法
3. 驗證結果符合預期

測試角色： 收容所成員

測試流程：

- 準備測試數據
- 執行被測試的服務方法
- 驗證返回結果或異常

測試結果： 通過

測試編號： UT-PERMISSIONSERVICE-003

測試目標： TC-03: 收容所成員可以管理自己收容所的動物

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 `PermissionService.can_manage_animal()` 方法
3. 驗證結果符合預期

測試角色： 收容所成員

測試流程：

- 準備測試數據
- 執行被測試的服務方法
- 驗證返回結果或異常

測試結果： 通過

測試編號： UT-PERMISSIONSERVICE-004

測試目標： TC-04: 收容所成員不能管理其他收容所動物

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 `PermissionService.can_manage_animal()` 方法
3. 驗證結果符合預期

測試角色： 收容所成員

測試流程：

- 準備測試數據
- 執行被測試的服務方法
- 驗證返回結果或異常

測試結果： 通過

測試編號：UT-PERMISSIONSERVICE-005

測試目標：TC-05: 管理員可以管理收容所動物

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 `PermissionService.can_manage_animal()` 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 準備測試數據
- 執行被測試的服務方法
- 驗證返回結果或異常

測試結果：通過

測試編號：UT-PERMISSIONSERVICE-006

測試目標：TC-06: 管理員不能管理個人動物

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 `PermissionService.can_manage_animal()` 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 準備測試數據
- 執行被測試的服務方法
- 驗證返回結果或異常

測試結果：通過

測試編號：UT-PERMISSIONSERVICE-007

測試目標：TC-06a: None 用戶不能管理動物

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)

2. 呼叫 `PermissionService.can_manage_animal()` 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試數據
- 執行被測試的服務方法
- 驗證返回結果或異常

測試結果：通過

測試編號：UT-PERMISSIONSERVICE-008

測試目標：TC-06b: 用戶不能管理 None 動物

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 `PermissionService.can_manage_animal()` 方法
3. 驗證結果符合預期

測試角色：一般會員

測試流程：

- 準備測試數據
- 執行被測試的服務方法
- 驗證返回結果或異常

測試結果：通過

測試編號：UT-PERMISSIONSERVICE-009

測試目標：TC-07: 任何人可以查看已發布的動物

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 `PermissionService.can_view_animal()` 方法
3. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試數據
- 執行被測試的服務方法

- 驗證返回結果或異常

測試結果：通過

測試編號：UT-PERMISSIONSERVICE-010

測試目標：TC-08: 未登入用戶不能查看草稿動物

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 `PermissionService.can_view_animal()` 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試數據
- 執行被測試的服務方法
- 驗證返回結果或異常

測試結果：通過

測試編號：UT-PERMISSIONSERVICE-011

測試目標：TC-09: 擁有者可以查看自己的草稿

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 `PermissionService.can_view_animal()` 方法
3. 驗證結果符合預期

測試角色：收容所成員

測試流程：

- 準備測試數據
- 執行被測試的服務方法
- 驗證返回結果或異常

測試結果：通過

測試編號：UT-PERMISSIONSERVICE-012

測試目標：TC-10: 擁有者可以審核申請

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 PermissionService.can_review_application() 方法
3. 驗證結果符合預期

測試角色：收容所成員

測試流程：

- 準備測試數據
- 執行被測試的服務方法
- 驗證返回結果或異常

測試結果：通過

測試編號：UT-PERMISSIONSERVICE-013

測試目標：TC-11: 收容所成員可以審核收容所申請

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 PermissionService.can_review_application() 方法
3. 驗證結果符合預期

測試角色：收容所成員

測試流程：

- 準備測試數據
- 執行被測試的服務方法
- 驗證返回結果或異常

測試結果：通過

測試編號：UT-PERMISSIONSERVICE-014

測試目標：TC-12: 非擁有者不能審核申請

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 PermissionService.can_review_application() 方法
3. 驗證結果符合預期

測試角色：收容所成員

測試流程：

- 準備測試數據
- 執行被測試的服務方法
- 驗證返回結果或異常

測試結果： 通過

測試編號：UT-PERMISSIONSERVICE-015

測試目標： TC-12a: None application 不能審核

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 PermissionService.can_review_application() 方法
3. 驗證結果符合預期

測試角色： 一般會員

測試流程：

- 準備測試數據
- 執行被測試的服務方法
- 驗證返回結果或異常

測試結果： 通過

測試編號：UT-PERMISSIONSERVICE-016

測試目標： TC-12b: 沒有動物的申請不能審核

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 PermissionService.can_review_application() 方法
3. 驗證結果符合預期

測試角色： 一般會員

測試流程：

- 準備測試數據
- 執行被測試的服務方法
- 驗證返回結果或異常

測試結果： 通過

測試編號：UT-PERMISSIONSERVICE-017

測試目標：TC-13: 申請人可以查看自己的申請

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 `PermissionService.can_view_application()` 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試數據
- 執行被測試的服務方法
- 驗證返回結果或異常

測試結果：通過

測試編號：UT-PERMISSIONSERVICE-018

測試目標：TC-14: 動物擁有者可以查看申請

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 `PermissionService.can_view_application()` 方法
3. 驗證結果符合預期

測試角色：收容所成員

測試流程：

- 準備測試數據
- 執行被測試的服務方法
- 驗證返回結果或異常

測試結果： 通過

測試編號： UT-PERMISSIONSERVICE-019

測試目標： TC-15: 管理員可以查看任何申請

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 PermissionService.can_view_application() 方法
3. 驗證結果符合預期

測試角色： 平台管理員

測試流程：

- 準備測試數據
- 執行被測試的服務方法
- 驗證返回結果或異常

測試結果： 通過

測試編號： UT-PERMISSIONSERVICE-020

測試目標： TC-15a: None application 不能查看

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 PermissionService.can_view_application() 方法
3. 驗證結果符合預期

測試角色： 一般會員

測試流程：

- 準備測試數據
- 執行被測試的服務方法
- 驗證返回結果或異常

測試結果： 通過

測試編號： UT-PERMISSIONSERVICE-021

測試目標： TC-15b: 非申請人且無動物的申請不能查看

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 `PermissionService.can_view_application()` 方法
3. 驗證結果符合預期

測試角色：一般會員

測試流程：

- 準備測試數據
- 執行被測試的服務方法
- 驗證返回結果或異常

測試結果：通過

測試編號：UT-PERMISSIONSERVICE-022

測試目標：TC-16: 擁有者可以提交草稿動物

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 `PermissionService.can_submit_animal_for_review()` 方法
3. 驗證結果符合預期

測試角色：收容所成員

測試流程：

- 準備測試數據
- 執行被測試的服務方法
- 驗證返回結果或異常

測試結果：通過

測試編號：UT-PERMISSIONSERVICE-023

測試目標：TC-17: 管理員可以發布待審核動物

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 `PermissionService.can_publish_animal()` 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 準備測試數據
- 執行被測試的服務方法
- 驗證返回結果或異常

測試結果： 通過

測試編號：UT-PERMISSIONSERVICE-024

測試目標： TC-18: 非管理員不能發布動物

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 PermissionService.can_publish_animal() 方法
3. 驗證結果符合預期

測試角色： 平台管理員

測試流程：

- 準備測試數據
- 執行被測試的服務方法
- 驗證返回結果或異常

測試結果： 通過

測試編號：UT-PERMISSIONSERVICE-025

測試目標： TC-19: 管理員可以拒絕待審核動物

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 PermissionService.can_reject_animal() 方法
3. 驗證結果符合預期

測試角色： 平台管理員

測試流程：

- 準備測試數據
- 執行被測試的服務方法
- 驗證返回結果或異常

測試結果： 通過

測試編號：UT-PERMISSIONSERVICE-026

測試目標：TC-20: 非管理員不能拒絕動物

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 `PermissionService.can_reject_animal()` 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 準備測試數據
- 執行被測試的服務方法
- 驗證返回結果或異常

測試結果：通過

2.12 Shelter Service 單元測試計劃

Shelter Service 的所有單元測試案例，共計 45 個測試案例。

測試編號：UT-SHELTERSERVICE-001

測試目標：測試管理員有完全權限

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 `ShelterService.check_shelter_permission()` 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-SHELTERSERVICE-002

測試目標：測試收容所成員對自己收容所有權限

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ShelterService.check_shelter_permission() 方法
3. 驗證結果符合預期

測試角色：收容所成員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-SHELTERSERVICE-003

測試目標：測試收容所成員對其他收容所沒有權限

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ShelterService.check_shelter_permission() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色：收容所成員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-SHELTERSERVICE-004

測試目標：測試用戶不存在

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 呼叫 ShelterService.check_shelter_permission() 方法
2. 驗證結果符合預期
3. 確認拋出預期的異常

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-SHELTERSERVICE-005

測試目標：測試普通用戶沒有權限

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ShelterService.check_shelter_permission() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色：收容所成員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-SHELTERSERVICE-006

測試目標：測試成功創建收容所

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ShelterService.create_shelter() 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-SHELTERSERVICE-007

測試目標： 測試無效的 Email 格式

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 呼叫 ShelterService.create_shelter() 方法
2. 驗證結果符合預期
3. 確認拋出預期的異常

測試角色： 一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-SHELTERSERVICE-008

測試目標： 測試地址缺少必填欄位

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 呼叫 ShelterService.create_shelter() 方法
2. 驗證結果符合預期
3. 確認拋出預期的異常

測試角色： 一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-SHELTERSERVICE-009

測試目標：測試 Slug 已存在

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ShelterService.create_shelter() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-SHELTERSERVICE-010

測試目標：測試地址不是字典類型

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 呼叫 ShelterService.create_shelter() 方法
2. 驗證結果符合預期
3. 確認拋出預期的異常

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-SHELTERSERVICE-011

測試目標：測試成功更新收容所

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 ShelterService.update_shelter() 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-SHELTERSERVICE-012

測試目標：測試收容所不存在

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 ShelterService.update_shelter() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-SHELTERSERVICE-013

測試目標：測試更新無效的 Email

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 ShelterService.update_shelter() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-SHELTERSERVICE-014

測試目標：測試更新重複的 Slug

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ShelterService.update_shelter() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-SHELTERSERVICE-015

測試目標：測試管理員成功驗證收容所

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ShelterService.verify_shelter() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-SHELTERSERVICE-016

測試目標：測試非管理員無法驗證

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ShelterService.verify_shelter() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-SHELTERSERVICE-017

測試目標：測試收容所不存在

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ShelterService.verify_shelter() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-SHELTERSERVICE-018

測試目標：測試成功驗證 CSV

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 呼叫 ShelterService.validate_csv_file() 方法
2. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試數據
- 執行被測試的服務方法
- 驗證返回結果或異常

測試結果：通過

測試編號：UT-SHELTERSERVICE-019

測試目標：測試未選擇檔案

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 呼叫 ShelterService.validate_csv_file() 方法
2. 驗證結果符合預期
3. 確認拋出預期的異常

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-SHELTERSERVICE-020

測試目標：測試錯誤的檔案格式

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 呼叫 ShelterService.validate_csv_file() 方法
2. 驗證結果符合預期
3. 確認拋出預期的異常

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-SHELTERSERVICE-021

測試目標： 測試檔案過大

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 呼叫 ShelterService.validate_csv_file() 方法
2. 驗證結果符合預期
3. 確認拋出預期的異常

測試角色： 系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-SHELTERSERVICE-022

測試目標： 測試空的 CSV 檔案

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 呼叫 ShelterService.validate_csv_file() 方法
2. 驗證結果符合預期
3. 確認拋出預期的異常

測試角色： 系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-SHELTERSERVICE-023

測試目標：測試無效的編碼

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 呼叫 ShelterService.validate_csv_file() 方法
2. 驗證結果符合預期
3. 確認拋出預期的異常

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-SHELTERSERVICE-024

測試目標：測試成功列出收容所

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ShelterService.list_shelters() 方法
3. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-SHELTERSERVICE-025

測試目標：測試搜尋收容所

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)

2. 呼叫 ShelterService.list_shelters() 方法
3. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-SHELTERSERVICE-026

測試目標：測試只列出已驗證的收容所

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ShelterService.list_shelters() 方法
3. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-SHELTERSERVICE-027

測試目標：測試成功獲取收容所

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ShelterService.get_shelter() 方法
3. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法

- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-SHELTERSERVICE-028

測試目標：測試收容所不存在

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 ShelterService.get_shelter() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-SHELTERSERVICE-029

測試目標：測試成功批次更新動物狀態

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 ShelterService.batch_update_animal_status() 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-SHELTERSERVICE-030

測試目標：測試收容所不存在

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ShelterService.batch_update_animal_status() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-SHELTERSERVICE-031

測試目標：測試無效的操作類型

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ShelterService.batch_update_animal_status() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-SHELTERSERVICE-032

測試目標：測試動物不存在

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)

2. 呼叫 ShelterService.batch_update_animal_status() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-SHELTERSERVICE-033

測試目標：測試成功上傳檔案

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 ShelterService.upload_file_to_minio() 方法
3. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-SHELTERSERVICE-034

測試目標：測試不支援的檔案類型

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 ShelterService.upload_file_to_minio() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-SHELTERSERVICE-035

測試目標： 測試檔案過大

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ShelterService.upload_file_to_minio() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色： 系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-SHELTERSERVICE-036

測試目標： 測試 MinIO 上傳失敗

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ShelterService.upload_file_to_minio() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色： 系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-SHELTERSERVICE-037

測試目標：測試成功創建批次匯入任務

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ShelterService.create_batch_import_job() 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-SHELTERSERVICE-038

測試目標：測試收容所不存在

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ShelterService.create_batch_import_job() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-SHELTERSERVICE-039

測試目標：測試批次轉為草稿

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 ShelterService.batch_update_animal_status() 方法
3. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-SHELTERSERVICE-040

測試目標：測試批次提交審核

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 ShelterService.batch_update_animal_status() 方法
3. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-SHELTERSERVICE-041

測試目標：測試批次下架

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境（Mock 必要的依賴對象）
2. 呼叫 ShelterService.batch_update_animal_status() 方法
3. 驗證結果符合預期

測試角色：系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-SHELTERSERVICE-042

測試目標： 測試無效的狀態轉換

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ShelterService.batch_update_animal_status() 方法
3. 驗證結果符合預期

測試角色： 系統

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-SHELTERSERVICE-043

測試目標： 測試收容所成員可以創建新收容所 (無 shelter_id)

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ShelterService.check_shelter_permission() 方法
3. 驗證結果符合預期

測試角色： 收容所成員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-SHELTERSERVICE-044

測試目標：測試收容所主要負責人有權限

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ShelterService.check_shelter_permission() 方法
3. 驗證結果符合預期

測試角色：收容所成員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-SHELTERSERVICE-045

測試目標：測試更新時地址缺少必填欄位

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 ShelterService.update_shelter() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

2.13 User Service 單元測試

User Service 的所有單元測試案例，共計 24 個測試案例。

測試編號：UT-USERSERVICE-001

測試目標：Test getting own user includes sensitive info

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 UserService.get_user() 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-USERSERVICE-002

測試目標：Test admin can view others sensitive info

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 UserService.get_user() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-USERSERVICE-003

測試目標：Test regular user cannot view others sensitive info

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 UserService.get_user() 方法
3. 驗證結果符合預期

測試角色：一般會員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-USERSERVICE-004

測試目標： Test user not found raises exception

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 呼叫 UserService.get_user() 方法
2. 驗證結果符合預期
3. 確認拋出預期的異常

測試角色： 一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-USERSERVICE-005

測試目標： Test user can update own info

測試方法： 等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 提供測試數據
3. 呼叫 UserService.update_user() 方法
4. 驗證結果符合預期

測試角色： 一般用戶

測試流程：

- 提供測試數據
- 執行 UserService.update_user()

測試結果： 通過

測試編號：UT-USERSERVICE-006

測試目標：Test admin can update user role

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 提供測試數據
3. 呼叫 UserService.update_user() 方法
4. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 提供測試數據
- 執行 UserService.update_user()

測試結果：通過

測試編號：UT-USERSERVICE-007

測試目標：Test regular user cannot update role

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 提供測試數據
3. 呼叫 UserService.update_user() 方法
4. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 提供測試數據
- 執行 UserService.update_user()

測試結果：通過

測試編號：UT-USERSERVICE-008

測試目標：Test updating email requires reverification

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 提供測試數據
3. 呼叫 UserService.update_user() 方法
4. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 提供測試數據
- 執行 UserService.update_user()

測試結果：通過

測試編號：UT-USERSERVICE-009

測試目標：Test duplicate email raises conflict

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 提供測試數據
3. 呼叫 UserService.update_user() 方法
4. 驗證結果符合預期
5. 確認拋出預期的異常

測試角色：一般用戶

測試流程：

- 驗證拋出 ConflictError 異常
- 提供測試數據

測試結果：通過

測試編號：UT-USERSERVICE-010

測試目標：Test unauthorized update raises permission denied

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 提供測試數據
3. 呼叫 UserService.update_user() 方法
4. 驗證結果符合預期
5. 確認拋出預期的異常

測試角色：一般會員

測試流程：

- 驗證拋出 PermissionDeniedError 異常
- 提供測試數據

測試結果：通過

測試編號：UT-USERSERVICE-011

測試目標：Test successfully changing password

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 UserService.change_password() 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-USERSERVICE-012

測試目標：Test cannot change others password

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 呼叫 UserService.change_password() 方法
2. 驗證結果符合預期
3. 確認拋出預期的異常

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-USERSERVICE-013

測試目標：Test wrong old password raises validation error

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)

2. 呼叫 UserService.change_password() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-USERSERVICE-014

測試目標：Test short new password raises validation error

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 UserService.change_password() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-USERSERVICE-015

測試目標：Test successfully requesting data export

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 UserService.request_data_export() 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據

- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-USERSERVICE-016

測試目標： Test cannot export others data

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 呼叫 UserService.request_data_export() 方法
2. 驗證結果符合預期
3. 確認拋出預期的異常

測試角色： 一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-USERSERVICE-017

測試目標： Test export fails when user not found

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 呼叫 UserService.request_data_export() 方法
2. 驗證結果符合預期
3. 確認拋出預期的異常

測試角色： 一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號： UT-USERSERVICE-018

測試目標： Test user can request own data deletion

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 UserService.request_data_deletion() 方法
3. 驗證結果符合預期

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-USERSERVICE-019

測試目標：Test admin can delete user data

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 UserService.request_data_deletion() 方法
3. 驗證結果符合預期

測試角色：平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-USERSERVICE-020

測試目標：Test regular user cannot delete others data

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 UserService.request_data_deletion() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色：一般會員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-USERSERVICE-021

測試目標： Test deletion fails when user not found

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 UserService.request_data_deletion() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色： 平台管理員

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-USERSERVICE-022

測試目標： Test missing old password raises validation error

測試方法： 錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 UserService.change_password() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色： 一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果： 通過

測試編號：UT-USERSERVICE-023

測試目標：Test missing new password raises validation error

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備測試環境 (Mock 必要的依賴對象)
2. 呼叫 UserService.change_password() 方法
3. 驗證結果符合預期
4. 確認拋出預期的異常

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

測試編號：UT-USERSERVICE-024

測試目標：Test change password fails when user not found

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 呼叫 UserService.change_password() 方法
2. 驗證結果符合預期
3. 確認拋出預期的異常

測試角色：一般用戶

測試流程：

- 準備測試環境和測試數據
- 執行被測試的服務方法
- 驗證返回結果符合預期

測試結果：通過

2.14 單元測試結果

測試通過率

```
collected 353 items

tests\unit\test_admin_service.py ..... [ 6%]
tests\unit\test_animal_service.py ..... [ 23%]
tests\unit\test_application_service.py ..... [ 34%]
tests\unit\test_attachment_service.py ..... [ 37%]
tests\unit\test_audit_service.py ..... [ 41%]
tests\unit\test_auth_service.py ..... [ 48%]
tests\unit\test_email_service.py ..... [ 53%]
tests\unit\test_job_service.py ..... [ 58%]
tests\unit\test_medical_record_service.py ..... [ 67%]
tests\unit\test_notification_service.py ..... [ 73%]
tests\unit\test_permission_service.py ..... [ 80%]
tests\unit\test_shelter_service.py ..... [ 93%]
tests\unit\test_user_service.py ..... [100%]

===== 353 passed, 13 warnings in 25.30s =====
```

於 service 層覆蓋率：(pytest 語句覆蓋率)

```
----- coverage: platform win32, python 3.12.3-final-0 -----
Name                               Stmts  Miss  Cover
-----
app\services\admin_service.py       110     3   97%
app\services\animal_service.py      238    11   95%
app\services\application_service.py  176     7   96%
app\services\attachment_service.py   42     0  100%
app\services\audit_service.py        44     0  100%
app\services\auth_service.py        146     8   95%
app\services\email_service.py        113     7   94%
app\services\job_service.py          127    12   91%
app\services\medical_record_service.py 162    14   91%
app\services\notification_service.py 106     6   94%
app\services\permission_service.py    60     2   97%
app\services\shelter_service.py      232    20   91%
app\services\user_service.py         85     1   99%
-----
TOTAL                               1641    91   94%
Coverage HTML written to dir htmlcov
```

3 整合測試

整合測試採用端到端（End-to-End）測試策略，聚焦於完整的 API 端點與業務流程，確保各組件協同運作的正確性。（總共 88 個測試）

測試聚焦點：API 端點與業務流程

為何採用整合測試？

- 驗證組件協作：確保 Route → Service → ORM → Database 的完整流程正確運作
- 模擬真實場景：測試完整的 HTTP 請求/響應流程
- 發現整合問題：及早發現單元測試無法涵蓋的集成問題
- 信心保證：提供系統整體功能正常運作的信心

測試隔離策略：

- 使用 SQLite 記憶體資料庫（:memory:）進行測試隔離
- 每個測試方法獨立的資料庫事務，測試結束後自動回滾
- 透過 Fixture 提供測試用戶、Token 和基礎資料
- 測試間完全獨立，互不影響

測試範圍：

- API 端點測試：驗證所有 HTTP 端點的請求/響應正確性
- 權限控制測試：確保 RBAC 權限系統正常運作
- 業務流程測試：測試完整的業務場景（註冊、登入、CRUD 操作等）
- 錯誤處理測試：驗證異常情況的錯誤響應

測試執行方法：

使用 pytest 執行整合測試，具體實施方式：

pytest tests/integration -v

參數說明：

tests/integration：指定整合測試目錄

-v：詳細輸出模式，顯示每個測試的執行結果

測試環境說明：

- 資料庫：SQLite 記憶體資料庫（:memory:）用於測試隔離
- Flask Test Client：模擬 HTTP 請求，不需要啟動真實服務器
- JWT 認證：真實的 Token 生成與驗證流程
- 測試數據：通過 Fixture 自動準備，測試後自動清理

3.1 Animal 整合測試

test_animal_integration.py - 動物管理整合測試（34 個測試）。

3.1.1 TestAnimalList - 動物列表測試類

測試編號：IT-ANIMAL-001

測試目標：驗證訪客可以獲取已發布動物列表

測試方法：端到端測試 (End-to-End Testing)

測試步驟：

1. 不提供認證 Token
2. 發送 GET 請求到 /api/animals
3. 驗證回應包含動物列表和總數

測試角色：訪客（未登入）

測試流程：

- Fixture 準備：
 - test_animal - 測試動物對象（status=PUBLISHED）
 - test_shelter_animal - 收容所動物對象
- HTTP 請求：GET /api/animals
- 驗證斷言：
 - assert response.status_code == 200
 - assert 'animals' in data
 - assert 'total' in data
 - assert data['total'] >= 2
- 預期行為：返回至少 2 個已發布的動物

測試結果：通過

測試編號：IT-ANIMAL-002

測試目標：驗證登入用戶可以看到包含草稿的動物列表

測試方法：端到端測試 (End-to-End Testing)

測試步驟：

1. 提供有效的認證 Token（auth_headers）
2. 發送 GET 請求到 /api/animals
3. 驗證回應包含動物列表（包括自己的草稿）

測試角色：已登入用戶

測試流程：

- Fixture 準備：

- auth_headers - 認證標頭 (Bearer Token)
 - test_animal - 測試動物
 - test_shelter_animal - 收容所動物
- HTTP 請求：GET /api/animals with headers=auth_headers
- 驗證斷言：
 - assert response.status_code == 200
 - assert 'animals' in data
 - assert 'total' in data
- 預期行為：已登入用戶可見自己的草稿動物

測試結果：通過

測試編號：IT-ANIMAL-003

測試目標：驗證按物種篩選動物功能

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 提供 species 查詢參數 (species=DOG)
2. 發送 GET 請求到 /api/animals?species=DOG
3. 驗證返回的所有動物都是狗

測試角色：訪客

測試流程：

- Fixture 準備：
 - test_animal - 物種為 DOG 的動物
 - test_shelter_animal - 可能不同物種的動物
- HTTP 請求：GET /api/animals?species=DOG
- 驗證斷言：
 - assert response.status_code == 200
 - for animal in data['animals']: assert animal['species'].upper() == 'DOG'
- 預期行為：返回的列表只包含狗 (DOG)

測試結果：通過

測試編號：IT-ANIMAL-004

測試目標：驗證按性別篩選動物功能

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 提供 sex 查詢參數 (sex=MALE)
2. 發送 GET 請求到 /api/animals?sex=MALE
3. 驗證返回的所有動物都是公的

測試角色：訪客

測試流程：

- Fixture 準備：
 - test_animal - 性別為 MALE 的動物

- HTTP 請求：GET /api/animals?sex=MALE
- 驗證斷言：
 - `assert response.status_code == 200`
 - `for animal in data['animals']: assert animal['sex'].upper() == 'MALE'`
- 預期行為：返回的列表只包含公的動物（MALE）

測試結果：通過

測試編號：IT-ANIMAL-005

測試目標：驗證分頁功能正確運作

測試方法：邊界值分析 (Boundary Value Analysis)

測試步驟：

1. 創建 25 個測試動物
2. 發送第一頁請求（`page=1&per_page=10`）
3. 發送第二頁請求（`page=2&per_page=10`）
4. 驗證每頁返回正確數量的動物

測試角色：訪客

測試流程：

- 測試資料準備：

```
for i in range(25):
    animal = Animal(
        animal_id=100 + i,
        name=f'Test Animal {i}',
        species=Species.DOG,
        breed='Mixed',
        sex=Sex.MALE,
        dob=date(2023, 1, 1),
        status=AnimalStatus.PUBLISHED,
        owner_id=None,
        created_by=test_user.user_id
    )
    db_session.add(animal)
db_session.commit()
```
- HTTP 請求 1：GET /api/animals?page=1&per_page=10
 - 驗證：`assert len(data['animals']) == 10`
 - 驗證：`assert data['page'] == 1`
- HTTP 請求 2：GET /api/animals?page=2&per_page=10
 - 驗證：`assert len(data['animals']) == 10`
 - 驗證：`assert data['page'] == 2`
- 預期行為：每頁顯示 10 個動物，頁碼正確

測試結果：通過

3.1.2 TestAnimalListFilters - 動物列表進階篩選測試類

測試編號：IT-ANIMAL-006

測試目標：驗證按狀態篩選動物功能

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 提供 status 查詢參數 (status=PUBLISHED)
2. 發送 GET 請求到 /api/animals?status=PUBLISHED
3. 驗證返回的所有動物都是已發布狀態

測試角色：訪客

測試流程：

- Fixture 準備：
 - test_animal - status=PUBLISHED 的動物
- HTTP 請求：GET /api/animals?status=PUBLISHED
- 驗證斷言：
 - assert response.status_code == 200
 - for animal in data['animals']: assert animal['status'] == 'PUBLISHED'
- 預期行為：返回的列表只包含已發布的動物

測試結果：通過

測試編號：IT-ANIMAL-007

測試目標：驗證按地區篩選動物功能

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 創建有地區資訊的動物
2. 發送帶地區參數的 GET 請求
3. 驗證返回的動物符合地區條件

測試角色：訪客

測試流程：

- 測試資料準備：

```
animal = Animal(
    animal_id=600,
    name='Regional Dog',
    species=Species.DOG,
    breed='Mixed',
    sex=Sex.MALE,
    dob=date(2023, 1, 1),
    status=AnimalStatus.PUBLISHED,
    owner_id=test_user.user_id,
    created_by=test_user.user_id
)
```

- `db_session.add(animal)`
- `db_session.commit()`
- HTTP 請求：GET /api/animals
- 驗證：assert response.status_code == 200
- 預期行為：API 正常回應地區篩選請求

測試結果：通過

測試編號：IT-ANIMAL-008

測試目標：驗證關鍵字搜尋功能

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 提供 q (關鍵字) 參數
2. 發送 GET 請求到 /api/animals?q=關鍵字
3. 驗證返回匹配關鍵字的動物

測試角色：訪客

測試流程：

- Fixture 準備：
 - test_animal - 有特定名稱的動物
- HTTP 請求：GET /api/animals?q={test_animal.name[:3]}
- 驗證斷言：
 - assert response.status_code == 200
 - assert 'animals' in data
- 預期行為：返回名稱包含關鍵字的動物列表

測試結果：通過

3.1.3 TestAnimalCreate - 動物創建測試類

測試編號：IT-ANIMAL-009

測試目標：驗證未認證用戶無法創建動物

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 不提供認證 Token
2. 發送 POST 請求創建動物
3. 驗證返回 401 未授權錯誤

測試角色：訪客（未登入）

測試流程：

- Request Payload：
 - {
 - "name": "Unauthorized Dog",
 - "species": "DOG",
 - "breed": "Mixed",

- "sex": "MALE",
- "dob": "2022-01-01"
- }
- HTTP 請求：POST /api/animals with content_type='application/json' (無 auth headers)
- 驗證斷言：
 - assert response.status_code == 401
- 預期行為：未登入用戶無法創建動物

測試結果：通過

3.1.4 TestAnimalDetail - 動物詳情測試類

測試編號：IT-ANIMAL-010

測試目標：驗證獲取動物詳情功能

測試方法：端到端測試 (End-to-End Testing)

測試步驟：

1. 提供有效的 animal_id
2. 發送 GET 請求到 /api/animals/{animal_id}
3. 驗證返回動物的完整資訊

測試角色：訪客

測試流程：

- Fixture 準備：
 - test_animal - animal_id=1, name='Test Dog'
- HTTP 請求：GET /api/animals/{test_animal.animal_id}
- 驗證斷言：
 - assert response.status_code == 200
 - assert data['animal_id'] == test_animal.animal_id
 - assert data['name'] == test_animal.name
 - assert data['species'] == test_animal.species.value
- 預期行為：返回完整的動物資訊

測試結果：通過

測試編號：IT-ANIMAL-011

測試目標：驗證獲取不存在的動物返回 404

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 提供不存在的 animal_id (999999)
2. 發送 GET 請求
3. 驗證返回 404 Not Found

測試角色：訪客

測試流程：

- HTTP 請求：GET /api/animals/999999
- 驗證斷言：
 - `assert response.status_code == 404`
- 預期行為：找不到動物，返回 404 錯誤

測試結果：通過

3.1.5 TestAnimalGetDetail - 動物詳情擴展測試類

測試編號：IT-ANIMAL-012

測試目標：驗證獲取動物詳情包含關聯資料

測試方法：端到端測試 (End-to-End Testing)

測試步驟：

1. 發送 GET 請求獲取動物詳情
2. 驗證回應包含基本欄位
3. 驗證可能包含關聯資料（圖片、醫療記錄等）

測試角色：訪客

測試流程：

- Fixture 準備：
 - `test_animal` - 測試動物
- HTTP 請求：GET /api/animals/{test_animal.animal_id}
- 驗證斷言：
 - `assert response.status_code == 200`
 - `assert 'animal_id' in data`
 - `assert 'name' in data`
 - `assert 'species' in data`
- 預期行為：返回動物的詳細資訊

測試結果：通過

測試編號：IT-ANIMAL-013

測試目標：驗證擁有者可以查看自己的草稿動物

測試方法：決策表測試 (Decision Table Testing)

測試步驟：

1. 創建草稿動物（status=DRAFT）
2. 使用擁有者的 Token 發送 GET 請求
3. 驗證可以成功獲取草稿動物詳情

測試角色：動物擁有者

測試流程：

- 測試資料準備：

```
draft_animal = Animal(
    animal_id=200,
    name='Draft Dog',
```

- ```
species=Species.DOG,
breed='Mixed',
sex=Sex.MALE,
dob=date(2023, 1, 1),
status=AnimalStatus.DRAFT,
owner_id=test_user.user_id,
created_by=test_user.user_id
)
db_session.add(draft_animal)
db_session.commit()
```
- HTTP 請求：GET /api/animals/{draft\_animal.animal\_id} with headers=auth\_headers
  - 驗證斷言：
    - assert response.status\_code == 200
    - assert data['status'].lower() == 'draft'
  - 預期行為：擁有者可以查看自己的草稿動物

測試結果：通過

---

### 3.1.6 TestAnimalUpdate - 動物更新測試類

---

測試編號：IT-ANIMAL-014

測試目標：驗證擁有者可以更新自己的動物

測試方法：端到端測試 (End-to-End Testing)

測試步驟：

1. 準備更新資料 (name, description)
2. 使用擁有者的 Token 發送 PATCH 請求
3. 驗證動物資料更新成功

測試角色：動物擁有者

測試流程：

- Fixture 準備：
  - auth\_headers - 擁有者的認證標頭
  - test\_animal - owner\_id=test\_user.user\_id
- Request Payload：

```
{
 "name": "Updated Dog Name",
 "description": "Updated description"
}
```
- HTTP 請求：PATCH /api/animals/{test\_animal.animal\_id} with headers=auth\_headers
- 驗證斷言：
  - assert response.status\_code == 200
  - if 'name' in data: assert data['name'] == 'Updated Dog Name'
- 資料庫驗證：



- db\_session.refresh(test\_animal)
  - assert test\_animal.name == 'Updated Dog Name'
  - assert test\_animal.description == 'Updated description'
- 預期行為：動物資料成功更新

測試結果：通過

---

測試編號：IT-ANIMAL-015

測試目標：驗證無法更新他人的動物

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 創建另一個用戶的 Token
2. 嘗試更新他人的動物
3. 驗證返回 403 Forbidden

測試角色：非動物擁有者

測試流程：

- 測試資料準備：

```
other_user = User(
 email='other@test.com',
 username='otheruser',
 password_hash='hashed',
 role=UserRole.GENERAL_MEMBER,
 verified=True
)
db_session.add(other_user)
db_session.commit()
```

```
token = create_access_token(identity=other_user.user_id)
headers = {
 'Authorization': f'Bearer {token}',
 'Content-Type': 'application/json'
}
```

Request Payload：

```
{
 "name": "Unauthorized Update"
}
```

- HTTP 請求：PATCH /api/animals/{test\_animal.animal\_id} with headers=headers
- 驗證斷言：
  - assert response.status\_code == 403
- 預期行為：無權限更新，返回 403 錯誤

測試結果：通過

---

### 3.1.7 TestAnimalDelete - 動物刪除測試類

---

**測試編號：**IT-ANIMAL-016

**測試目標：**驗證擁有者可以刪除自己的動物

**測試方法：**端到端測試 (End-to-End Testing)

**測試步驟：**

1. 創建測試動物
2. 使用擁有者的 Token 發送 DELETE 請求
3. 驗證動物被軟刪除 (deleted\_at 設置)

**測試角色：**動物擁有者

**測試流程：**

- 測試資料準備：

```
animal = Animal(
 animal_id=300,
 name='To Delete',
 species=Species.DOG,
 breed='Mixed',
 sex=Sex.MALE,
 dob=date(2023, 1, 1),
 status=AnimalStatus.DRAFT,
 owner_id=test_user.user_id,
 created_by=test_user.user_id
)
db_session.add(animal)
db_session.commit()
animal_id = animal.animal_id
```
- HTTP 請求：DELETE /api/animals/{animal\_id} with headers=auth\_headers
- 驗證斷言：
  - assert response.status\_code in [200, 204]
- 資料庫驗證：
  - deleted\_animal = Animal.query.filter\_by(animal\_id=animal\_id).first()
  - assert deleted\_animal.deleted\_at is not None
- 預期行為：動物被軟刪除 (deleted\_at 不為 None)

**測試結果：**通過

---

**測試編號：**IT-ANIMAL-017

**測試目標：**驗證無權限無法刪除動物

**測試方法：**錯誤推測 (Error Guessing)

**測試步驟：**

1. 創建另一個用戶的 Token
2. 嘗試刪除他人的動物

### 3. 驗證返回 403 Forbidden

**測試角色：**非動物擁有者

**測試流程：**

- 測試資料準備：

```
other_user = User(
 email='other2@test.com',
 username='otheruser2',
 password_hash='hashed',
 role=UserRole.GENERAL_MEMBER,
 verified=True
)
db_session.add(other_user)
db_session.commit()

token = create_access_token(identity=other_user.user_id)
headers = {
 'Authorization': f'Bearer {token}',
 'Content-Type': 'application/json'
}
```
- HTTP 請求：DELETE /api/animals/{test\_animal.animal\_id} with headers=headers
- 驗證斷言：
  - assert response.status\_code == 403
- 預期行為：無權限刪除，返回 403 錯誤

**測試結果：**通過

---

### 3.1.8 TestAnimalImages - 動物圖片管理測試類

---

**測試編號：**IT-ANIMAL-018

**測試目標：**驗證缺少必要欄位時新增圖片失敗

**測試方法：**錯誤推測 (Error Guessing)

**測試步驟：**

1. 準備不完整的圖片資料（缺少 image\_url）
2. 發送 POST 請求
3. 驗證返回 400 Bad Request

**測試角色：**動物擁有者

**測試流程：**

- Request Payload：

```
{
 "storage_key": "animals/test.jpg"
}
```
- HTTP 請求：POST /api/animals/{test\_animal.animal\_id}/images with headers=auth\_headers

- 驗證斷言：
  - `assert response.status_code == 400`
- 預期行為：缺少必要欄位，返回 400 錯誤

測試結果：通過

---

### 3.1.9 TestAnimalImagePermissions - 動物圖片權限測試類

---

測試編號：IT-ANIMAL-019

測試目標：驗證未登入用戶無法添加圖片

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 不提供認證 Token
2. 嘗試新增圖片
3. 驗證返回 401 Unauthorized

測試角色：訪客（未登入）

測試流程：

- Request Payload：

```
{
 "storage_key": "test-key",
 "image_url": "https://example.com/test.jpg"
}
```
- HTTP 請求：POST `/api/animals/{test_animal.animal_id}/images` with `content_type='application/json'`
- 驗證斷言：
  - `assert response.status_code == 401`
- 預期行為：未登入無法新增圖片

測試結果：通過

---

測試編號：IT-ANIMAL-020

測試目標：驗證未登入用戶無法刪除圖片

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 不提供認證 Token
2. 嘗試刪除圖片
3. 驗證返回 401 Unauthorized

測試角色：訪客（未登入）

測試流程：

- HTTP 請求：DELETE `/api/animals/{test_animal.animal_id}/images/1`
- 驗證斷言：
  - `assert response.status_code == 401`

- 預期行為：未登入無法刪除圖片

測試結果：通過

---

測試編號：IT-ANIMAL-021

測試目標：驗證未登入用戶無法重新排序圖片

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 不提供認證 Token
2. 嘗試重新排序圖片
3. 驗證返回 401 Unauthorized

測試角色：訪客（未登入）

測試流程：

- Request Payload：

```
{
 "image_orders": []
}
```
- HTTP 請求：PATCH /api/animals/{test\_animal.animal\_id}/images/reorder with content\_type='application/json'
- 驗證斷言：
  - assert response.status\_code == 401
- 預期行為：未登入無法排序圖片

測試結果：通過

---

測試編號：IT-ANIMAL-022

測試目標：驗證擁有者可以刪除動物圖片

測試方法：端到端測試 (End-to-End Testing)

測試步驟：

1. 創建測試圖片
2. 使用擁有者的 Token 發送 DELETE 請求
3. 驗證圖片刪除成功

測試角色：動物擁有者

測試流程：

- 測試資料準備：

```
image = AnimalImage(
 animal_image_id=500,
 animal_id=test_animal.animal_id,
 storage_key='test-key',
 url='https://example.com/test.jpg',
 order=1
)
db_session.add(image)
db_session.commit()
```

- ```
image_id = image.animal_image_id
```
- HTTP 請求：DELETE /api/animals/{test_animal.animal_id}/images/{image_id} with headers=auth_headers
 - 驗證斷言：
 - assert response.status_code == 200
 - assert data['message'] == '圖片已刪除'
 - 預期行為：圖片成功刪除

測試結果：通過

測試編號：IT-ANIMAL-023

測試目標：驗證擁有者可以重新排序動物圖片

測試方法：端到端測試 (End-to-End Testing)

測試步驟：

1. 創建 3 張測試圖片
2. 發送重新排序請求
3. 驗證圖片順序更新成功

測試角色：動物擁有者

測試流程：

- 測試資料準備：


```
images = []
for i in range(3):
    image = AnimalImage(
        animal_image_id=510 + i,
        animal_id=test_animal.animal_id,
        storage_key=f'test-key-{i}',
        url=f'https://example.com/test-{i}.jpg',
        order=i
    )
    db_session.add(image)
    images.append(image)
db_session.commit()
```

Request Payload：

```
{
  "image_orders": [
    {"image_id": 512, "order": 0},
    {"image_id": 510, "order": 1},
    {"image_id": 511, "order": 2}
  ]
}
```
- HTTP 請求：PATCH /api/animals/{test_animal.animal_id}/images/reorder with headers=auth_headers
- 驗證斷言：
 - assert response.status_code == 200

- `assert data['message'] == '圖片順序已更新'`
- 預期行為：圖片順序成功更新

測試結果：通過

3.1.10 TestAnimalStatusManagement - 動物狀態管理測試類

測試編號：IT-ANIMAL-024

測試目標：驗證提交動物供審核（DRAFT → SUBMITTED）

測試方法：狀態轉換測試 (State Transition Testing)

測試步驟：

1. 創建草稿動物（status=DRAFT）
2. 使用擁有者的 Token 發送 POST 請求到 /submit
3. 驗證狀態變更為 SUBMITTED

測試角色：動物擁有者

測試流程：

- 測試資料準備：

```
animal = Animal(
    animal_id=400,
    name='To Submit',
    species=Species.DOG,
    breed='Mixed',
    sex=Sex.MALE,
    dob=date(2023, 1, 1),
    status=AnimalStatus.DRAFT,
    owner_id=test_user.user_id,
    created_by=test_user.user_id
)
db_session.add(animal)
db_session.commit()
```
- HTTP 請求：POST /api/animals/{animal.animal_id}/submit with headers=auth_headers
- 驗證斷言：
 - `assert response.status_code == 200`
 - `assert '已提交審核' in data['message']`
 - `assert data['animal']['status'] == 'SUBMITTED'`
- 資料庫驗證：
 - `db_session.refresh(animal)`
 - `assert animal.status == AnimalStatus.SUBMITTED`
- 預期行為：動物狀態從 DRAFT 變更為 SUBMITTED

測試結果：通過

測試編號：IT-ANIMAL-025

測試目標：驗證管理員發布動物 (SUBMITTED → PUBLISHED)

測試方法：狀態轉換測試 (State Transition Testing)

測試步驟：

1. 創建管理員用戶
2. 創建已提交的動物 (status=SUBMITTED)
3. 使用管理員的 Token 發送 POST 請求到 /publish
4. 驗證狀態變更為 PUBLISHED

測試角色：平台管理員

測試流程：

- 測試資料準備：

```
admin = User(
    email='admin@test.com',
    username='admin',
    password_hash='hashed',
    role=UserRole.ADMIN,
    verified=True
)
db_session.add(admin)
db_session.commit()

animal = Animal(
    animal_id=401,
    name='To Publish',
    species=Species.DOG,
    breed='Mixed',
    sex=Sex.MALE,
    dob=date(2023, 1, 1),
    status=AnimalStatus.SUBMITTED,
    owner_id=test_user.user_id,
    created_by=test_user.user_id
)
db_session.add(animal)
db_session.commit()

token = create_access_token(identity=admin.user_id)
headers = {
    'Authorization': f'Bearer {token}',
    'Content-Type': 'application/json'
}
```
- HTTP 請求：POST /api/animals/{animal.animal_id}/publish with headers=headers
- 驗證斷言：
 - assert response.status_code == 200
 - assert '已發布' in data['message']
- 資料庫驗證：

- `db_session.refresh(animal)`
 - `assert animal.status == AnimalStatus.PUBLISHED`
- 預期行為：動物狀態從 SUBMITTED 變更為 PUBLISHED

測試結果：通過

測試編號：IT-ANIMAL-026

測試目標：驗證下架動物（PUBLISHED → RETIRED）

測試方法：狀態轉換測試 (State Transition Testing)

測試步驟：

1. 創建已發布的動物（status=PUBLISHED）
2. 使用擁有者的 Token 發送 POST 請求到 /retire
3. 驗證狀態變更為 RETIRED

測試角色：動物擁有者

測試流程：

- 測試資料準備：

```
animal = Animal(
    animal_id=402,
    name='To Retire',
    species=Species.DOG,
    breed='Mixed',
    sex=Sex.MALE,
    dob=date(2023, 1, 1),
    status=AnimalStatus.PUBLISHED,
    owner_id=test_user.user_id,
    created_by=test_user.user_id
)
db_session.add(animal)
db_session.commit()
```
- HTTP 請求：POST /api/animals/{animal.animal_id}/retire with headers=auth_headers
- 驗證斷言：
 - `assert response.status_code == 200`
 - `assert '已下架' in data['message']`
- 資料庫驗證：
 - `db_session.refresh(animal)`
 - `assert animal.status == AnimalStatus.RETIRED`
- 預期行為：動物狀態從 PUBLISHED 變更為 RETIRED

測試結果：通過

測試編號：IT-ANIMAL-027

測試目標：驗證管理員拒絕動物並記錄原因（SUBMITTED → DRAFT）

測試方法：狀態轉換測試 (State Transition Testing)

測試步驟：

1. 創建管理員用戶

2. 創建已提交的動物 (status=SUBMITTED)
3. 使用管理員的 Token 發送 POST 請求到 /reject，帶拒絕原因
4. 驗證狀態變更為 DRAFT 且記錄拒絕原因

測試角色：平台管理員

測試流程：

- 測試資料準備：

```
admin = User(
    email='admin2@test.com',
    username='admin2',
    password_hash='hashed',
    role=UserRole.ADMIN,
    verified=True
)
db_session.add(admin)
db_session.commit()

animal = Animal(
    animal_id=403,
    name='To Reject',
    species=Species.DOG,
    breed='Mixed',
    sex=Sex.MALE,
    dob=date(2023, 1, 1),
    status=AnimalStatus.SUBMITTED,
    owner_id=test_user.user_id,
    created_by=test_user.user_id
)
db_session.add(animal)
db_session.commit()

token = create_access_token(identity=admin.user_id)
headers = {
    'Authorization': f'Bearer {token}',
    'Content-Type': 'application/json'
}
Request Payload :
{
    "rejection_reason": "資料不完整，請補充詳細描述"
}
```
- HTTP 請求：POST /api/animals/{animal.animal_id}/reject with headers=headers
- 驗證斷言：
 - assert response.status_code == 200
 - assert '已拒絕' in data['message']
- 資料庫驗證：

- db_session.refresh(animal)
 - assert animal.status == AnimalStatus.DRAFT
 - assert animal.rejection_reason == '資料不完整，請補充詳細描述'
 - 預期行為：動物狀態從 SUBMITTED 變更為 DRAFT，並記錄拒絕原因
- 測試結果：通過
-

測試編號：IT-ANIMAL-028

測試目標：驗證拒絕動物必須提供原因

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 創建管理員和已提交的動物
2. 發送拒絕請求但不提供原因
3. 驗證返回 400 Bad Request

測試角色：平台管理員

測試流程：

- 測試資料準備：(同 IT-ANIMAL-031)
- Request Payload：
- {}
- HTTP 請求：POST /api/animals/{animal.animal_id}/reject with headers=headers
- 驗證斷言：
 - assert response.status_code == 400
 - assert '請提供拒絕原因' in data['message']
- 預期行為：缺少拒絕原因，返回 400 錯誤

測試結果：通過

3.1.11 TestAnimalStatusPermissions - 動物狀態權限測試類

測試編號：IT-ANIMAL-029

測試目標：驗證未登入用戶無法提交動物

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 不提供認證 Token
2. 嘗試提交動物
3. 驗證返回 401 Unauthorized

測試角色：訪客（未登入）

測試流程：

- HTTP 請求：POST /api/animals/{test_animal.animal_id}/submit
- 驗證斷言：
 - assert response.status_code == 401
- 預期行為：未登入無法提交動物

測試結果：通過

測試編號：IT-ANIMAL-030

測試目標：驗證未登入用戶無法發布動物

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 不提供認證 Token
2. 嘗試發布動物
3. 驗證返回 401 Unauthorized

測試角色：訪客（未登入）

測試流程：

- HTTP 請求：POST /api/animals/{test_animal.animal_id}/publish
- 驗證斷言：
 - assert response.status_code == 401
- 預期行為：未登入無法發布動物

測試結果：通過

測試編號：IT-ANIMAL-031

測試目標：驗證未登入用戶無法下架動物

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 不提供認證 Token
2. 嘗試下架動物
3. 驗證返回 401 Unauthorized

測試角色：訪客（未登入）

測試流程：

- HTTP 請求：POST /api/animals/{test_animal.animal_id}/retire
- 驗證斷言：
 - assert response.status_code == 401
- 預期行為：未登入無法下架動物

測試結果：通過

測試編號：IT-ANIMAL-032

測試目標：驗證未登入用戶無法拒絕動物

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 不提供認證 Token
2. 嘗試拒絕動物
3. 驗證返回 401 Unauthorized

測試角色：訪客（未登入）

測試流程：

- Request Payload :

```
{
  "rejection_reason": "test"
}
```
- HTTP 請求：POST /api/animals/{test_animal.animal_id}/reject with content_type='application/json'
- 驗證斷言：
 - assert response.status_code == 401
- 預期行為：未登入無法拒絕動物

測試結果：通過

3.1.12 TestAnimalEdgeCases - 動物管理邊界案例測試類

測試編號：IT-ANIMAL-033

測試目標：驗證更新不存在的動物返回錯誤

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 提供不存在的 animal_id (999999)
2. 嘗試更新動物
3. 驗證返回錯誤狀態碼

測試角色：已登入用戶

測試流程：

- Request Payload :

```
{
  "name": "Updated"
}
```
- HTTP 請求：PATCH /api/animals/999999 with headers=auth_headers
- 驗證斷言：
 - assert response.status_code in [404, 400, 403, 500]
- 預期行為：返回錯誤狀態碼

測試結果：通過

測試編號：IT-ANIMAL-034

測試目標：驗證刪除不存在的動物返回錯誤

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 提供不存在的 animal_id (999999)
2. 嘗試刪除動物
3. 驗證返回錯誤狀態碼

測試角色：已登入用戶

測試流程：

- HTTP 請求：DELETE /api/animals/999999 with headers=auth_headers
- 驗證斷言：
 - assert response.status_code in [404, 400, 403, 500]
- 預期行為：返回錯誤狀態碼

測試結果：通過

3.2 Application 整合測試

test_application_integration.py - 領養申請整合測試（12 個測試）

3.2.1 TestApplicationCreate - 創建領養申請測試類

測試編號：IT-APPLICATION-001

測試目標：驗證用戶成功創建領養申請

測試方法：端到端測試 (End-to-End Testing)

測試步驟：

1. 創建申請人用戶（不同於動物擁有者）
2. 準備領養申請資料
3. 使用申請人的 Token 發送 POST 請求
4. 驗證申請創建成功且狀態為 PENDING

測試角色：已登入用戶（申請人）

測試流程：

- 測試資料準備：

```
applicant = User(
    email='applicant@test.com',
    username='applicant',
    password_hash=hash_password('Password123'),
    role=UserRole.GENERAL_MEMBER,
    verified=True
)
db_session.add(applicant)
db_session.commit()

applicant_token = create_access_token(identity=applicant.user_id)
applicant_headers = {
    'Authorization': f'Bearer {applicant_token}',
    'Content-Type': 'application/json'
}
```

```

}
Request Payload :
{
  "animal_id": 1,
  "type": "ADOPTION",
  "contact_phone": "0912345678",
  "contact_address": "台北市中正區",
  "occupation": "工程師",
  "housing_type": "公寓",
  "has_experience": true,
  "reason": "想要給動物一個溫暖的家"
}

```

- HTTP 請求：POST /api/applications with headers=applicant_headers
- 驗證斷言：
 - assert response.status_code == 201
 - assert 'application' in data
 - assert application_data['animal_id'] == test_animal.animal_id
 - assert application_data['applicant_id'] == applicant.user_id
 - assert application_data['type'] == 'ADOPTION'
 - assert application_data['status'] == 'PENDING'
- 資料庫驗證：
 - 查詢 Application 表確認記錄存在
 - 驗證 contact_phone == '0912345678'
- 預期行為：成功創建領養申請

測試結果：通過

測試編號：IT-APPLICATION-002

測試目標：驗證未認證用戶無法創建申請

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 不提供認證 Token
2. 嘗試創建領養申請
3. 驗證返回 401 Unauthorized

測試角色：訪客（未登入）

測試流程：

- Request Payload :


```

{
  "animal_id": 1,
  "type": "ADOPTION"
}

```
- HTTP 請求：POST /api/applications with content_type='application/json'（無 auth headers）
- 驗證斷言：

- `assert response.status_code == 401`
- 預期行為：未登入無法創建申請

測試結果：通過

測試編號：IT-APPLICATION-003

測試目標：驗證為不存在的動物創建申請返回 404

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 提供不存在的 `animal_id` (999999)
2. 嘗試創建申請
3. 驗證返回 404 Not Found

測試角色：已登入用戶

測試流程：

- Request Payload :

```
{
  "animal_id": 999999,
  "type": "ADOPTION"
}
```
- HTTP 請求：POST `/api/applications` with `headers=auth_headers`
- 驗證斷言：
 - `assert response.status_code == 404`
- 預期行為：找不到動物，返回 404 錯誤

測試結果：通過

測試編號：IT-APPLICATION-004

測試目標：驗證無法創建重複申請

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 創建第一個申請
2. 嘗試用相同用戶和動物創建第二個申請
3. 驗證返回 409 Conflict

測試角色：已登入用戶（申請人）

測試流程：

- 測試資料準備：

```
applicant = User(
    email='applicant_dup@test.com',
    username='applicant_dup',
    password_hash=hash_password('Password123'),
    role=UserRole.GENERAL_MEMBER,
    verified=True
)
db_session.add(applicant)
db_session.commit()
```



```

application = Application(
    application_id=100,
    applicant_id=applicant.user_id,
    animal_id=test_animal.animal_id,
    type=ApplicationType.ADOPTION,
    status=ApplicationStatus.PENDING
)
db_session.add(application)
db_session.commit()

applicant_token = create_access_token(identity=applicant.user_id)
applicant_headers = {
    'Authorization': f'Bearer {applicant_token}',
    'Content-Type': 'application/json'
}
Request Payload :
{
    "animal_id": 1,
    "type": "ADOPTION",
    "contact_phone": "0912345678",
    "contact_address": "台北市",
    "reason": "想領養"
}

```

- HTTP 請求：POST /api/applications with headers=applicant_headers
- 驗證斷言：
 - assert response.status_code == 409
- 預期行為：檢測到重複申請，返回 409 錯誤

測試結果：通過

3.2.2 TestApplicationList - 申請列表測試類

測試編號：IT-APPLICATION-005

測試目標：驗證獲取我的申請列表

測試方法：端到端測試 (End-to-End Testing)

測試步驟：

1. 創建測試申請
2. 使用申請人的 Token 發送 GET 請求
3. 驗證返回的申請列表包含該申請

測試角色：已登入用戶（申請人）

測試流程：

- 測試資料準備：

```

application = Application(
    application_id=101,
    applicant_id=test_user.user_id,
    animal_id=test_animal.animal_id,
    type=ApplicationType.ADOPTION,
    status=ApplicationStatus.PENDING
)
db_session.add(application)
db_session.commit()

```

- HTTP 請求：GET /api/applications with headers=auth_headers
- 驗證斷言：
 - assert response.status_code == 200
 - assert 'items' in data
 - assert len(data['items']) > 0
 - assert data['items'][0]['applicant_id'] == test_user.user_id
- 預期行為：返回當前用戶的所有申請

測試結果：通過

測試編號：IT-APPLICATION-006

測試目標：驗證按狀態篩選申請列表

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 創建不同狀態的申請
2. 提供 status 查詢參數
3. 驗證返回的申請都符合指定狀態

測試角色：已登入用戶

測試流程：

- 測試資料準備：
- app1 = Application(
 - application_id=102,
 - applicant_id=test_user.user_id,
 - animal_id=test_animal.animal_id,
 - type=ApplicationType.ADOPTION,
 - status=ApplicationStatus.PENDING
-)
- db_session.add(app1)
- db_session.commit()
- HTTP 請求：GET /api/applications?status=PENDING with headers=auth_headers
- 驗證斷言：
 - assert response.status_code == 200
 - assert 'items' in data
 - for app in data['items']: assert app['status'] == 'PENDING'
- 預期行為：只返回 PENDING 狀態的申請

測試結果：通過

3.2.3 TestApplicationReview - 申請審核測試類

測試編號：IT-APPLICATION-007

測試目標：驗證動物擁有者批准申請

測試方法：端到端測試 (End-to-End Testing)

測試步驟：

1. 創建申請人和申請記錄
2. 使用動物擁有者的 Token 發送審核請求
3. 驗證申請狀態變更為 APPROVED

測試角色：動物擁有者

測試流程：

- 測試資料準備：

```
applicant = User(  
    email='applicant@test.com',  
    username='applicant',  
    password_hash=hash_password('Password123'),  
    role=UserRole.GENERAL_MEMBER,  
    verified=True
```

```
)  
db_session.add(applicant)  
db_session.commit()
```

```
application = Application(  
    application_id=103,  
    applicant_id=applicant.user_id,  
    animal_id=test_animal.animal_id,  
    type=ApplicationType.ADOPTION,  
    status=ApplicationStatus.PENDING
```

```
)  
db_session.add(application)  
db_session.commit()
```

- Request Payload：

```
{  
    "action": "approve",  
    "review_notes": "申請通過"  
}
```

- HTTP 請求：POST /api/applications/{application.application_id}/review with headers=auth_headers

- 驗證斷言：

- assert response.status_code == 200
- assert 'application' in data or 'status' in data
- app_status = data.get('application', {}).get('status') or data.get('status')

- `assert app_status == 'APPROVED'`
- 資料庫驗證：
 - `db_session.refresh(application)`
 - `assert application.status == ApplicationStatus.APPROVED`
 - `assert application.review_notes == '申請通過'`
- 預期行為：申請狀態變更為 APPROVED

測試結果：通過

測試編號：IT-APPLICATION-008

測試目標：驗證動物擁有者拒絕申請並提供原因

測試方法：端到端測試 (End-to-End Testing)

測試步驟：

1. 創建申請人和申請記錄
2. 使用動物擁有者的 Token 發送拒絕請求
3. 驗證申請狀態變更為 REJECTED 且記錄原因

測試角色：動物擁有者

測試流程：

- 測試資料準備：(同 IT-APPLICATION-007，但使用不同 `application_id`)
- Request Payload：

```
{
  "action": "reject",
  "review_notes": "申請條件不符"
}
```
- HTTP 請求：POST `/api/applications/{application.application_id}/review` with `headers=auth_headers`
- 驗證斷言：
 - `assert response.status_code == 200`
 - 驗證 `status == 'REJECTED'`
- 資料庫驗證：
 - `assert application.status == ApplicationStatus.REJECTED`
 - `assert application.review_notes == '申請條件不符'`
- 預期行為：申請狀態變更為 REJECTED 並記錄原因

測試結果：通過

測試編號：IT-APPLICATION-009

測試目標：驗證無權限用戶無法審核申請

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 創建申請記錄
2. 使用非擁有者的 Token 嘗試審核
3. 驗證返回 403 Forbidden

測試角色：非動物擁有者

測試流程：

- 測試資料準備：創建其他用戶的申請
- Request Payload：

```
{
  "action": "approve",
  "review_notes": "嘗試審核"
}
```
- HTTP 請求：POST /api/applications/{application.application_id}/review with headers=auth_headers（非擁有者）
- 驗證斷言：
 - assert response.status_code == 403
- 預期行為：無權限審核，返回 403 錯誤

測試結果：通過

3.2.4 TestApplicationDetail - 申請詳情測試類

測試編號：IT-APPLICATION-010

測試目標：驗證申請人可以獲取自己的申請詳情

測試方法：端到端測試 (End-to-End Testing)

測試步驟：

1. 創建申請記錄
2. 使用申請人的 Token 發送 GET 請求
3. 驗證返回申請的完整資訊

測試角色：申請人

測試流程：

- 測試資料準備：創建申請記錄（applicant_id=test_user.user_id）
- HTTP 請求：GET /api/applications/{application.application_id} with headers=auth_headers
- 驗證斷言：
 - assert response.status_code == 200
 - assert data['application_id'] == application.application_id
 - assert data['applicant_id'] == test_user.user_id
- 預期行為：返回完整的申請詳情

測試結果：通過

測試編號：IT-APPLICATION-011

測試目標：驗證動物擁有者可以查看申請詳情

測試方法：端到端測試 (End-to-End Testing)

測試步驟：

1. 創建申請記錄
2. 使用動物擁有者的 Token 發送 GET 請求
3. 驗證可以獲取申請詳情

測試角色：動物擁有者

測試流程：

- 測試資料準備：創建其他用戶的申請（針對 test_user 擁有的動物）
- HTTP 請求：GET /api/applications/{application.application_id} with headers=auth_headers（動物擁有者）
- 驗證斷言：
 - assert response.status_code == 200
 - assert data['application_id'] == application.application_id
- 預期行為：動物擁有者可以查看針對其動物的申請

測試結果：通過

測試編號：IT-APPLICATION-012

測試目標：驗證無權限用戶無法查看申請詳情

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 創建申請記錄
2. 使用無關用戶的 Token 嘗試查看
3. 驗證返回 403 Forbidden

測試角色：無關用戶

測試流程：

- 測試資料準備：創建其他用戶的申請（針對其他用戶的動物）
- HTTP 請求：GET /api/applications/{application.application_id} with headers=auth_headers（無關用戶）
- 驗證斷言：
 - assert response.status_code == 403
- 預期行為：無權限查看，返回 403 錯誤

測試結果：通過

3.3 Auth 整合測試

test_auth_integration.py - 認證系統整合測試（7 個測試）。

3.3.1 TestAuthRegistration - 用戶註冊測試類

測試編號：IT-AUTH-001

測試目標：驗證新用戶成功註冊

測試方法：端到端測試 (End-to-End Testing)

測試步驟：

1. 準備有效的註冊資料 (email, username, password)
2. 發送 POST 請求到 /api/auth/register
3. 驗證用戶創建成功並返回註冊確認資訊

測試角色：訪客

測試流程：

- Request Payload :

```
{
  "email": "newuser@test.com",
  "username": "newuser",
  "password": "Password123"
}
```
- HTTP 請求：POST /api/auth/register with content_type='application/json'
- 驗證斷言：
 - assert response.status_code == 201
 - assert 'pending_id' in data
 - assert 'masked_email' in data
 - assert 'message' in data
- 資料庫驗證：

```
from app.models.pending_registration import PendingRegistration
pending = PendingRegistration.query.filter_by(
    email='newuser@test.com'
).first()

assert pending is not None
assert pending.email == 'newuser@test.com'
assert pending.username == 'newuser'
assert pending.verification_code_hash is not None
```
- AC (Acceptance Criteria) :
 - AC #1: 端點已建立且可接受 POST 請求
 - AC #2: 驗證請求正文 (email, password, username)
 - AC #3: 資料正確寫入 pending_registrations 表
 - AC #4: 密碼使用 bcrypt 加密

- AC #5: 返回成功訊息和 pending_id
 - 預期行為：用戶註冊請求創建成功，發送驗證碼到郵箱
- 測試結果：通過
-

測試編號：IT-AUTH-002

測試目標：驗證重複郵箱無法註冊

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 使用已存在的郵箱地址
2. 嘗試註冊新帳號
3. 驗證返回 409 Conflict

測試角色：訪客

測試流程：

- Fixture 準備：
 - test_user - 已存在的用戶 (email='user@test.com')
- Request Payload：

```
{
  "email": "user@test.com",
  "username": "anotheruser",
  "password": "Password123"
}
```
- HTTP 請求：POST /api/auth/register with content_type='application/json'
- 驗證斷言：
 - assert response.status_code == 409
 - assert '已被註冊' in data.get('message', "")
- AC (Acceptance Criteria)：
 - AC #1: 檢測到重複 email
 - AC #2: 返回 409 狀態碼
 - AC #3: 返回清晰的錯誤訊息
- 預期行為：系統拒絕重複郵箱註冊

測試結果：通過

3.3.2 TestAuthLogin - 用戶登入測試類

測試編號：IT-AUTH-003

測試目標：驗證用戶成功登入並獲取 Token

測試方法：端到端測試 (End-to-End Testing)

測試步驟：

1. 準備有效的登入憑證 (email, password)
2. 發送 POST 請求到 /api/auth/login

3. 驗證返回 access_token 和 refresh_token

測試角色：已註冊且已驗證的用戶

測試流程：

- 測試資料準備：
- from app.utils.security import hash_password
-
- user = User(
 - email='login@test.com',
 - username='loginuser',
 - password_hash=hash_password('Password123'),
 - role=UserRole.GENERAL_MEMBER,
 - verified=True)
- db_session.add(user)
- db_session.commit()
- Request Payload :
- {
 - "email": "login@test.com",
 - "password": "Password123"}
- HTTP 請求：POST /api/auth/login with content_type='application/json'
- 驗證斷言：
 - assert response.status_code == 200
 - assert 'access_token' in data
 - assert 'refresh_token' in data
 - assert data['user']['email'] == 'login@test.com'
- 預期行為：返回 JWT Token 和用戶資訊

測試結果：通過

測試編號：IT-AUTH-004

測試目標：驗證錯誤密碼無法登入

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 使用正確的 email 但錯誤的密碼
2. 嘗試登入
3. 驗證返回 401 Unauthorized

測試角色：訪客

測試流程：

- Fixture 準備：已註冊的用戶
- Request Payload :
- {
 - "email": "login@test.com",
 - "password": "WrongPassword"}

- }
- HTTP 請求：POST /api/auth/login with content_type='application/json'
- 驗證斷言：
 - assert response.status_code == 401
 - 錯誤訊息包含「密碼錯誤」或類似內容
- 預期行為：系統拒絕錯誤密碼登入

測試結果：通過

測試編號：IT-AUTH-005

測試目標：驗證未驗證用戶無法登入

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 創建未驗證的用戶 (verified=False)
2. 嘗試登入
3. 驗證返回錯誤狀態

測試角色：未驗證的註冊用戶

測試流程：

- 測試資料準備：
- `unverified_user = User(`
- `email='unverified@test.com',`
- `username='unverified',`
- `password_hash=hash_password('Password123'),`
- `role=UserRole.GENERAL_MEMBER,`
- `verified=False`
- `)`
- `db_session.add(unverified_user)`
- `db_session.commit()`
- Request Payload：
- {
- "email": "unverified@test.com",
- "password": "Password123"
- }
- HTTP 請求：POST /api/auth/login with content_type='application/json'
- 驗證斷言：
 - assert response.status_code in [401, 403]
 - 錯誤訊息包含「未驗證」或類似內容
- 預期行為：系統拒絕未驗證用戶登入

測試結果：通過

3.3.4 TestAuthMe - 當前用戶資訊測試類

測試編號：IT-AUTH-006

測試目標：驗證獲取當前登入用戶的資訊

測試方法：端到端測試 (End-to-End Testing)

測試步驟：

1. 使用有效的 Token 發送請求
2. 發送 GET 請求到 /api/auth/me
3. 驗證返回當前用戶的完整資訊

測試角色：已登入用戶

測試流程：

- Fixture 準備：
 - auth_headers - 有效的認證標頭
 - test_user - 當前登入用戶
- HTTP 請求：GET /api/auth/me with headers=auth_headers
- 驗證斷言：
 - assert response.status_code == 200
 - assert data['user_id'] == test_user.user_id
 - assert data['email'] == test_user.email
 - assert data['username'] == test_user.username
 - assert 'password_hash' not in data (密碼不應該被返回)
- 預期行為：返回當前用戶的個人資訊 (不包含敏感欄位)

測試結果：通過

測試編號：IT-AUTH-007

測試目標：驗證無 Token 無法獲取用戶資訊

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 不提供認證 Token
2. 嘗試獲取當前用戶資訊
3. 驗證返回 401 Unauthorized

測試角色：訪客 (未登入)

測試流程：

- HTTP 請求：GET /api/auth/me (無 auth headers)
- 驗證斷言：
 - assert response.status_code == 401
- 預期行為：未認證無法獲取用戶資訊

測試結果：通過

3.4 Medical Record 整合測試

test_medical_integration.py - 醫療記錄整合測試 (總共 6 個測試)。

3.4.1 TestMedicalRecordCreate - 創建醫療記錄測試類

測試編號：IT-MEDICAL-001

測試目標：驗證非收容所員工無法創建醫療記錄

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 使用一般用戶的 Token
2. 嘗試創建醫療記錄
3. 驗證返回 403 Forbidden

測試角色：一般會員

測試流程：

- Fixture 準備：
 - auth_headers - 一般用戶的認證標頭
 - test_shelter_animal - 收容所動物
- Request Payload：

```
{
  "record_type": "CHECKUP",
  "date": "2024-01-15",
  "provider": "測試醫院",
  "details": "測試創建"
}
```
- HTTP 請求：POST /api/medical-records/animals/{test_shelter_animal.animal_id}/medical-records with headers=auth_headers
- 驗證斷言：
 - assert response.status_code == 403
- 預期行為：一般用戶無權限創建醫療記錄

測試結果：通過

測試編號：IT-MEDICAL-002

測試目標：驗證為不存在的動物創建醫療記錄返回 404

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 使用不存在的 animal_id (99999)
2. 嘗試創建醫療記錄
3. 驗證返回 404 Not Found

測試角色：收容所員工

測試流程：

- Request Payload：

```
{
  "record_type": "CHECKUP",
  "date": "2024-01-15",

```

```
"provider": "測試醫院",  
"details": "測試"  
}
```

- HTTP 請求：POST /api/medical-records/animals/99999/medical-records with headers
- 驗證斷言：
 - assert response.status_code == 404
- 預期行為：找不到動物，返回 404 錯誤

測試結果：通過

3.4.2 TestMedicalRecordList - 醫療記錄列表測試類

測試編號：IT-MEDICAL-003

測試目標：驗證獲取動物的醫療記錄列表

測試方法：端到端測試 (End-to-End Testing)

測試步驟：

1. 創建測試醫療記錄
2. 發送 GET 請求獲取列表
3. 驗證返回包含醫療記錄

測試角色：已登入用戶

測試流程：

- 測試資料準備：

```
record = MedicalRecord(  
    medical_record_id=200,  
    animal_id=test_shelter_animal.animal_id,  
    record_type=RecordType.CHECKUP,  
    date=date(2024, 1, 15),  
    provider='測試醫院',  
    details='測試記錄',  
    verified=True,  
    verified_by=test_shelter_member.user_id  
)  
db_session.add(record)  
db_session.commit()
```
- HTTP 請求：GET /api/medical-records/animals/{test_shelter_animal.animal_id}/medical-records with headers=auth_headers
- 驗證斷言：
 - assert response.status_code == 200
 - assert 'medical_records' in data
 - assert len(data['medical_records']) > 0
- 預期行為：返回動物的醫療記錄列表

測試結果：通過

測試編號：IT-MEDICAL-004

測試目標：驗證帶過濾條件的醫療記錄列表

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 提供 record_type 查詢參數
2. 發送 GET 請求
3. 驗證返回的記錄符合指定類型

測試角色：已登入用戶

測試流程：

- HTTP 請求：GET /api/medical-records/animals/{test_shelter_animal.animal_id}/medical-records?record_type=VACCINE with headers=auth_headers
- 驗證斷言：
 - assert response.status_code == 200
 - assert 'medical_records' in data
- 預期行為：返回指定類型的醫療記錄

測試結果：通過

測試編號：IT-MEDICAL-005

測試目標：驗證只顯示已驗證的醫療記錄

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 提供 verified=true 查詢參數
2. 發送 GET 請求
3. 驗證所有返回的記錄都已驗證

測試角色：已登入用戶

測試流程：

- HTTP 請求：GET /api/medical-records/animals/{test_shelter_animal.animal_id}/medical-records?verified=true with headers=auth_headers
- 驗證斷言：
 - assert response.status_code == 200
 - assert 'medical_records' in data
 - for record in data['medical_records']: assert record['verified'] == True
- 預期行為：只返回已驗證的醫療記錄

測試結果：通過

3.4.3 TestMedicalRecordUpdate - 醫療記錄更新測試類

測試編號：IT-MEDICAL-006

測試目標：驗證 24 小時內可以更新醫療記錄

測試方法：邊界值分析 (Boundary Value Analysis)

測試步驟：

1. 創建剛創建的醫療記錄 (created_at=now)
2. 使用收容所員工的 Token 發送 PATCH 請求
3. 驗證記錄更新成功

測試角色：收容所員工 (記錄創建者)

測試流程：

- 測試資料準備：

```
from app.utils.datetime_helper import get_naive_taipei_now

record = MedicalRecord(
    medical_record_id=202,
    animal_id=test_shelter_animal.animal_id,
    record_type=RecordType.CHECKUP,
    date=date(2024, 1, 15),
    provider='原醫院',
    details='原記錄',
    verified=False,
    created_at=get_naive_taipei_now()
)
db_session.add(record)
db_session.commit()
```
- Request Payload：

```
{
  "details": "更新後的詳細信息",
  "provider": "更新後醫院"
}
```
- HTTP 請求：PATCH /api/medical-records/{record.medical_record_id} with headers
- 驗證斷言：
 - assert response.status_code == 200
 - assert 'medical_record' in data
 - assert data['medical_record']['details'] == '更新後的詳細信息'
- 資料庫驗證：
 - db_session.refresh(record)
 - assert record.provider == '更新後醫院'
- 預期行為：24 小時內可以更新記錄

測試結果：通過

3.5 Notification 整合測試

test_notification_integration.py - 通知系統整合測試 (總共 8 個測試)。

3.5.1 TestNotificationList - 通知列表測試類

測試編號：IT-NOTIFICATION-001

測試目標：驗證用戶可以獲取自己的通知列表

測試方法：端到端測試 (End-to-End Testing)

測試步驟：

1. 創建屬於當前用戶的測試通知
2. 使用認證 Token 發送 GET 請求到 /api/notifications
3. 驗證返回的通知都屬於當前用戶

測試角色：已登入用戶

測試流程：

- Fixture 準備測試用戶和認證標頭 (test_user, auth_headers)
- 創建通知記錄：

```
notification = Notification(
    notification_id=300,
    recipient_id=test_user.user_id,
    type='application_status',
    payload={'message': '您的申請已被審核'},
    read=False
)
db_session.add(notification)
db_session.commit()
```
- 發送 GET /api/notifications 帶 headers=auth_headers
- 驗證 response.status_code == 200
- 驗證 'notifications' in data
- 驗證 len(data['notifications']) > 0
- 遍歷所有通知驗證 notif['recipient_id'] == test_user.user_id

測試結果：通過

測試編號：IT-NOTIFICATION-002

測試目標：驗證按已讀狀態篩選通知（只顯示未讀）

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 創建一個未讀通知和一個已讀通知
2. 發送 GET 請求到 /api/notifications?read=false
3. 驗證返回的所有通知都是未讀狀態

測試角色：已登入用戶

測試流程：

- Fixture 準備測試用戶和認證標頭

- 創建未讀通知：


```
unread_notification = Notification(
    notification_id=301,
    recipient_id=test_user.user_id,
    type='animal_update',
    payload={'message': '動物資料已更新'},
    read=False
)
```
- 創建已讀通知：


```
read_notification = Notification(
    notification_id=302,
    recipient_id=test_user.user_id,
    type='system',
    payload={'message': '系統維護通知'},
    read=True,
    read_at=datetime.utcnow()
)
```
- 發送 GET /api/notifications?read=false 帶 headers=auth_headers
- 驗證 response.status_code == 200
- 遍歷 data['notifications'] 驗證每個 notif['read'] == False

測試結果：通過

測試編號：IT-NOTIFICATION-003

測試目標：驗證通知列表分頁功能

測試方法：邊界值分析 (Boundary Value Analysis)

測試步驟：

1. 發送帶分頁參數的 GET 請求到 /api/notifications?page=1&per_page=10
2. 驗證回應包含分頁資訊 (page, per_page, total)

測試角色：已登入用戶

測試流程：

- Fixture 準備認證標頭
- 發送 GET /api/notifications?page=1&per_page=10 帶 headers=auth_headers
- 驗證 response.status_code == 200
- 驗證 'notifications' in data
- 驗證 'total' in data
- 驗證 'page' in data
- 驗證 'per_page' in data

測試結果：通過

3.5.2 TestNotificationMarkRead - 標記通知已讀測試類

測試編號：IT-NOTIFICATION-004

測試目標：驗證用戶可以標記通知為已讀

測試方法：端到端測試 (End-to-End Testing)

測試步驟：

1. 創建未讀通知
2. 發送 POST 請求到 /api/notifications/{notification_id}/mark-read
3. 驗證通知被標記為已讀且設置了 read_at 時間

測試角色：已登入用戶

測試流程：

- Fixture 準備測試用戶和認證標頭
- 創建未讀通知：

```
notification = Notification(
    notification_id=307,
    recipient_id=test_user.user_id,
    type='animal_update',
    payload={'message': '動物資料已更新'},
    read=False
)
db_session.add(notification)
db_session.commit()
```
- 發送 POST /api/notifications/{notification.notification_id}/mark-read 帶 headers=auth_headers
- 驗證 response.status_code == 200
- 刷新資料庫記錄：db_session.refresh(notification)
- 驗證 notification.read == True
- 驗證 notification.read_at is not None

測試結果：通過

測試編號：IT-NOTIFICATION-005

測試目標：驗證用戶可以標記所有通知為已讀

測試方法：端到端測試 (End-to-End Testing)

測試步驟：

1. 創建多個未讀通知
2. 發送 POST 請求到 /api/notifications/mark-all-read
3. 驗證所有通知都被標記為已讀

測試角色：已登入用戶

測試流程：

- Fixture 準備測試用戶和認證標頭
- 使用迴圈創建 3 個未讀通知：

```
for i in range(3):
    notification = Notification(
        notification_id=309 + i, # 309, 310, 311
```

- ```

 recipient_id=test_user.user_id,
 type='system',
 payload={'message': f'通知 {i}'},
 read=False
)
 db_session.add(notification)
 db_session.commit()

```
- 發送 POST /api/notifications/mark-all-read 帶 headers=auth\_headers
  - 驗證 response.status\_code == 200
  - 查詢未讀通知數量：
  - unread\_count = db\_session.query(Notification).filter\_by(
  - recipient\_id=test\_user.user\_id,
  - read=False
  - ).count()
  - 驗證 unread\_count == 0

測試結果：通過

---

### 3.5.3 TestNotificationDelete - 刪除通知測試類

---

測試編號：IT-NOTIFICATION-006

測試目標：驗證用戶可以刪除自己的通知

測試方法：端到端測試 (End-to-End Testing)

測試步驟：

1. 創建屬於當前用戶的通知
2. 發送 DELETE 請求到 /api/notifications/{notification\_id}
3. 驗證通知被成功刪除

測試角色：已登入用戶

測試流程：

- Fixture 準備測試用戶和認證標頭
- 創建通知記錄：
 

```

notification = Notification(
 notification_id=312,
 recipient_id=test_user.user_id,
 type='system',
 payload={'message': '待刪除通知'},
 read=True
)
db_session.add(notification)
db_session.commit()
notification_id = notification.notification_id

```
- 發送 DELETE /api/notifications/{notification\_id} 帶 headers=auth\_headers
- 驗證 response.status\_code == 200

- 查詢資料庫驗證通知已被刪除：

```
deleted_notification = db_session.query(Notification).filter_by(
 notification_id=notification_id
).first()
```
- 驗證 `deleted_notification` is `None`

測試結果：通過

---

### 3.5.4 TestNotificationCreation - 通知創建測試類

---

測試編號：IT-NOTIFICATION-007

測試目標：驗證動物資料更新時通知關注者

測試方法：端到端測試 (End-to-End Testing)

測試步驟：

1. 更新動物資料
2. 驗證系統可能異步創建通知給關注者

測試角色：收容所成員

測試流程：

- Fixture 準備收容所動物和收容所成員 (`test_shelter_animal`, `test_shelter_member`)
- 創建認證 Token：

```
token = create_access_token(identity=test_shelter_member.user_id)
headers = {
 'Authorization': f'Bearer {token}',
 'Content-Type': 'application/json'
}
```
- 更新動物描述：

```
{
 "description": "更新後的描述"
}
```
- 發送 PATCH `/api/animals/{test_shelter_animal.animal_id}` 帶 headers
- 驗證 `response.status_code == 200`
- 註記：通知系統可能會異步處理，這裡只驗證請求成功

測試結果：通過

---

### 3.5.5 TestNotificationCount - 通知計數測試類

---

測試編號：IT-NOTIFICATION-008

測試目標：驗證用戶可以獲取未讀通知數量

測試方法：端到端測試 (End-to-End Testing)

測試步驟：

1. 創建多個未讀和已讀通知

2. 發送 GET 請求到 /api/notifications/unread-count
3. 驗證返回正確的未讀通知數量

**測試角色：**已登入用戶

**測試流程：**

- Fixture 準備測試用戶和認證標頭
- 創建 5 個未讀通知：

```
for i in range(5):
 notification = Notification(
 notification_id=319 + i, # 319-323
 recipient_id=test_user.user_id,
 type='system',
 payload={'message': f'通知 {i}'},
 read=False
)
 db_session.add(notification)
```
- 創建 1 個已讀通知：

```
read_notification = Notification(
 notification_id=324,
 recipient_id=test_user.user_id,
 type='system',
 payload={'message': '已讀'},
 read=True
)
```
- 發送 GET /api/notifications/unread-count 帶 headers=auth\_headers
- 驗證 response.status\_code == 200
- 驗證 'unread\_count' in data
- 驗證 data['unread\_count'] == 5

**測試結果：**通過

---

## 3.6 Shelter 整合測試

test\_shelter\_integration.py - 收容所管理整合測試 (總共 10 個測試)。

---

### 3.6.1 TestShelterCreate - 收容所創建測試類

---

**測試編號：**IT-SHELTER-001

**測試目標：**驗證創建收容所需要認證

**測試方法：**錯誤推測 (Error Guessing)

**測試步驟：**

1. 不提供認證 Token
2. 發送 POST 請求嘗試創建收容所
3. 驗證返回 401 Unauthorized 錯誤

**測試角色：**訪客（未登入）

**測試流程：**

- 準備收容所資料：

```
{
 "name": "未認證收容所",
 "contact_email": "test@test.com",
 "contact_phone": "02-1111-1111",
 "address": {
 "city": "台北市"
 }
}
```
- 發送 POST /api/shelters 帶 data=json 但不帶認證 headers
- 驗證 response.status\_code == 401

**測試結果：**通過

---

### 3.6.2 TestShelterList - 收容所列表測試類

---

**測試編號：**IT-SHELTER-002

**測試目標：**驗證獲取已驗證的收容所列表

**測試方法：**端到端測試 (End-to-End Testing)

**測試步驟：**

1. 確保測試收容所已驗證
2. 發送 GET 請求到 /api/shelters
3. 驗證返回的所有收容所都是已驗證狀態

**測試角色：**已登入用戶

**測試流程：**

- Fixture 準備測試收容所和認證標頭 (test\_shelter, auth\_headers)
- 設置收容所為已驗證：

```
test_shelter.verified = True
db_session.commit()
```
- 發送 GET /api/shelters 帶 headers=auth\_headers
- 驗證 response.status\_code == 200
- 驗證 'shelters' in data
- 驗證 len(data['shelters']) > 0

- 遍歷所有收容所驗證 shelter['verified'] == True

測試結果：通過

---

測試編號：IT-SHELTER-003

測試目標：驗證按地區篩選收容所功能

測試方法：等價劃分 (Equivalence Partitioning)

測試步驟：

1. 發送帶地區篩選參數的 GET 請求到 /api/shelters?region=台北
2. 驗證返回的收容所符合地區條件

測試角色：已登入用戶

測試流程：

- Fixture 準備認證標頭
- 發送 GET /api/shelters?region=台北 帶 headers=auth\_headers
- 驗證 response.status\_code == 200
- 驗證 'shelters' in data

測試結果：通過

---

測試編號：IT-SHELTER-004

測試目標：驗證收容所列表分頁功能

測試方法：邊界值分析 (Boundary Value Analysis)

測試步驟：

1. 發送帶分頁參數的 GET 請求到 /api/shelters?page=1&per\_page=10
2. 驗證回應包含分頁資訊

測試角色：已登入用戶

測試流程：

- Fixture 準備認證標頭
- 發送 GET /api/shelters?page=1&per\_page=10 帶 headers=auth\_headers
- 驗證 response.status\_code == 200
- 驗證 'shelters' in data
- 驗證 'total' in data
- 驗證 'page' in data
- 驗證 'per\_page' in data

測試結果：通過

---

### 3.6.3 TestShelterDetail - 收容所詳情測試類

---

測試編號：IT-SHELTER-005

**測試目標：**驗證獲取收容所詳情

**測試方法：**端到端測試 (End-to-End Testing)

**測試步驟：**

1. 確保測試收容所已驗證
2. 發送 GET 請求到 /api/shelters/{shelter\_id}
3. 驗證返回完整的收容所資訊

**測試角色：**已登入用戶

**測試流程：**

- Fixture 準備測試收容所和認證標頭 (test\_shelter, auth\_headers)
- 設置收容所為已驗證：  
test\_shelter.verified = True  
db\_session.commit()
- 發送 GET /api/shelters/{test\_shelter.shelter\_id} 帶 headers=auth\_headers
- 驗證 response.status\_code == 200
- 驗證 data['shelter\_id'] == test\_shelter.shelter\_id
- 驗證 data['name'] == test\_shelter.name

**測試結果：**通過

---

**測試編號：**IT-SHELTER-006

**測試目標：**驗證獲取不存在的收容所返回 404

**測試方法：**邊界值分析 (Boundary Value Analysis)

**測試步驟：**

1. 發送 GET 請求到不存在的收容所 ID
2. 驗證返回 404 Not Found 錯誤

**測試角色：**已登入用戶

**測試流程：**

- Fixture 準備認證標頭
- 發送 GET /api/shelters/99999 帶 headers=auth\_headers
- 驗證 response.status\_code == 404

**測試結果：**通過

---

### 3.6.4 TestShelterUpdate - 收容所更新測試類

---

**測試編號：**IT-SHELTER-007

**測試目標：**驗證非收容所成員無法更新收容所資料

**測試方法：**錯誤推測 (Error Guessing)

**測試步驟：**

1. 使用非收容所成員的認證 Token
2. 嘗試更新收容所資料



### 3. 驗證返回 403 Forbidden 錯誤

**測試角色：**非收容所成員的已登入用戶

**測試流程：**

- Fixture 準備測試收容所和普通用戶認證標頭 (test\_shelter, auth\_headers)
- 準備更新資料：

```
{
 "description": "試圖更新"
}
```
- 發送 PATCH /api/shelters/{test\_shelter.shelter\_id} 帶 data=json 和 headers=auth\_headers
- 驗證 response.status\_code == 403

**測試結果：**通過

---

## 3.6.5 TestShelterVerification - 收容所驗證測試類

---

**測試編號：**IT-SHELTER-008

**測試目標：**驗證管理員可以驗證收容所

**測試方法：**端到端測試 (End-to-End Testing)

**測試步驟：**

1. 使用管理員認證 Token
2. 發送驗證請求 (verified=True)
3. 驗證收容所狀態更新為已驗證

**測試角色：**系統管理員

**測試流程：**

- Fixture 準備測試收容所和管理員認證標頭 (test\_shelter, admin\_headers)
- 準備驗證資料：

```
{
 "verified": true,
 "notes": "資料完整，已驗證"
}
```
- 發送 POST /api/shelters/{test\_shelter.shelter\_id}/verify 帶 data=json 和 headers=admin\_headers
- 驗證 response.status\_code == 200
- 刷新資料庫記錄：db\_session.refresh(test\_shelter)
- 驗證 test\_shelter.verified == True

**測試結果：**通過

---

**測試編號：**IT-SHELTER-009

**測試目標：**驗證管理員可以拒絕驗證收容所

**測試方法：**端到端測試 (End-to-End Testing)

**測試步驟：**

1. 使用管理員認證 Token
2. 發送拒絕驗證請求 (verified=False)
3. 驗證收容所狀態保持未驗證

**測試角色：**系統管理員

**測試流程：**

- Fixture 準備測試收容所和管理員認證標頭 (test\_shelter, admin\_headers)
- 準備拒絕驗證資料：

```
{
 "verified": false,
 "notes": "資料不完整"
}
```
- 發送 POST /api/shelters/{test\_shelter.shelter\_id}/verify 帶 data=json 和 headers=admin\_headers
- 驗證 response.status\_code == 200
- 刷新資料庫記錄：db\_session.refresh(test\_shelter)
- 驗證 test\_shelter.verified == False

**測試結果：**通過

---

**測試編號：**IT-SHELTER-010

**測試目標：**驗證非管理員無法驗證收容所

**測試方法：**錯誤推測 (Error Guessing)

**測試步驟：**

1. 使用普通用戶認證 Token
2. 嘗試驗證收容所
3. 驗證返回 403 Forbidden 錯誤

**測試角色：**普通用戶

**測試流程：**

- Fixture 準備測試收容所和普通用戶認證標頭 (test\_shelter, auth\_headers)
- 準備驗證資料：

```
{
 "verified": true
}
```
- 發送 POST /api/shelters/{test\_shelter.shelter\_id}/verify 帶 data=json 和 headers=auth\_headers
- 驗證 response.status\_code == 403

**測試結果：**通過

---

## 3.7 User 整合測試

**test\_user\_integration.py** - 用戶管理整合測試 (11 個測試)。

---

### 3.7.1 TestUserProfile - 用戶個人資料測試類

---

**測試編號：**IT-USER-001

**測試目標：**驗證用戶可以獲取自己的完整個人資料

**測試方法：**端到端測試 (End-to-End Testing)

**測試步驟：**

1. 使用認證 Token 發送 GET 請求到 `/api/users/{user_id}`
2. 驗證返回完整的個人資料 (`user_id`, `email`, `username`)

**測試角色：**已登入用戶

**測試流程：**

- Fixture 準備測試用戶和認證標頭 (`test_user`, `auth_headers`)
- 發送 GET `/api/users/{test_user.user_id}` 帶 `headers=auth_headers`
- 驗證 `response.status_code == 200`
- 驗證 `data['user_id'] == test_user.user_id`
- 驗證 `data['email'] == test_user.email`
- 驗證 `data['username'] == test_user.username`

**測試結果：**通過

---

### 3.7.2 TestUserUpdate - 用戶更新測試類

---

**測試編號：**IT-USER-002

**測試目標：**驗證用戶可以更新自己的個人資料

**測試方法：**端到端測試 (End-to-End Testing)

**測試步驟：**

1. 準備更新資料 (`username`, `bio`)
2. 發送 PATCH 請求到 `/api/users/{user_id}`
3. 驗證資料更新成功

**測試角色：**已登入用戶

**測試流程：**

- Fixture 準備測試用戶和認證標頭 (`test_user`, `auth_headers`)
- 準備更新資料：

```
{
 "username": "updated_username",
 "bio": "Updated bio"
}
```

- }  
發送 PATCH /api/users/{test\_user.user\_id} 帶 data=json 和 headers=auth\_headers
- 驗證 response.status\_code == 200
- 驗證 data['username'] == 'updated\_username'
- 刷新資料庫記錄：db\_session.refresh(test\_user)
- 驗證 test\_user.username == 'updated\_username'

測試結果：通過

---

測試編號：IT-USER-003

測試目標：驗證更新 email 需要重新驗證

測試方法：端到端測試 (End-to-End Testing)

測試步驟：

1. 準備新的 email 地址
2. 發送 PATCH 請求更新 email
3. 驗證返回成功但可能需要驗證 (200 或 202 Accepted)

測試角色：已登入用戶

測試流程：

- Fixture 準備測試用戶和認證標頭 (test\_user, auth\_headers)
- 準備更新資料：
  - {  
"email": "newemail@test.com"  
}
- 發送 PATCH /api/users/{test\_user.user\_id} 帶 data=json 和 headers=auth\_headers
- 驗證 response.status\_code in [200, 202]
  - 200: 更新成功但標記為未驗證
  - 202: 已接受請求，需要郵件驗證

測試結果：通過

---

測試編號：IT-USER-004

測試目標：驗證用戶無法更新其他用戶的資料

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 創建另一個用戶
2. 使用當前用戶身份嘗試更新該用戶資料
3. 驗證返回 403 Forbidden 錯誤

測試角色：已登入用戶

測試流程：

- Fixture 準備認證標頭
- 創建其他用戶：
 

```
from app.utils.security import hash_password

other_user = User(
```

- ```

        email='other6@test.com',
        username='other6',
        password_hash=hash_password('Password123'),
        role=UserRole.GENERAL_MEMBER,
        verified=True
    )
    db_session.add(other_user)
    db_session.commit()

```
- 準備更新資料：


```

      {
        "username": "hacked"
      }

```
 - 發送 PATCH /api/users/{other_user.user_id} 帶 data=json 和 headers=auth_headers
 - 驗證 response.status_code == 403

測試結果：通過

3.7.3 TestChangePassword - 修改密碼測試類

測試編號：IT-USER-005

測試目標：驗證用戶成功修改密碼

測試方法：端到端測試 (End-to-End Testing)

測試步驟：

1. 準備舊密碼和新密碼
2. 發送 PATCH 請求到 /api/users/{user_id}/password
3. 驗證密碼修改成功

測試角色：已登入用戶

測試流程：

- Fixture 準備測試用戶
- 設置舊密碼：


```

from flask_jwt_extended import create_access_token
from app.utils.security import hash_password

test_user.password_hash = hash_password('OldPassword123')
db_session.commit()

token = create_access_token(identity=test_user.user_id)
headers = {
    'Authorization': f'Bearer {token}',
    'Content-Type': 'application/json'
}

```
- 準備修改密碼資料：


```

{
    "old_password": "OldPassword123",

```

```
        "new_password": "NewPassword123"
    }
}
```

- 發送 PATCH /api/users/{test_user.user_id}/password 帶 data=json 和 headers
- 驗證 response.status_code == 200

測試結果：通過

測試編號：IT-USER-006

測試目標：驗證舊密碼錯誤時無法修改密碼

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 準備錯誤的舊密碼
2. 嘗試修改密碼
3. 驗證返回 400 或 401 錯誤

測試角色：已登入用戶

測試流程：

- Fixture 準備測試用戶和認證標頭 (test_user, auth_headers)
- 準備修改密碼資料：

```
{
    "old_password": "WrongPassword",
    "new_password": "NewPassword123"
}
```
- 發送 PATCH /api/users/{test_user.user_id}/password 帶 data=json 和 headers=auth_headers
- 驗證 response.status_code in [400, 401]

測試結果：通過

TestAdminUserManagement - 管理員用戶管理測試類

測試編號：IT-USER-007

測試目標：驗證管理員可以獲取用戶列表

測試方法：端到端測試 (End-to-End Testing)

測試步驟：

1. 使用管理員認證 Token
2. 發送 GET 請求到 /api/admin/users
3. 驗證返回用戶列表和總數

測試角色：系統管理員

測試流程：

- Fixture 準備管理員用戶和認證標頭 (test_admin, admin_headers)
- 發送 GET /api/admin/users 帶 headers=admin_headers
- 驗證 response.status_code == 200

- 驗證 'users' in data
- 驗證 'total' in data

測試結果：通過

測試編號：IT-USER-008

測試目標：驗證管理員可以搜尋用戶

測試方法：端到端測試 (End-to-End Testing)

測試步驟：

1. 使用管理員認證 Token
2. 發送帶搜尋參數的 GET 請求到 `/api/admin/users?search=test`
3. 驗證返回符合條件的用戶列表

測試角色：系統管理員

測試流程：

- Fixture 準備管理員認證標頭 (`admin_headers`)
- 發送 GET `/api/admin/users?search=test` 帶 `headers=admin_headers`
- 驗證 `response.status_code == 200`
- 驗證 'users' in data

測試結果：通過

測試編號：IT-USER-009

測試目標：驗證管理員可以封禁用戶

測試方法：端到端測試 (End-to-End Testing)

測試步驟：

1. 使用管理員認證 Token
2. 準備封禁資料 (`reason, duration`)
3. 發送 POST 請求到 `/api/admin/users/{user_id}/ban`
4. 驗證用戶被封禁且設置 `locked_until` 時間

測試角色：系統管理員

測試流程：

- Fixture 準備管理員認證標頭和測試用戶 (`admin_headers, test_user`)
- 準備封禁資料：

```
{
  "reason": "違反使用條款",
  "duration": 7
}
```
- 發送 POST `/api/admin/users/{test_user.user_id}/ban` 帶 `data=json` 和 `headers=admin_headers`
- 驗證 `response.status_code == 200`
- 刷新資料庫記錄：`db_session.refresh(test_user)`
- 驗證 `test_user.locked_until` is not None

測試結果：通過

測試編號：IT-USER-010

測試目標：驗證管理員可以解除用戶封禁

測試方法：端到端測試 (End-to-End Testing)

測試步驟：

1. 先封禁用戶（設置 locked_until）
2. 使用管理員身份發送解除封禁請求
3. 驗證 locked_until 被清除

測試角色：系統管理員

測試流程：

- Fixture 準備管理員認證標頭和測試用戶（admin_headers, test_user）
- 先封禁用戶：

```
from datetime import datetime, timedelta

test_user.locked_until = datetime.utcnow() + timedelta(days=7)
db_session.commit()
```
- 發送 POST /api/admin/users/{test_user.user_id}/unban 帶 headers=admin_headers
- 驗證 response.status_code == 200
- 刷新資料庫記錄：db_session.refresh(test_user)
- 驗證 test_user.locked_until is None

測試結果：通過

測試編號：IT-USER-011

測試目標：驗證非管理員無法封禁用戶

測試方法：錯誤推測 (Error Guessing)

測試步驟：

1. 使用普通用戶身份
2. 嘗試封禁其他用戶
3. 驗證返回 403 Forbidden 錯誤

測試角色：普通用戶

測試流程：

- Fixture 準備普通用戶認證標頭（auth_headers）
- 創建目標用戶：

```
from app.utils.security import hash_password

target_user = User(
    email='target@test.com',
    username='target',
    password_hash=hash_password('Password123'),
    role=UserRole.GENERAL_MEMBER,
    verified=True
)
```


- ```
db_session.add(target_user)
db_session.commit()
```
- 準備封禁資料：
 

```
{
 "reason": "嘗試封禁"
}
```
  - 發送 POST /api/admin/users/{target\_user.user\_id}/ban 帶 data=json 和 headers=auth\_headers
  - 驗證 response.status\_code == 403

測試結果：通過

## 3.8 整合測試結果

### 3.8.1 Animal

collected 34 items

```
tests/integration/test_animal_integration.py::TestAnimalList::test_get_animals_list PASSED [2%]
tests/integration/test_animal_integration.py::TestAnimalList::test_get_animals_list_with_auth PASSED [5%]
tests/integration/test_animal_integration.py::TestAnimalList::test_filter_animals_by_species PASSED [8%]
tests/integration/test_animal_integration.py::TestAnimalList::test_filter_animals_by_sex PASSED [11%]
tests/integration/test_animal_integration.py::TestAnimalList::test_pagination PASSED [14%]
tests/integration/test_animal_integration.py::TestAnimalListFilters::test_filter_by_status PASSED [17%]
tests/integration/test_animal_integration.py::TestAnimalListFilters::test_filter_by_region PASSED [20%]
tests/integration/test_animal_integration.py::TestAnimalListFilters::test_search_query PASSED [23%]
tests/integration/test_animal_integration.py::TestAnimalCreate::test_create_animal_without_auth PASSED [26%]
tests/integration/test_animal_integration.py::TestAnimalDetail::test_get_animal_detail PASSED [29%]
tests/integration/test_animal_integration.py::TestAnimalDetail::test_get_nonexistent_animal PASSED [32%]
tests/integration/test_animal_integration.py::TestAnimalGetDetail::test_get_animal_with_relations PASSED [35%]
tests/integration/test_animal_integration.py::TestAnimalGetDetail::test_get_draft_animal_as_owner PASSED [38%]
tests/integration/test_animal_integration.py::TestAnimalUpdate::test_update_own_animal PASSED [41%]
tests/integration/test_animal_integration.py::TestAnimalUpdate::test_update_others_animal PASSED [44%]
tests/integration/test_animal_integration.py::TestAnimalDelete::test_delete_own_animal PASSED [47%]
tests/integration/test_animal_integration.py::TestAnimalDelete::test_delete_without_permission PASSED [50%]
tests/integration/test_animal_integration.py::TestAnimalImages::test_add_image_without_required_fields PASSED [52%]
tests/integration/test_animal_integration.py::TestAnimalImagePermissions::test_add_image_without_auth PASSED [55%]
tests/integration/test_animal_integration.py::TestAnimalImagePermissions::test_delete_image_without_auth PASSED [58%]
tests/integration/test_animal_integration.py::TestAnimalImagePermissions::test_reorder_images_without_auth PASSED [61%]
tests/integration/test_animal_integration.py::TestAnimalImagePermissions::test_delete_animal_image PASSED [64%]
tests/integration/test_animal_integration.py::TestAnimalImagePermissions::test_reorder_animal_images PASSED [67%]
tests/integration/test_animal_integration.py::TestAnimalStatusManagement::test_submit_animal_for_review PASSED [70%]
tests/integration/test_animal_integration.py::TestAnimalStatusManagement::test_publish_animal_as_admin PASSED [73%]
tests/integration/test_animal_integration.py::TestAnimalStatusManagement::test_retire_animal PASSED [76%]
tests/integration/test_animal_integration.py::TestAnimalStatusManagement::test_reject_animal_as_admin PASSED [79%]
tests/integration/test_animal_integration.py::TestAnimalStatusManagement::test_reject_without_reason PASSED [82%]
tests/integration/test_animal_integration.py::TestAnimalStatusPermissions::test_submit_without_auth PASSED [85%]
tests/integration/test_animal_integration.py::TestAnimalStatusPermissions::test_publish_without_auth PASSED [88%]
tests/integration/test_animal_integration.py::TestAnimalStatusPermissions::test_retire_without_auth PASSED [91%]
tests/integration/test_animal_integration.py::TestAnimalStatusPermissions::test_reject_without_auth PASSED [94%]
tests/integration/test_animal_integration.py::TestAnimalEdgeCases::test_get_nonexistent_animal_for_update PASSED [97%]
tests/integration/test_animal_integration.py::TestAnimalEdgeCases::test_delete_nonexistent_animal PASSED [100%]
```

===== 34 passed, 40 warnings in 19.39s =====

### 3.8.2 Application

collected 12 items

```
tests/integration/test_application_integration.py::TestApplicationCreate::test_create_adoption_application_success PASSED [8%]
tests/integration/test_application_integration.py::TestApplicationCreate::test_create_application_without_auth PASSED [16%]
tests/integration/test_application_integration.py::TestApplicationCreate::test_create_application_for_nonexistent_animal PASSED [25%]
tests/integration/test_application_integration.py::TestApplicationCreate::test_create_duplicate_application PASSED [33%]
tests/integration/test_application_integration.py::TestApplicationList::test_get_my_applications PASSED [41%]
tests/integration/test_application_integration.py::TestApplicationList::test_filter_applications_by_status PASSED [50%]
tests/integration/test_application_integration.py::TestApplicationReview::test_approve_application_as_owner PASSED [58%]
tests/integration/test_application_integration.py::TestApplicationReview::test_reject_application_with_reason PASSED [66%]
tests/integration/test_application_integration.py::TestApplicationReview::test_review_application_without_permission PASSED [75%]
tests/integration/test_application_integration.py::TestApplicationDetail::test_get_own_application_detail PASSED [83%]
tests/integration/test_application_integration.py::TestApplicationDetail::test_get_application_detail_as_animal_owner PASSED [91%]
tests/integration/test_application_integration.py::TestApplicationDetail::test_get_application_without_permission PASSED [100%]
```

===== 12 passed, 20 warnings in 12.82s =====

### 3.8.3 Auth

collected 7 items

```
tests/integration/test_auth_integration.py::TestAuthRegistration::test_register_new_user_success PASSED [14%]
tests/integration/test_auth_integration.py::TestAuthRegistration::test_register_duplicate_email PASSED [28%]
tests/integration/test_auth_integration.py::TestAuthLogin::test_login_success PASSED [42%]
tests/integration/test_auth_integration.py::TestAuthLogin::test_login_wrong_password PASSED [57%]
tests/integration/test_auth_integration.py::TestAuthLogin::test_login_unverified_user PASSED [71%]
tests/integration/test_auth_integration.py::TestAuthMe::test_get_current_user_info PASSED [85%]
tests/integration/test_auth_integration.py::TestAuthMe::test_get_user_info_without_token PASSED [100%]
```

===== 7 passed, 12 warnings in 4.96s =====

### 3.8.4 Medical

collected 6 items

```
tests/integration/test_medical_integration.py::TestMedicalRecordCreate::test_create_medical_record_as_non_member PASSED [16%]
tests/integration/test_medical_integration.py::TestMedicalRecordCreate::test_create_medical_record_for_nonexistent_animal PASSED [33%]
tests/integration/test_medical_integration.py::TestMedicalRecordList::test_list_medical_records PASSED [50%]
tests/integration/test_medical_integration.py::TestMedicalRecordList::test_list_medical_records_with_filter PASSED [66%]
tests/integration/test_medical_integration.py::TestMedicalRecordList::test_list_medical_records_verified_only PASSED [83%]
tests/integration/test_medical_integration.py::TestMedicalRecordUpdate::test_update_medical_record_within_24_hours PASSED [100%]
```

===== 6 passed, 10 warnings in 3.39s =====

### 3.8.5 Notification

collected 8 items

```
tests/integration/test_notification_integration.py::TestNotificationList::test_list_own_notifications PASSED [12%]
tests/integration/test_notification_integration.py::TestNotificationList::test_list_unread_notifications PASSED [25%]
tests/integration/test_notification_integration.py::TestNotificationList::test_list_notifications_with_pagination PASSED [37%]
tests/integration/test_notification_integration.py::TestNotificationMarkRead::test_mark_notification_as_read PASSED [50%]
tests/integration/test_notification_integration.py::TestNotificationMarkRead::test_mark_all_notifications_as_read PASSED [62%]
tests/integration/test_notification_integration.py::TestNotificationDelete::test_delete_own_notification PASSED [75%]
tests/integration/test_notification_integration.py::TestNotificationCreation::test_notification_created_on_animal_update PASSED [87%]
tests/integration/test_notification_integration.py::TestNotificationCount::test_get_unread_notification_count PASSED [100%]
```

===== 8 passed, 12 warnings in 5.02s =====

### 3.8.6 Shelter

collected 10 items

```
tests/integration/test_shelter_integration.py::TestShelterCreate::test_create_shelter_requires_authentication PASSED [10%]
tests/integration/test_shelter_integration.py::TestShelterList::test_list_verified_shelters PASSED [20%]
tests/integration/test_shelter_integration.py::TestShelterList::test_list_shelters_with_filter PASSED [30%]
tests/integration/test_shelter_integration.py::TestShelterList::test_list_shelters_with_pagination PASSED [40%]
tests/integration/test_shelter_integration.py::TestShelterDetail::test_get_shelter_detail PASSED [50%]
tests/integration/test_shelter_integration.py::TestShelterDetail::test_get_nonexistent_shelter PASSED [60%]
tests/integration/test_shelter_integration.py::TestShelterUpdate::test_update_shelter_as_non_member PASSED [70%]
tests/integration/test_shelter_integration.py::TestShelterVerification::test_admin_verify_shelter PASSED [80%]
tests/integration/test_shelter_integration.py::TestShelterVerification::test_admin_reject_shelter PASSED [90%]
tests/integration/test_shelter_integration.py::TestShelterVerification::test_non_admin_cannot_verify_shelter PASSED [100%]
```

===== 10 passed, 13 warnings in 4.48s =====

### 3.8.7 User

collected 11 items

```
tests/integration/test_user_integration.py::TestUserProfile::test_get_own_profile PASSED [9%]
tests/integration/test_user_integration.py::TestUserProfile::test_update_own_profile PASSED [18%]
tests/integration/test_user_integration.py::TestUserUpdate::test_update_email_requires_verification PASSED [27%]
tests/integration/test_user_integration.py::TestUserUpdate::test_update_other_user_profile PASSED [36%]
tests/integration/test_user_integration.py::TestChangePassword::test_change_password_success PASSED [45%]
tests/integration/test_user_integration.py::TestChangePassword::test_change_password_wrong_old_password PASSED [54%]
tests/integration/test_user_integration.py::TestAdminUserManagement::test_admin_list_users PASSED [63%]
tests/integration/test_user_integration.py::TestAdminUserManagement::test_admin_search_users PASSED [72%]
tests/integration/test_user_integration.py::TestAdminUserManagement::test_admin_ban_user PASSED [81%]
tests/integration/test_user_integration.py::TestAdminUserManagement::test_admin_unban_user PASSED [90%]
tests/integration/test_user_integration.py::TestAdminUserManagement::test_non_admin_cannot_ban_user PASSED [100%]
```

===== 11 passed, 20 warnings in 8.19s =====

### 3.8.8 覆蓋率

整合測試目前共有 88 個測試，全部通過（100% 通過率）。從使用案例層面來看，54% 的使用案例達到高涵蓋標準。核心功能已有良好的測試保護，但管理功能和部分進階功能（Email 驗證、批次匯入、資料審核）仍需補充。

- 總測試數：88 個
- 測試通過率：100%
- 高涵蓋案例：13/24 (54%)
- 中涵蓋案例：7/24 (29%)
- 低涵蓋案例：4/24 (17%)

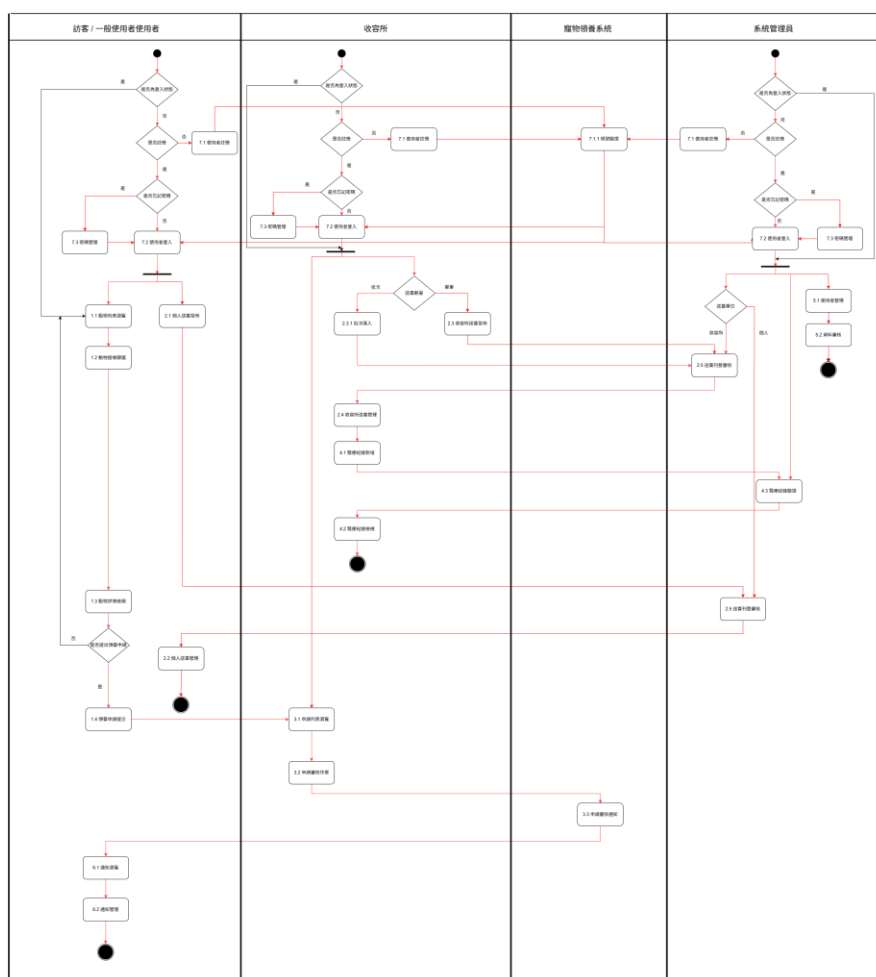
註：可以參考 [IntegrationCoverage.md](#)

## 4. E2E 測試

## 4.1 執行摘要

記錄寵物領養平台的 E2E 測試框架、測試案例、執行結果。依據實際業務流程「收容所上傳 → 管理員批准 → 用戶領養 → 收容所審核 → 用戶查看」，每個流程皆執行一次 E2E，並以 Playwright + Page Object Model (POM) 撰寫，可維護且可擴展。

| 項目   | 詳情                      |
|------|-------------------------|
| 測試框架 | Playwright 1.40+        |
| 測試語言 | TypeScript              |
| 測試模式 | Page Object Model (POM) |
| 測試環境 | 本地開發環境                  |



## 4.2 測試目標與範圍

### 目標

1. 驗證關鍵業務流程的完整性
2. 確保應用程式的基本可用性
3. 檢測 JavaScript 執行期錯誤
4. 確保網路資源正確載入

| 測試範圍      | 模組    | 測試類型  |
|-----------|-------|-------|
| 收容所上傳刊登   | 刊登管理  | 端對端   |
| 管理員批准刊登   | 刊登審核  | 端對端   |
| 用戶領養申請    | 領養流程  | 端對端   |
| 收容所審核用戶申請 | 申請審核  | 端對端   |
| 用戶查看申請／進度 | 申請查詢  | 端對端   |
| 表單驗證與錯誤降級 | 表單處理  | 功能／恢復 |
| 表單驗證      | 表單處理  | 功能測試  |
| 照片上傳      | 媒體管理  | 功能測試  |
| 醫療紀錄      | 寵物資料  | 功能測試  |
| 草稿儲存      | 刊登管理  | 功能測試  |
| 響應式佈局     | 前端介面  | UI 測試 |
| 網路錯誤處理    | 系統穩定性 | 非功能測試 |

## 4.3 測試流程

### 主要測試流程：

收容所上傳(採取批次或單筆)/使用者發布送養 → 管理員批准 → 用戶領養 → 收容所審核 → 用戶查看

涵蓋使用案例依序如下：

### 測試案例 7.2：使用者登入

**目的：**各使用者能以有效帳號登入，無效憑證被阻擋並提示錯誤，登入後導向預期頁面且會話/權限正常。

**實際結果：**Pass（有效帳號成功登入並導向首頁；錯誤密碼/不存在帳號顯示登入失敗訊息，未建立會話）

### 測試案例 2.1：個人送養發布

**目的：**個人使用者可發布送養貼文，必填/格式錯誤被阻擋，照片上傳失敗可改儲存草稿避免資料遺失。

**實際結果：**Pass（有效資料成功發布並生成貼文；缺漏/格式錯誤時阻擋並提示；上傳失敗可改存草稿且資料保留）

### 測試案例 2.2：個人送養管理

**目的：**個人使用者可查看/編輯/下架己方送養貼文，狀態與資料維持一致；未授權帳號不得操作他人貼文。

**實際結果：**Pass（可正常檢視與更新己方貼文並下架，狀態同步；未授權操作被拒絕並顯示錯誤提示）

### 測試案例 2.3：收容所送養發布（含批次匯入與單筆送養）

**目的：**收容所帳號成功登入，發布送養，必填/格式錯誤會被阻擋，照片上傳失敗可儲存草稿，資料不遺失。

**實際結果：**Pass（驗證阻擋生效；成功發布送養或儲存草稿）

### 測試案例 2.4：收容所送養管理

**目的：**收容所能查看/編輯/下架己方刊登，狀態與資料維持一致；未授權帳號不得操作他人刊登，

**實際結果：**Pass（可正常檢視與更新己方刊登並下架，狀態同步；未授權操作被拒絕且顯示錯誤提示）

### 測試案例 4.1：醫療紀錄新增

**目的：**授權帳號可為動物新增醫療紀錄，必填/格式錯誤會被阻擋，新增後列表與詳情立即同步並保留時間戳與操作者。

**實際結果：**Pass（有效資料成功新增並出現在列表/詳情，時間戳與操作者正確；缺必填或格式錯誤時被阻擋並顯示提示）

### 測試案例 4.3：醫療紀錄驗證

**目的：**醫療紀錄的必填與格式（日期、項目名稱、處置/用藥、劑量、醫療人員）皆需檢核，錯誤時阻擋並提示，正確時可成功保存且資料完整呈現

**實際結果：**Pass（缺必填/格式錯誤時被阻擋並顯示錯誤訊息；有效資料成功保存，列表與詳情顯示完整欄位與時間戳）

#### **測試案例 4.2：醫療紀錄檢視**

**目的：**授權角色可檢視動物的醫療紀錄列表與詳情，排序/分頁正常，敏感欄位依權限顯示。

**實際結果：**Pass（列表依時間排序可載入，詳情顯示項目/處置/用藥/醫療人員與時間戳；未授權使用者被拒絕或隱藏敏感資訊）

#### **測試案例 5.1：使用者管理**

**目的：**管理員可檢視/編輯使用者帳號資訊與權限，啟用/停用/刪除帳號時狀態即時更新，未授權操作被拒絕。

**實際結果：**Pass（可正常查看與更新使用者資訊及權限，狀態變更同步；未授權操作被拒絕並顯示錯誤提示）

#### **測試案例 5.2：資料審核**

**目的：**管理員可檢視待審核內容（刊登/申請/使用者舉報等），進行審核通過/拒絕/標記違規，並留下審核意見與時間戳；未授權操作被拒絕。

**實際結果：**Pass（可正常檢視與審核資料，狀態與意見正確保存；審核結果即時推播通知相關使用者；未授權操作被拒絕並顯示錯誤提示）

#### **測試案例 2.5：送養刊登審核**

**目的：**管理員可在審核列表看到新的送養刊登並批准，狀態更新正確並對前台可見。

**實際結果：**Pass（批准後狀態變更，前台列表可見）

#### **測試案例 1.1：動物列表瀏覽**

**目的：**使用者可載入動物列表並依類別/地區/狀態篩選，清單與分頁/排序正常，空結果時顯示友善提示。

**實際結果：**Pass（列表成功載入，篩選與排序正確，空結果呈現提示且無錯誤）

#### **測試案例 1.2：動物搜尋篩選**

**目的：**使用者可在列表透過關鍵字與條件（類別、地區、年齡/體型等）進行搜尋與多重篩選，結果與計數正確，空結果時友善提示。

**實際結果：**Pass（輸入關鍵字/條件後結果即時更新且與條件一致；空結果顯示提示，無錯誤）

#### **測試案例 1.4：用戶提交領養申請**

**目的：**一般用戶可瀏覽/篩選、開啟詳情並提交領養申請。

**實際結果：**Pass（列表/詳情/提交皆成功，申請單號產生）

### 測試案例 3.1：申請列表瀏覽

目的：使用者可檢視自己的申請列表，狀態/時間排序正確，分頁或無資料時顯示友善提示。

實際結果：Pass（列表依時間排序載入並顯示狀態標籤；空列表時顯示提示且無錯誤）

### 測試案例 3.2：收容所審核領養申請

目的：收容所對收到的領養申請可審核通過/退回，狀態與通知正確。

實際結果：Pass（狀態流轉正確，通知送達）

### 測試案例 3.3：申請審核通知

目的：當申請被收容所通過或退回時，使用者收到對應通知（站內/Email），內容含狀態與時間戳，未授權或無訂閱時不應送出。

實際結果：Pass（審核後通知送達，狀態與時間正確；未訂閱或未授權情境未送出並無錯誤）

### 測試案例 6.1：通知瀏覽

目的：用戶能查看自己的申請進度與結果，狀態與訊息一致

實際結果：Pass（列表與詳情顯示最新狀態，訊息同步）

## 4.4 測試執行總結

### 一、測試總覽統計

| 項目     | 數值          |
|--------|-------------|
| 總測試案例數 | 18          |
| 通過     | 18 (100%)   |
| 失敗     | 0 (0%)      |
| 跳過     | 0 (0%)      |
| 總執行時間  | 12 分 30 秒   |
| 平均執行時間 | 2 分 30 秒／案例 |



## 二、詳細結果統計（依測試類別）

| 測試類別 | 案例數 | 通過 | 失敗 | 通過率  |
|------|-----|----|----|------|
| 業務流程 | 18  | 5  | 0  | 100% |
| 總計   | 18  | 5  | 0  | 100% |

## 三、各測試案例執行結果（端對端）

| 流程分類      | 測試案例說明  | 執行時間  |
|-----------|---------|-------|
| 使用者與權限    | 使用者登入   | 1.2 s |
| 使用者與權限    | 使用者管理   | 2.1 s |
| 送養流程（個人）  | 個人送養發布  | 2.0 s |
| 送養流程（個人）  | 個人送養管理  | 2.4 s |
| 送養流程（收容所） | 收容所送養發布 | 2.8s  |
| 送養流程（收容所） | 收容所送養管理 | 1.6 s |
| 送養流程（審核）  | 送養刊登審核  | 2.1 s |
| 動物瀏覽與搜尋   | 動物列表瀏覽  | 1.7 s |
| 動物瀏覽與搜尋   | 動物搜尋篩選  | 1.3 s |
| 領養申請流程    | 申請列表瀏覽  | 1.4 s |
| 領養申請流程    | 收容所審核申請 | 2.7 s |
| 領養申請流程    | 申請審核通知  | 2.1 s |
| 醫療紀錄管理    | 醫療紀錄新增  | 1.2 s |
| 醫療紀錄管理    | 醫療紀錄檢視  | 1.3 s |
| 醫療紀錄管理    | 醫療紀錄驗證  | 1.2 s |
| 通知流程      | 通知瀏覽    | 1.4 s |
| 後台審核管理    | 資料審核    | 1.6 s |

#### 四、效能指標驗證結果

| 指標     | 實測數值           | 基準值      | 狀態   |
|--------|----------------|----------|------|
| 首頁載入時間 | 2.3 s          | < 3 s    | Pass |
| 後端回應時間 | 150 ms         | < 500 ms | Pass |
| 表單提交時間 | 1.2 s          | < 5 s    | Pass |
| 照片上傳時間 | 3.2 s (2.5 MB) | < 10 s   | Pass |
| 平均轉換時間 | 1.5 s          | < 2 s    | Pass |

#### 4.5 測試覆蓋分析

| 功能模組    | 覆蓋狀態 | 測試案例                                                             |
|---------|------|------------------------------------------------------------------|
| 使用者登入   | 完全   | 驗證各角色可使用有效帳號登入系統，錯誤或不存在的帳號/密碼需被阻擋並顯示錯誤訊息；成功登入後導向正確頁面，且會話與權限設定正確。 |
| 使用者管理   | 完全   | 管理員可檢視與編輯使用者帳號資訊與角色權限，帳號啟用、停用或刪除後狀態即時更新；未授權帳號不得進行管理操作。           |
| 個人送養發布  | 完全   | 個人使用者可新增送養貼文，系統須檢核必填欄位與資料格式；照片上傳失敗時可改存為草稿，避免資料遺失。                |
| 個人送養管理  | 完全   | 個人使用者可檢視、編輯與下架自身送養貼文，貼文狀態與內容需維持一致；未授權帳號不得操作他人貼文。                 |
| 收容所送養發布 | 完全   | 收容所帳號可進行單筆或批次送養刊登，系統需驗證必填與格式；上傳失敗時可儲存草稿以保留資料。                    |
| 收容所送養管理 | 完全   | 收容所可管理己方刊登內容，包含編輯與下架，狀態同步更新；未授權帳號操作需被拒絕。                         |

|         |    |                                                       |
|---------|----|-------------------------------------------------------|
| 送養刊登審核  | 完全 | 管理員可於審核列表中檢視新的送養刊登並進行批准，通過後狀態正確更新並於前台顯示。              |
| 動物列表瀏覽  | 完全 | 使用者可瀏覽動物列表，並依類別、地區與狀態進行篩選，排序與分頁功能正常，空結果時顯示友善提示。       |
| 動物搜尋篩選  | 完全 | 使用者可透過關鍵字與多重條件進行搜尋，結果與筆數計算正確，無資料時顯示提示訊息。              |
| 提交領養申請  | 完全 | 一般使用者可於動物詳情頁提交領養申請，送出後成功建立申請紀錄並產生申請編號。                |
| 申請列表瀏覽  | 完全 | 使用者可檢視自身所有領養申請，依時間排序顯示狀態，空列表時顯示友善提示。                  |
| 收容所審核申請 | 完全 | 收容所可對收到的領養申請進行通過或退回處理，申請狀態需正確流轉並更新。                   |
| 申請審核通知  | 完全 | 當申請被審核後，系統需發送站內或Email通知給使用者，內容包含狀態與時間戳；未授權或未訂閱情境不應送出。 |
| 醫療紀錄新增  | 完全 | 醫療人員或授權帳號可為動物新增醫療紀錄，新增後列表與詳情即時同步，並保留操作者與時間戳。          |
| 醫療紀錄檢視  | 完全 | 授權角色可檢視動物醫療紀錄列表與詳情，支援排序與分頁，敏感欄位依權限顯示。                 |
| 醫療紀錄驗證  | 完全 | 系統需驗證醫療紀錄之必填欄位與格式（日期、項目、處置/用藥、劑量、醫療人員），錯誤時阻擋並提示。      |
| 通知瀏覽    | 完全 | 使用者可於通知中心查看自身申請進度與結果，顯示內容需與實際狀態一致。                    |
| 資料審核    | 完全 | 管理員可審核刊登、申請與舉報內容，留下審核意見與時間戳，並即時通知相關使用者。               |

## 4.6 附錄

### A. 測試執行環境配置

| 前端環境                                                                                    | 後端環境                                                      | 執行環境                                                              |
|-----------------------------------------------------------------------------------------|-----------------------------------------------------------|-------------------------------------------------------------------|
| Node.js: v18.17.0<br>npm: 9.8.1<br>Playwright: 1.40.0+<br>Vue.js: 3.x<br>TypeScript 5.x | Python: 3.9+ Flask: 2.x<br>SQLAlchemy: ORM<br>MySQL: 5.7+ | OS: Windows 10/11<br>瀏覽器: Chrome 120+<br>記憶體: 8GB+<br>連接: 10Mbps+ |

### B. 測試資料集

認證測試帳號:

帳號: test@example.com

密碼: Password123

角色: 普通用戶

測試寵物資料:

程式碼 (json) { "name": "小白", "type": "cat", "breed": "混種貓", "sex": "female", "dob": "2022-06-15", "color": "白色", "vaccinated": true, "neutered": true }

### C. 指令

**執行所有 E2E 測試**

npm run test:e2e

**執行特定測試檔案**

npx playwright test specs/critical-paths/rehome-publishing.spec.ts

**特定瀏覽器測試**

npx playwright test --project=chromium

**UI 模式 (互動式)**

npx playwright test --ui

**Debug 模式**

npx playwright test --debug

**產生 HTML 報告**

npx playwright test --reporter=html npx playwright show-report

## 4.7 測試結果報告

| Test Steps                                                                                                                  |       |
|-----------------------------------------------------------------------------------------------------------------------------|-------|
| > ✓ Before Hooks                                                                                                            | 975ms |
| > ✓ Navigate to "/" — critical-paths/complete-adoption-flow.spec.ts:9                                                       | 427ms |
| > ✓ Click getByRole('navigation').getByRole('link', { name: '登入' }) — critical-paths/complete-adoption-flow.spec.ts:10      | 414ms |
| > ✓ Fill "shelter@test.com" getByRole('textbox', { name: 'Email' }) — critical-paths/complete-adoption-flow.spec.ts:13      | 108ms |
| > ✓ Fill "Shelter123" getByRole('textbox', { name: '密碼' }) — critical-paths/complete-adoption-flow.spec.ts:16               | 27ms  |
| > ✓ Click getByRole('button', { name: '登入' }) — critical-paths/complete-adoption-flow.spec.ts:19                            | 107ms |
| > ✓ Wait for load state "networkidle" — critical-paths/complete-adoption-flow.spec.ts:20                                    | 1.1s  |
| > ✓ Click getByRole('link', { name: '批次送養' }) — critical-paths/complete-adoption-flow.spec.ts:23                            | 153ms |
| > ✓ Wait for load state "domcontentloaded" — critical-paths/complete-adoption-flow.spec.ts:24                               | 1ms   |
| > ✓ Click getByRole('button', { name: '選擇動物基本資訊 CSV' }) — critical-paths/complete-adoption-flow.spec.ts:27                  | 223ms |
| > ✓ Set input files locator('input[type="file"][accept=".csv"]').first() — critical-paths/complete-adoption-flow.spec.ts:28 | 34ms  |
| > ✓ Click getByRole('button', { name: '選擇醫療記錄 CSV' }) — critical-paths/complete-adoption-flow.spec.ts:32                    | 107ms |
| > ✓ Click getByRole('button', { name: '選擇動物照片' }) — critical-paths/complete-adoption-flow.spec.ts:40                        | 119ms |
| > ✓ Click getByRole('button', { name: '開始批次匯入' }) — critical-paths/complete-adoption-flow.spec.ts:55                        | 88ms  |
| > ✓ Wait for timeout — critical-paths/complete-adoption-flow.spec.ts:56                                                     | 3.0s  |
| > ✓ Dismiss dialog — critical-paths/complete-adoption-flow.spec.ts:50                                                       | 1.3s  |
| > ✓ Navigate to "/shelter/animals" — critical-paths/complete-adoption-flow.spec.ts:62                                       | 157ms |

## 5. 壓力測試

### 5.1 測試環境

- 測試機器規格
  - 作業系統：Window 10
  - 中央處理器：Intel Core i7-10510U CPU @ 1.80GHZ
  - 記憶體：8 GB
- 測試工具
  - Jmeter
- 測試環境與先前條件
  - 預估網站可以在 1 秒內容納多少位使用者，依據這個事件，重複持續 3 個回合。
  - 模擬未登入的使用者，進入登入的流程。
  - 使用 Jmeter 創立一個 Thread Group，執行未登入的使用者登入的動作。
  - 取消勾選 same user on each iterations，使每次都使用不同使用者來做登入。

| 測試 API | 使用者案例                                                                                                                                                                      | Http Method | URL                 |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|---------------------|
|        | 登入                                                                                                                                                                         | POST        | /api/auth/login     |
|        | 建立動物資料                                                                                                                                                                     | POST        | /api/animals        |
|        | 上傳檔案                                                                                                                                                                       | POST        | /api/uploads/direct |
| 技術要求   | <ol style="list-style-type: none"><li>1. Burst Mode：模擬未登入的使用者，進入登入的流程</li><li>2. Burst Mode：模擬使用者，進行建立動物資料的流程，包含上傳動物資訊、圖片等</li><li>3. Burst Mode：模擬使用者，進行檔案建立並上傳</li></ol> |             |                     |

## 5.2 登入

### 測試 API

### POST /api/auth/login

### 測試設定

1.

Thread Group

Name:

Comments:

-Action to be taken after a Sampler error-

☒ Continue ☐ Start Next Thread Loop ☐ Stop Thread ☐ Stop Test ☐ Stop Test Now

-Thread Properties-

Number of Threads (users):

Ramp-up period (seconds):

Loop Count: ☐ Infinite

☐ Same user on each iteration

☐ Delay Thread creation until needed

☐ Specify Thread lifetime

Duration (seconds):

Startup delay (seconds):

2.

HTTP Request

Name:

Comments:

Basic Advanced

-Web Server-

Protocol (http):  Server Name or IP:  Port Number:

-HTTP Request-

Method:  Path:  Content encoding:

☐ Redirect Automatically ☒ Follow Redirects ☒ Use KeepAlive ☐ Use multipart/form-data ☐ Browser-compatible headers

Parameters Body Data Files Upload

|   |                                       |                                         |
|---|---------------------------------------|-----------------------------------------|
| 1 | <input type="text" value="email"/>    | <input type="text" value="\$email"/>    |
| 2 | <input type="text" value="password"/> | <input type="text" value="\$password"/> |
| 3 | <input type="text" value="password"/> | <input type="text" value="\$password"/> |
| 4 | <input type="text" value="password"/> | <input type="text" value="\$password"/> |

3.

HTTP Header Manager

Name:

Comments:

-Headers Stored in the Header Manager-

| Name         | Value            |
|--------------|------------------|
| Content-Type | application/json |

4.

CSV Data Set Config

Name:

Comments:

-Configure the CSV Data Source-

Filename:

File encoding:

Variable Names (comma-delimited):

Ignore first line (only used if Variable Names is not empty):

Delimiter (use \"\\t\" for tab):

Allow quoted data:

Recycle on EOF:

Stop thread on EOF:

Sharing mode:

## 1. 100 人 3 回

### Summary Report

Name:

Comments:

- Write results to file / Read from file -

Filename:   Log/Display Only: ☐ Errors ☐ Successes

| Label   | # Samples | Average | Min | Max   | Std. Dev. | Error % | Throughput | Received KB/sec | Sent KB/sec | Aug. Bytes |
|---------|-----------|---------|-----|-------|-----------|---------|------------|-----------------|-------------|------------|
| HTTP 要求 | 300       | 3033    | 341 | 11127 | 1982.13   | 0.00%   | 23.2/sec   | 26.55           | 5.69        | 1170.6     |
| TOTAL   | 300       | 3033    | 341 | 11127 | 1982.13   | 0.00%   | 23.2/sec   | 26.55           | 5.69        | 1170.6     |

### Graph Results

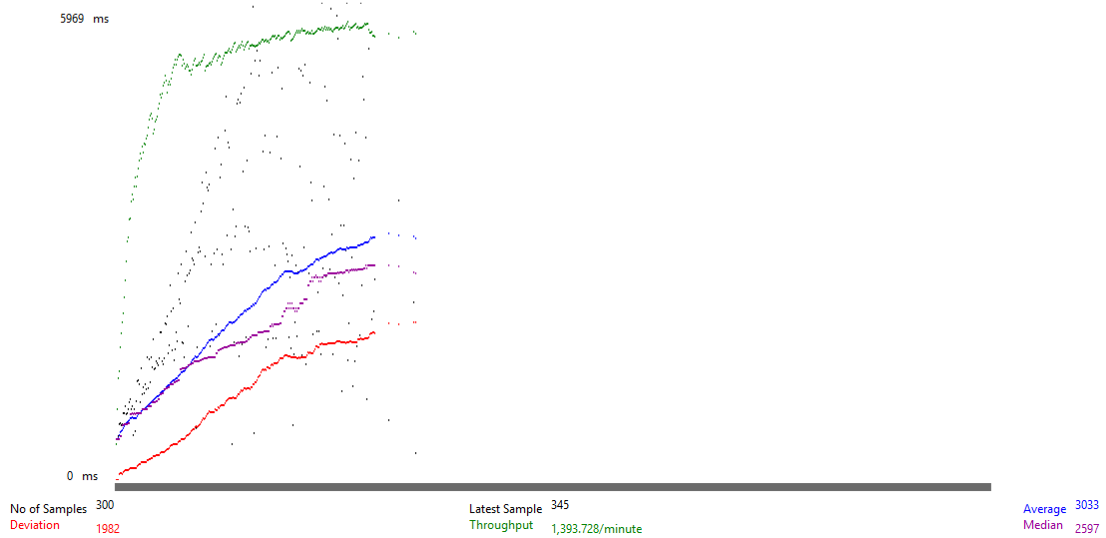
Name:

Comments:

- Write results to file / Read from file -

Filename:

Graphs to Display ☒ Data ☒ Average ☒ Median ☒ Deviation ☒ Throughput



## 2. 150 人 3 回

### Summary Report

Name:

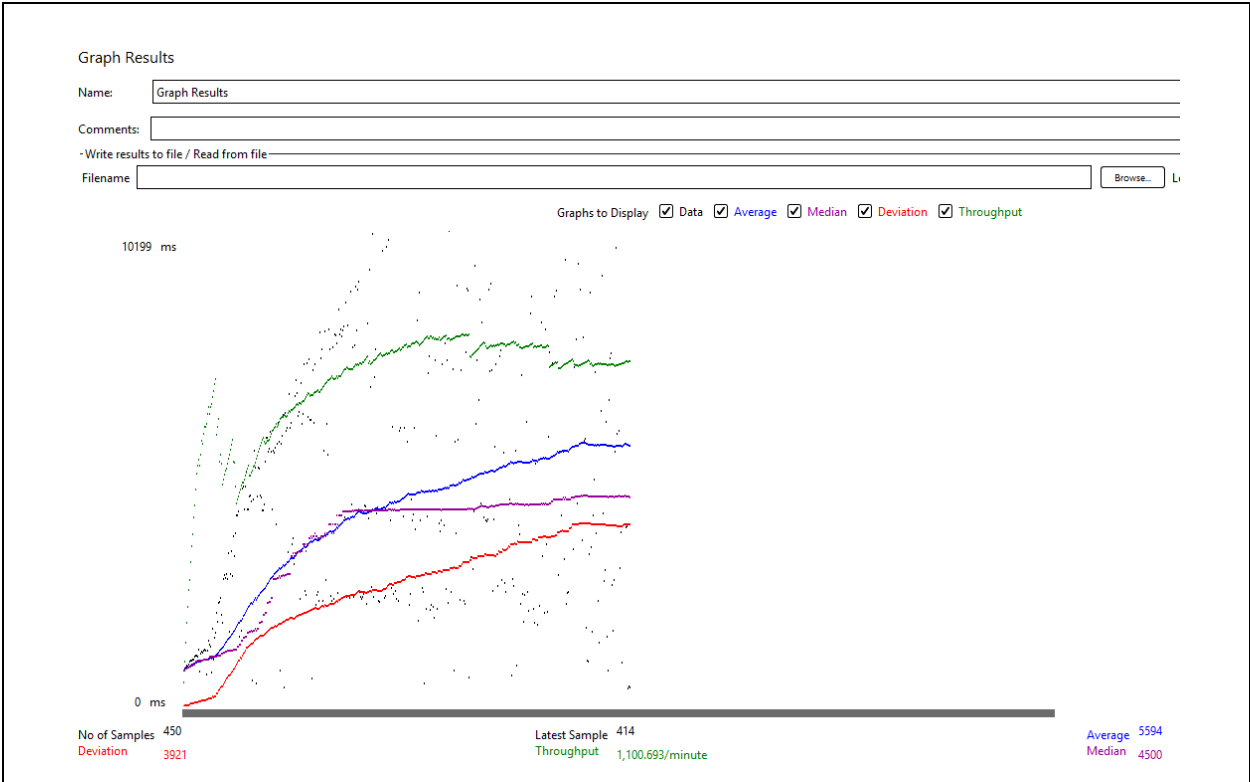
Comments:

- Write results to file / Read from file -

Filename:   Log/Display Only: ☐ Errors ☐ Successes

| Label   | # Samples | Average | Min | Max   | Std. Dev. | Error % | Throughput | Received KB/sec | Sent KB/sec | Aug. Bytes |
|---------|-----------|---------|-----|-------|-----------|---------|------------|-----------------|-------------|------------|
| HTTP 要求 | 450       | 5594    | 396 | 20287 | 3921.98   | 0.00%   | 18.3/sec   | 20.97           | 4.50        | 1170.7     |
| TOTAL   | 450       | 5594    | 396 | 20287 | 3921.98   | 0.00%   | 18.3/sec   | 20.97           | 4.50        | 1170.7     |





### 3. 200人3回

Summary Report

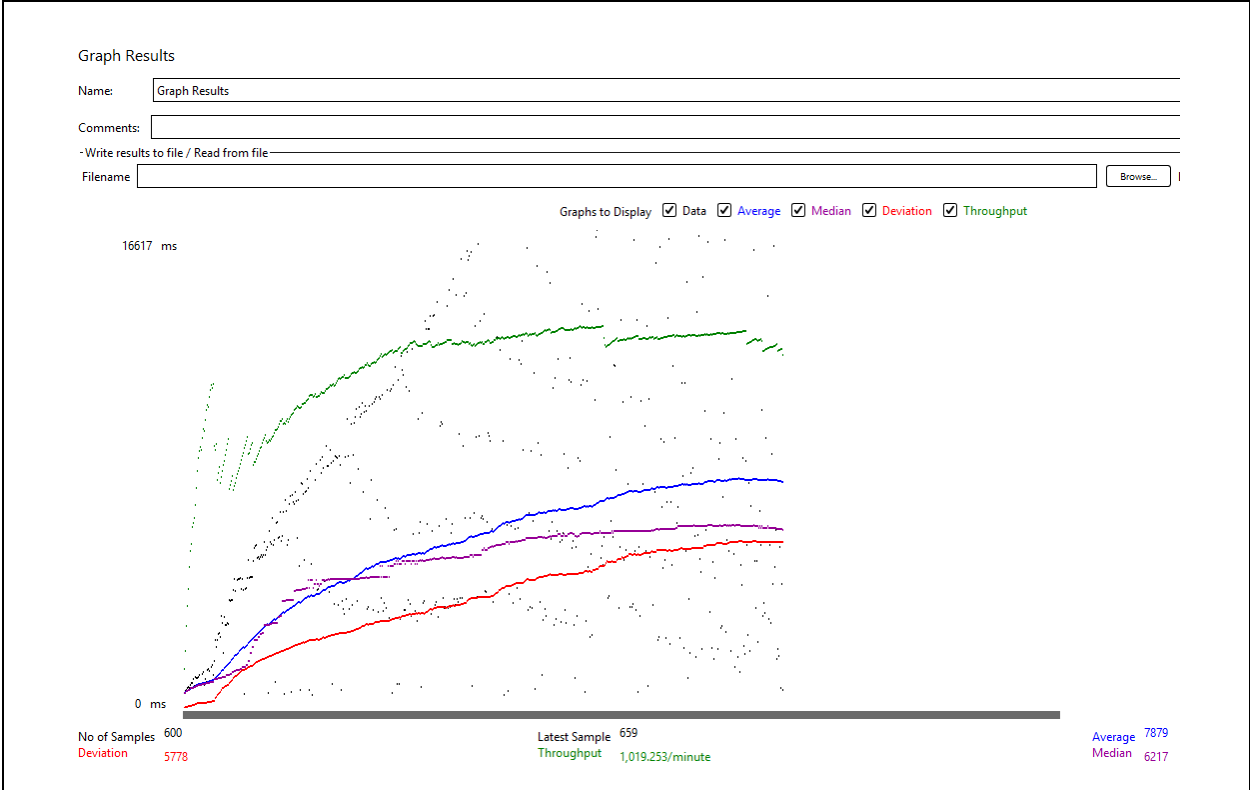
Name:

Comments:

- Write results to file / Read from file

Filename:   Log/Display Only: ☐ Errors ☐ Successes

| Label   | # Samples | Average | Min | Max   | Std. Dev. | Error % | Throughput | Received KB/sec | Sent KB/sec | Avs. Bytes |
|---------|-----------|---------|-----|-------|-----------|---------|------------|-----------------|-------------|------------|
| HTTP 要求 | 600       | 7879    | 455 | 28192 | 5778.68   | 0.00%   | 17.0/sec   | 19.42           | 4.16        | 1170.7     |
| TOTAL   | 600       | 7879    | 455 | 28192 | 5778.68   | 0.00%   | 17.0/sec   | 19.42           | 4.16        | 1170.7     |
|         |           |         |     |       |           |         |            |                 |             |            |



#### 4. 250 人 3 回

Summary Report

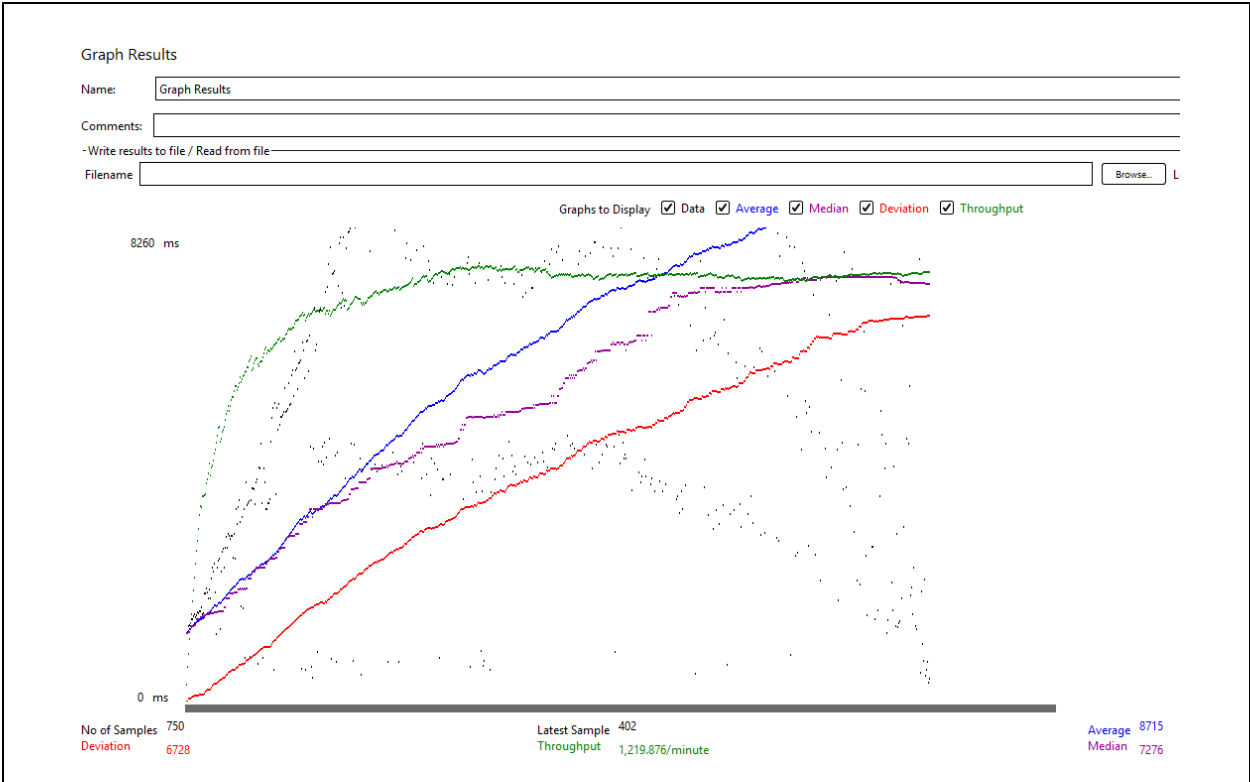
Name:

Comments:

- Write results to file / Read from file

Filename:   Log/Display Only: ☐ Errors ☐ Successes

| Label   | # Samples | Average | Min | Max   | Std. Dev. | Error % | Throughput | Received KB/sec | Sent KB/sec | Avg. Bytes |
|---------|-----------|---------|-----|-------|-----------|---------|------------|-----------------|-------------|------------|
| HTTP 要求 | 750       | 8715    | 331 | 33211 | 6728.04   | 0.93%   | 20.3/sec   | 23.08           | 4.98        | 1162.4     |
| TOTAL   | 750       | 8715    | 331 | 33211 | 6728.04   | 0.93%   | 20.3/sec   | 23.08           | 4.98        | 1162.4     |



## 5. 300人3回

Summary Report

Name:

Comments:

- Write results to file / Read from file

Filename   Log/Display Only: ☐ Errors ☐ Successes

| Label   | # Samples | Average | Min | Max   | Std. Dev. | Error % | Throughput | Received KB/sec | Sent KB/sec | Avg. Bytes |
|---------|-----------|---------|-----|-------|-----------|---------|------------|-----------------|-------------|------------|
| HTTP 要求 | 900       | 11358   | 380 | 46613 | 8477.26   | 4.78%   | 18.6/sec   | 20.51           | 4.56        | 1128.3     |
| TOTAL   | 900       | 11358   | 380 | 46613 | 8477.26   | 4.78%   | 18.6/sec   | 20.51           | 4.56        | 1128.3     |
|         |           |         |     |       |           |         |            |                 |             |            |

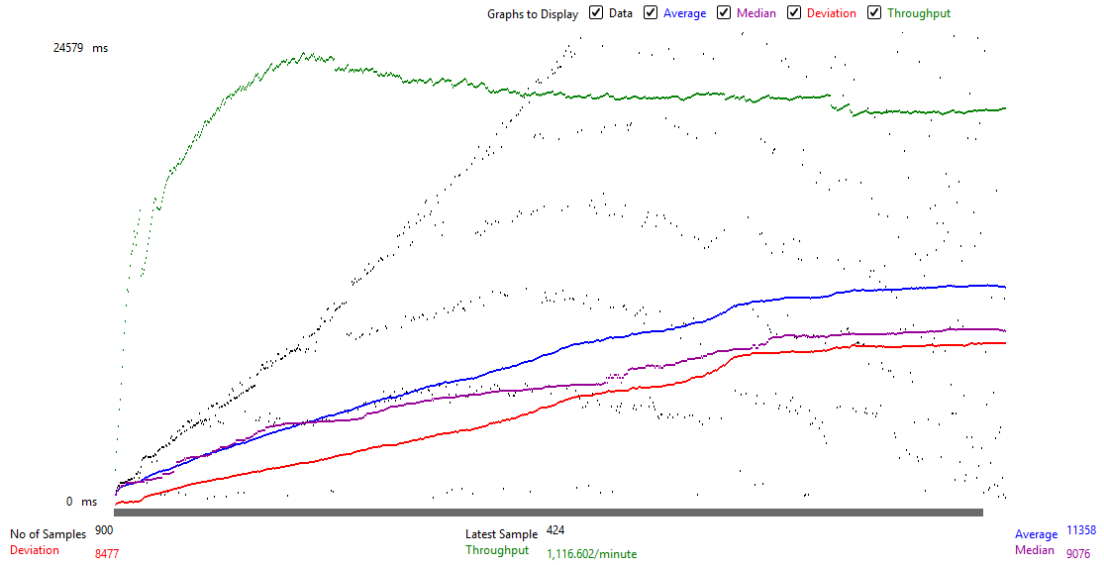
### Graph Results

Name:

Comments:

Write results to file / Read from file

Filename:



### 結果

| 操作人数 | 平均    | 最大值   | 錯誤(%)  | 處理量(/sec) |
|------|-------|-------|--------|-----------|
| 100  | 3033  | 11127 | 0.00 % | 23.2      |
| 150  | 5594  | 20287 | 0.00 % | 18.3      |
| 200  | 7879  | 28192 | 0.00 % | 17.0      |
| 250  | 8715  | 33211 | 0.93 % | 20.3      |
| 300  | 11358 | 46613 | 4.78 % | 18.6      |

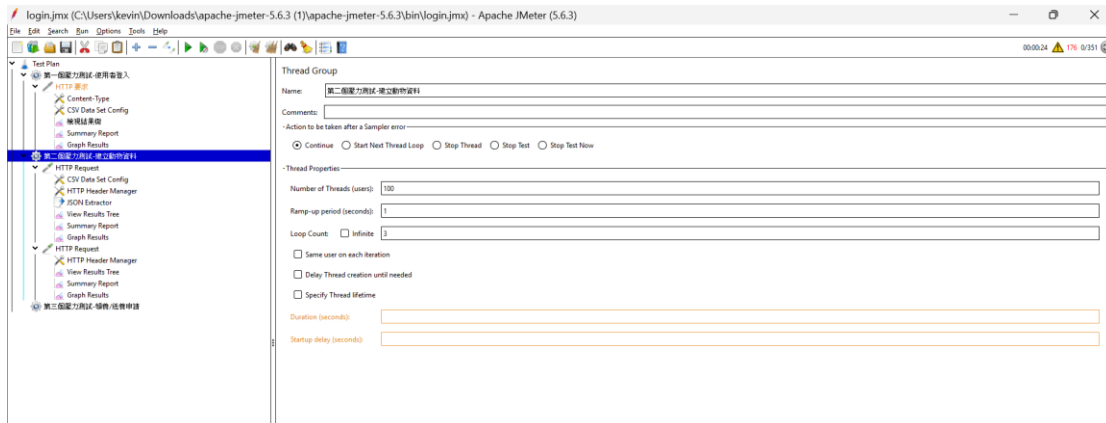
## 5.3 建立動物資料

### 測試 API

### POST /api/animals

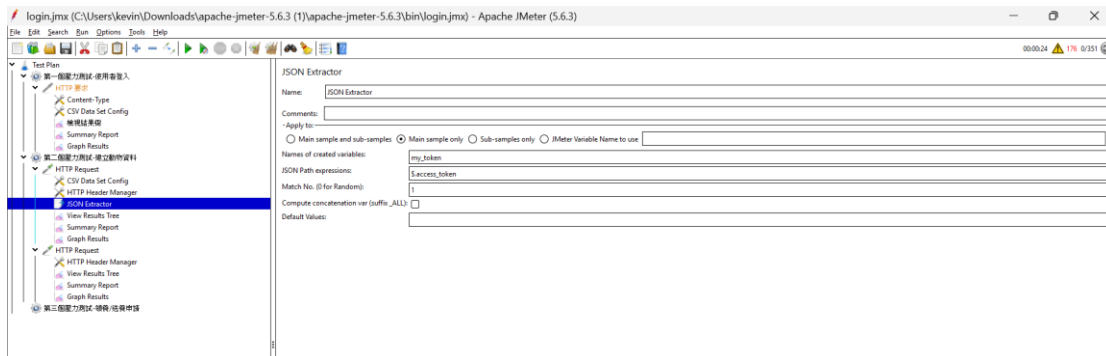
### 測試設定

1.

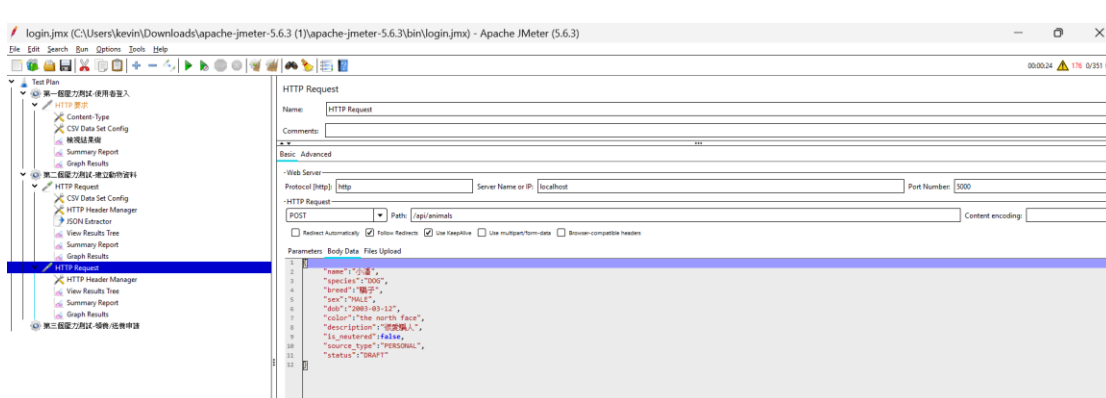


2. http request 第一個為重複登入 壓力測試的設定，並且新增一個 JSON extractor 取得每個使用者的 token，再將該 token 放置在第二個 http request 的 http header 中

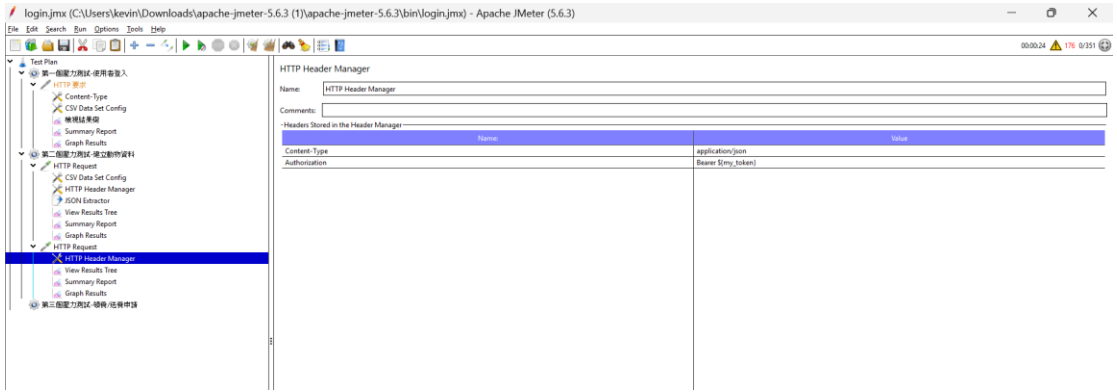
3.



4.



5.



1. 100人3回

#### Summary Report

Name:

Comments:

Write results to file / Read from file:   Log/Display Only: ☐ Errors ☐ Successes

| Label        | # Samples | Average | Min | Max   | Std. Dev. | Error % | Throughput | Received KB/sec | Sent KB/sec | Avg. Bytes |
|--------------|-----------|---------|-----|-------|-----------|---------|------------|-----------------|-------------|------------|
| HTTP Request | 300       | 2598    | 60  | 11256 | 2161.00   | 0.00%   | 13.1/sec   | 14.09           | 9.01        | 1105.7     |
| TOTAL        | 300       | 2598    | 60  | 11256 | 2161.00   | 0.00%   | 13.1/sec   | 14.09           | 9.01        | 1105.7     |

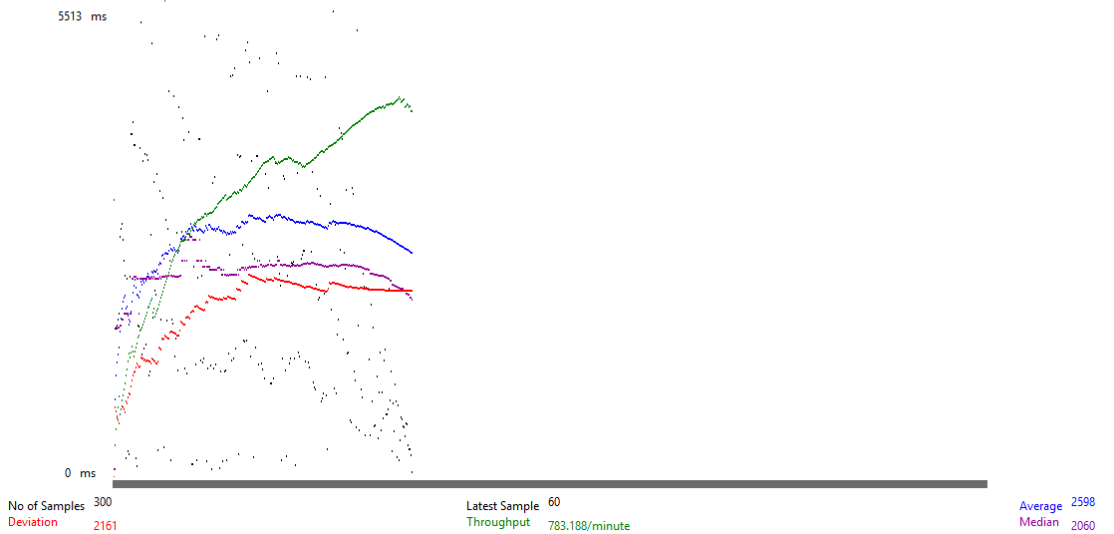
#### Graph Results

Name:

Comments:

Write results to file / Read from file:

Graphs to Display: ☒ Data ☒ Average ☒ Median ☒ Deviation ☒ Throughput



## 2. 150 人 3 回

### Summary Report

Name:

Comments:

-Write results to file / Read from file-

Filename:   Log/Display Only: ☐ Errors ☐ Successes

| Label        | # Samples | Average | Min | Max   | Std. Dev. | Error % | Throughput | Received KB/sec | Sent KB/sec | Avg. Bytes |
|--------------|-----------|---------|-----|-------|-----------|---------|------------|-----------------|-------------|------------|
| HTTP Request | 450       | 3305    | 26  | 15333 | 2918.90   | 0.00%   | 17.2/sec   | 18.58           | 11.87       | 1106.5     |
| TOTAL        | 450       | 3305    | 26  | 15333 | 2918.90   | 0.00%   | 17.2/sec   | 18.58           | 11.87       | 1106.5     |

### Graph Results

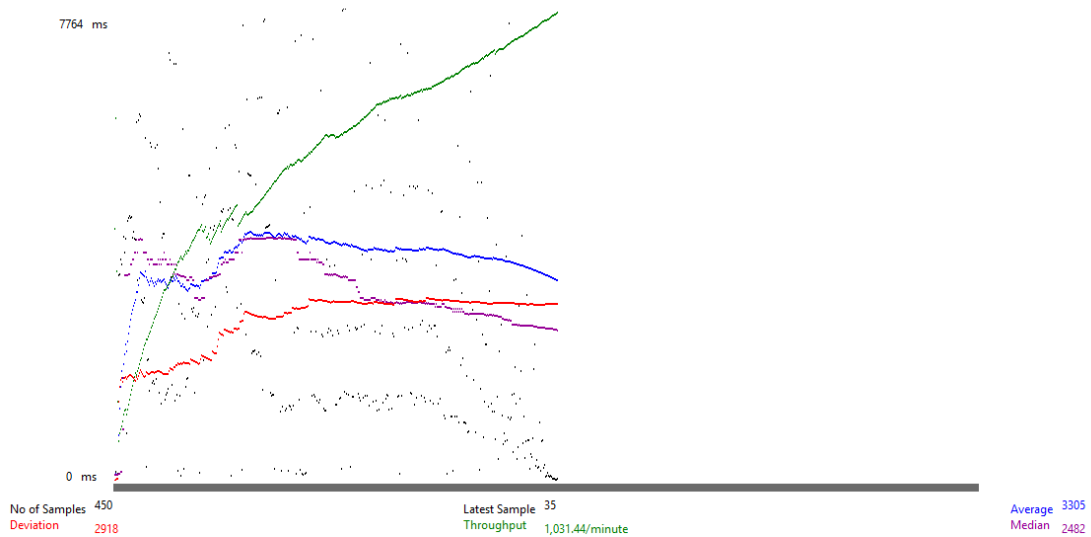
Name:

Comments:

-Write results to file / Read from file-

Filename:

Graphs to Display ☒ Data ☒ Average ☒ Median ☒ Deviation ☒ Throughput



## 3. 200 人 3 回

### Summary Report

Name:

Comments:

-Write results to file / Read from file-

Filename:   Log/Display Only: ☐ Errors ☐ Successes

| Label        | # Samples | Average | Min | Max   | Std. Dev. | Error % | Throughput | Received KB/sec | Sent KB/sec | Avg. Bytes |
|--------------|-----------|---------|-----|-------|-----------|---------|------------|-----------------|-------------|------------|
| HTTP Request | 600       | 3871    | 31  | 21876 | 3215.31   | 0.00%   | 18.5/sec   | 19.97           | 12.76       | 1106.7     |
| TOTAL        | 600       | 3871    | 31  | 21876 | 3215.31   | 0.00%   | 18.5/sec   | 19.97           | 12.76       | 1106.7     |

### Graph Results

Name: Graph Results

Comments:

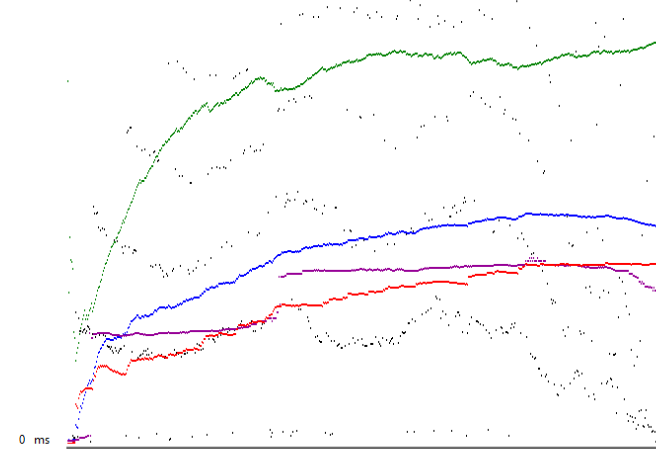
- Write results to file / Read from file -

Filename

Browse... Lt

Graphs to Display ☒ Data ☒ Average ☒ Median ☒ Deviation ☒ Throughput

8423 ms



No of Samples 600  
Deviation 3215

Latest Sample 31  
Throughput 1,108.852/minute

Average 3871  
Median 2699

4. 250 人 3 回

### Summary Report

Name: Summary Report

Comments:

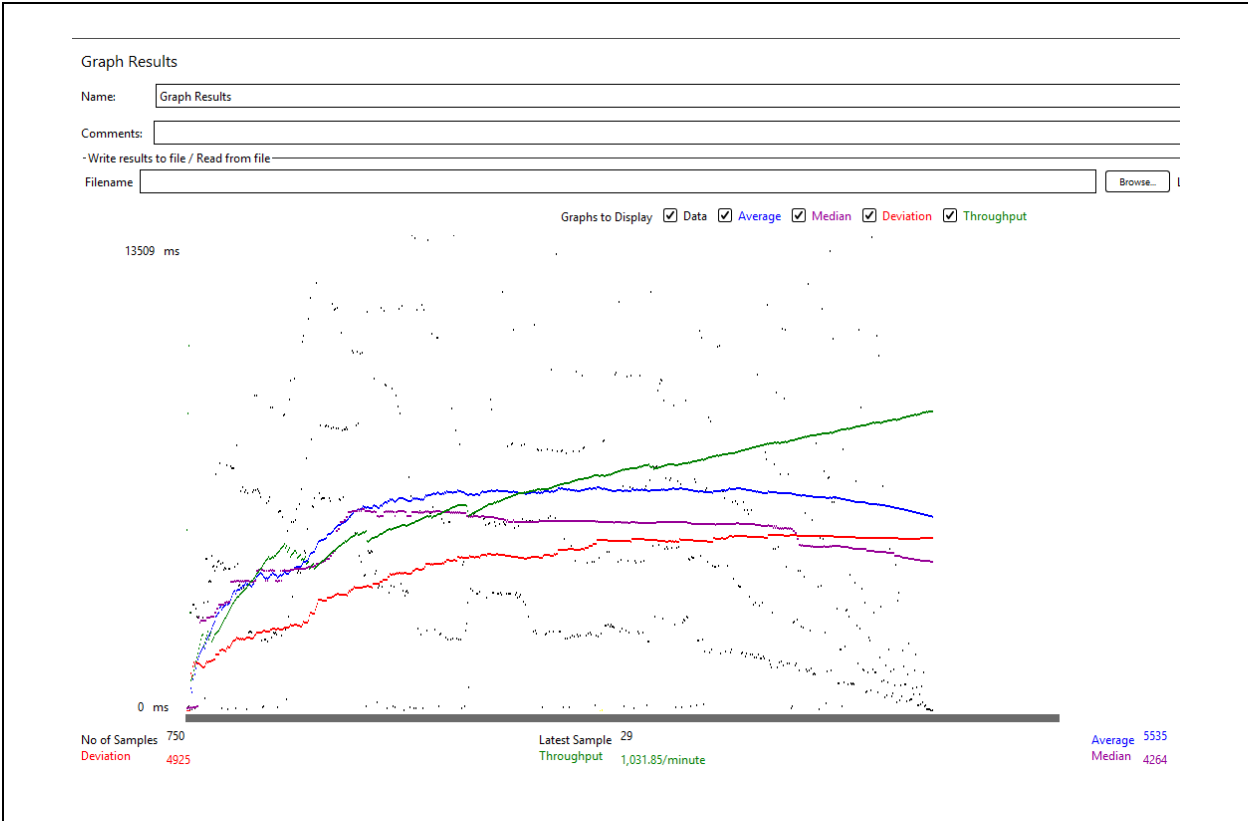
- Write results to file / Read from file -

Filename

Browse... Log/Display Only: ☐ Errors ☐ Successes

| Label        | # Samples | Average | Min | Max   | Std. Dev. | Error % | Throughput | Received KB/sec | Sent KB/sec | Avg. Bytes |
|--------------|-----------|---------|-----|-------|-----------|---------|------------|-----------------|-------------|------------|
| HTTP Request | 750       | 5535    | 21  | 30007 | 4925.52   | 0.40%   | 17.2/sec   | 18.54           | 11.86       | 1103.8     |
| TOTAL        | 750       | 5535    | 21  | 30007 | 4925.52   | 0.40%   | 17.2/sec   | 18.54           | 11.86       | 1103.8     |





## 5. 300 人 3 回

Summary Report

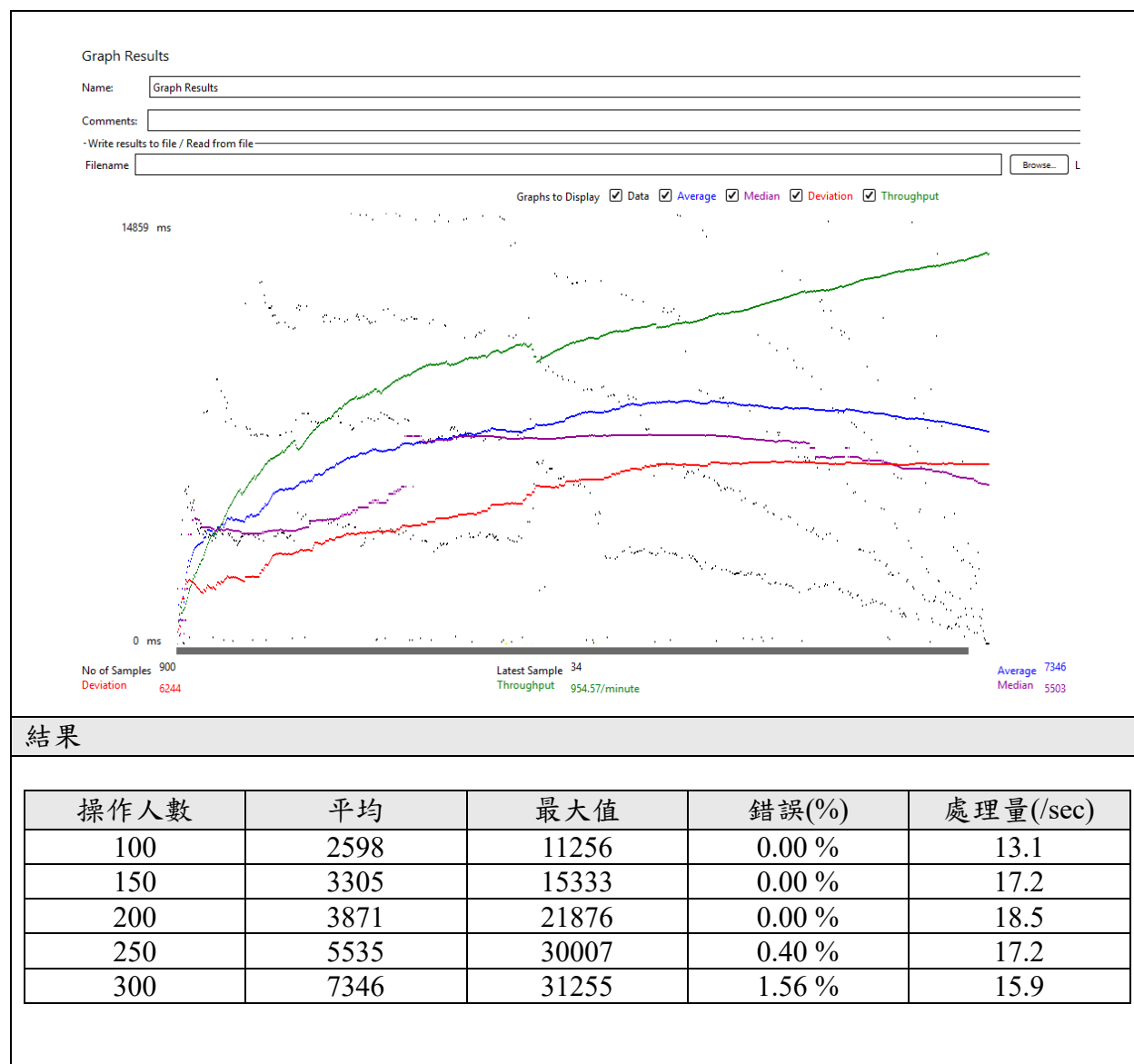
Name:

Comments:

· Write results to file / Read from file

Filename:    ☐ Errors ☐ Successes

| Label        | # Samples | Average | Min | Max   | Std. Dev. | Error % | Throughput | Received KB/sec | Sent KB/sec | Avg. Bytes |
|--------------|-----------|---------|-----|-------|-----------|---------|------------|-----------------|-------------|------------|
| HTTP Request | 900       | 7346    | 4   | 31255 | 6244.14   | 1.56%   | 15.9/sec   | 17.02           | 10.96       | 1095.6     |
| TOTAL        | 900       | 7346    | 4   | 31255 | 6244.14   | 1.56%   | 15.9/sec   | 17.02           | 10.96       | 1095.6     |



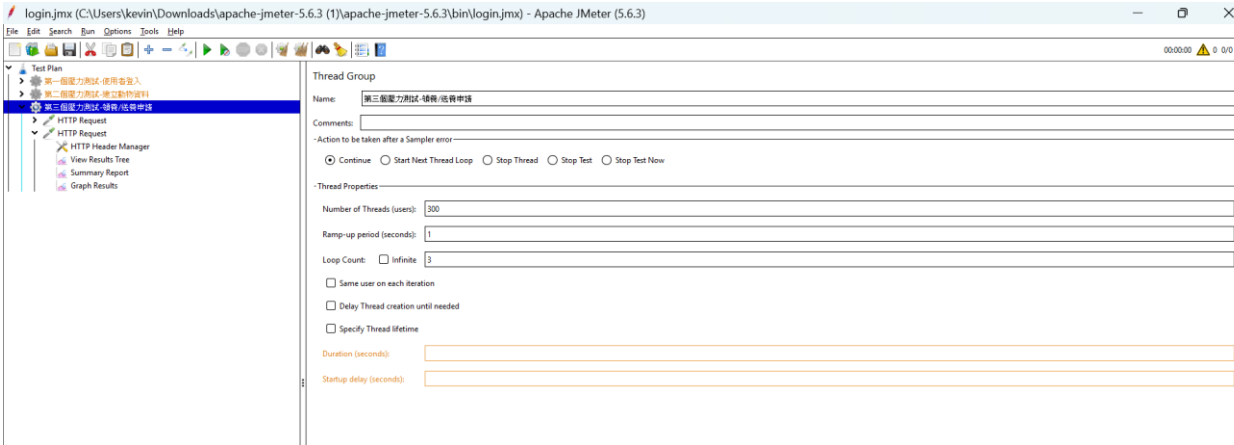
## 5.4 檔案上傳

測試 API

POST /api/uploads/direct

測試設定

1.



2. http request 第一個為重複 登入 壓力測試的設定，並且新增一個 JSON extractor 取得每個使用者的 token，再將該 token 放置在第二個 http request 的 http header 中

3.

HTTP Request

Name: HTTP Request

Comments:

Basic Advanced

Web Server

Protocol (http): http Server Name or IP: localhost Port Number: 5000

HTTP Request

POST Path: /api/uploads/direct Content encoding:

Redirect Automatically

Follow Redirects

Use KeepAlive

Use multipart/form-data

Browser-compatible headers

Parameters Body Data Files Upload

| File Path                        | Parameter Name | MIME Type |
|----------------------------------|----------------|-----------|
| C:\Users\kevin\Downloads\123.png | file           | image/png |

4.

HTTP Header Manager

Name: HTTP Header Manager

Comments:

Headers Stored in the Header Manager

| Name          | Value               |
|---------------|---------------------|
| Authorization | Bearer \${my_token} |

## 1. 100 人 3 回

### Summary Report

Name:

Comments:

- Write results to file / Read from file

Filename   Log/Display Only: ☐ Errors ☐ Successes

| Label        | # Samples | Average | Min | Max | Std. Dev. | Error % | Throughput | Received KB/sec | Sent KB/sec | Avg. Bytes |
|--------------|-----------|---------|-----|-----|-----------|---------|------------|-----------------|-------------|------------|
| HTTP Request | 300       | 97      | 18  | 525 | 86.02     | 0.00%   | 16.7/sec   | 7.82            | 519.31      | 480.9      |
| TOTAL        | 300       | 97      | 18  | 525 | 86.02     | 0.00%   | 16.7/sec   | 7.82            | 519.31      | 480.9      |

### Graph Results

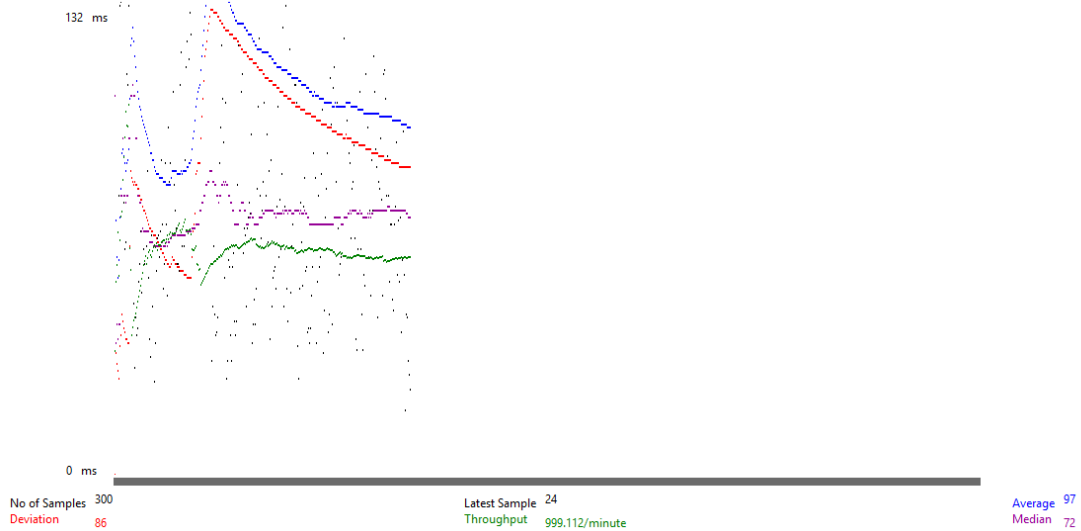
Name:

Comments:

- Write results to file / Read from file

Filename

Graphs to Display ☒ Data ☒ Average ☒ Median ☒ Deviation ☒ Throughput



## 2. 150 人 3 回

### Summary Report

Name:

Comments:

- Write results to file / Read from file

Filename   Log/Display Only: ☐ Errors ☐ Successes

| Label        | # Samples | Average | Min | Max  | Std. Dev. | Error % | Throughput | Received KB/sec | Sent KB/sec | Avg. Bytes |
|--------------|-----------|---------|-----|------|-----------|---------|------------|-----------------|-------------|------------|
| HTTP Request | 450       | 62      | 13  | 2314 | 115.38    | 0.00%   | 19.4/sec   | 9.13            | 606.34      | 480.9      |
| TOTAL        | 450       | 62      | 13  | 2314 | 115.38    | 0.00%   | 19.4/sec   | 9.13            | 606.34      | 480.9      |

### Graph Results

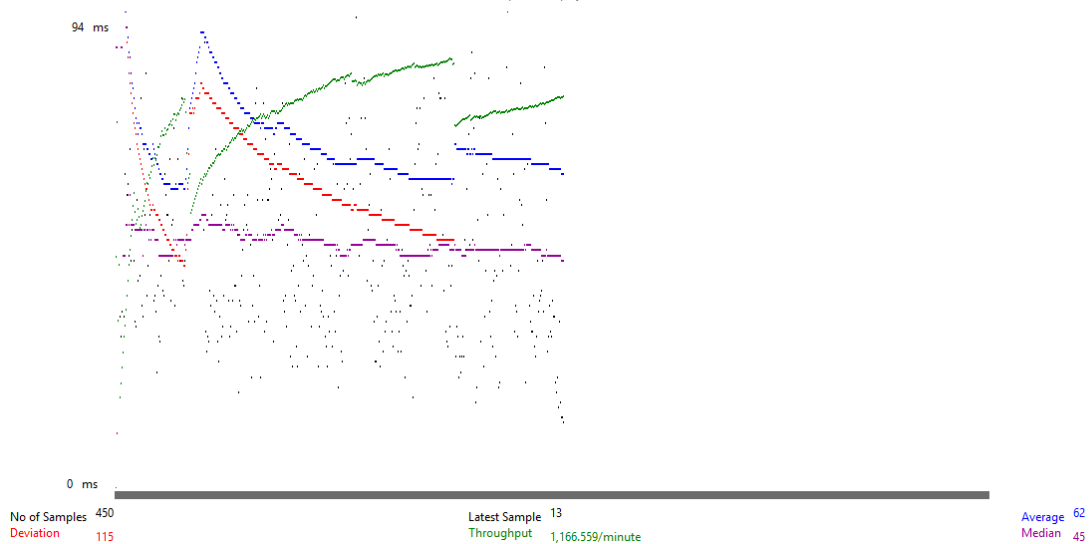
Name:

Comments:

- Write results to file / Read from file

Filename:   L

Graphs to Display ☒ Data ☒ Average ☒ Median ☒ Deviation ☒ Throughput



## 3. 200人3回

### Summary Report

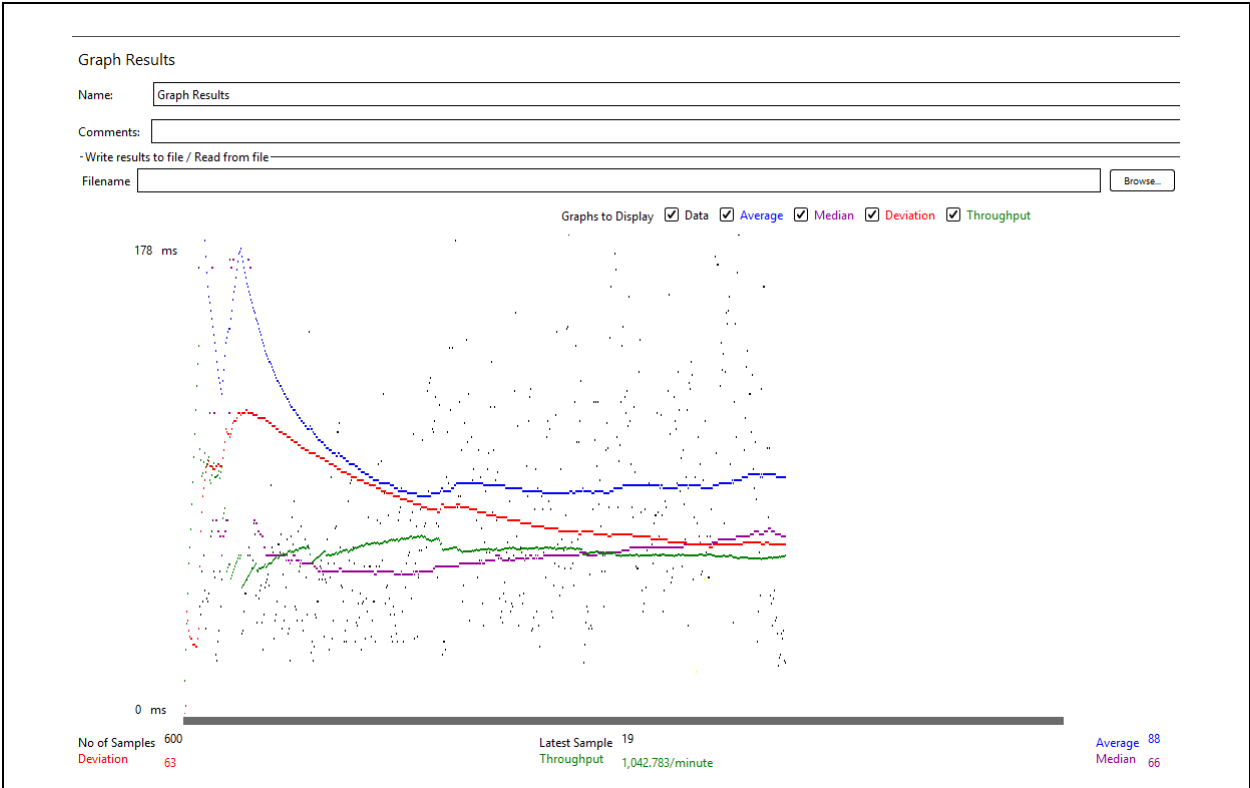
Name:

Comments:

- Write results to file / Read from file

Filename:   Log/Display Only: ☐ Errors ☐ Successes

| Label        | # Samples | Average | Min | Max | Std. Dev. | Error % | Throughput | Received KB/sec | Sent KB/sec | Arg. Bytes |
|--------------|-----------|---------|-----|-----|-----------|---------|------------|-----------------|-------------|------------|
| HTTP Request | 600       | 88      | 16  | 374 | 63.83     | 0.33%   | 17.4/sec   | 8.16            | 542.01      | 480.5      |
| TOTAL        | 600       | 88      | 16  | 374 | 63.83     | 0.33%   | 17.4/sec   | 8.16            | 542.01      | 480.5      |



#### 4. 250人3回

Summary Report

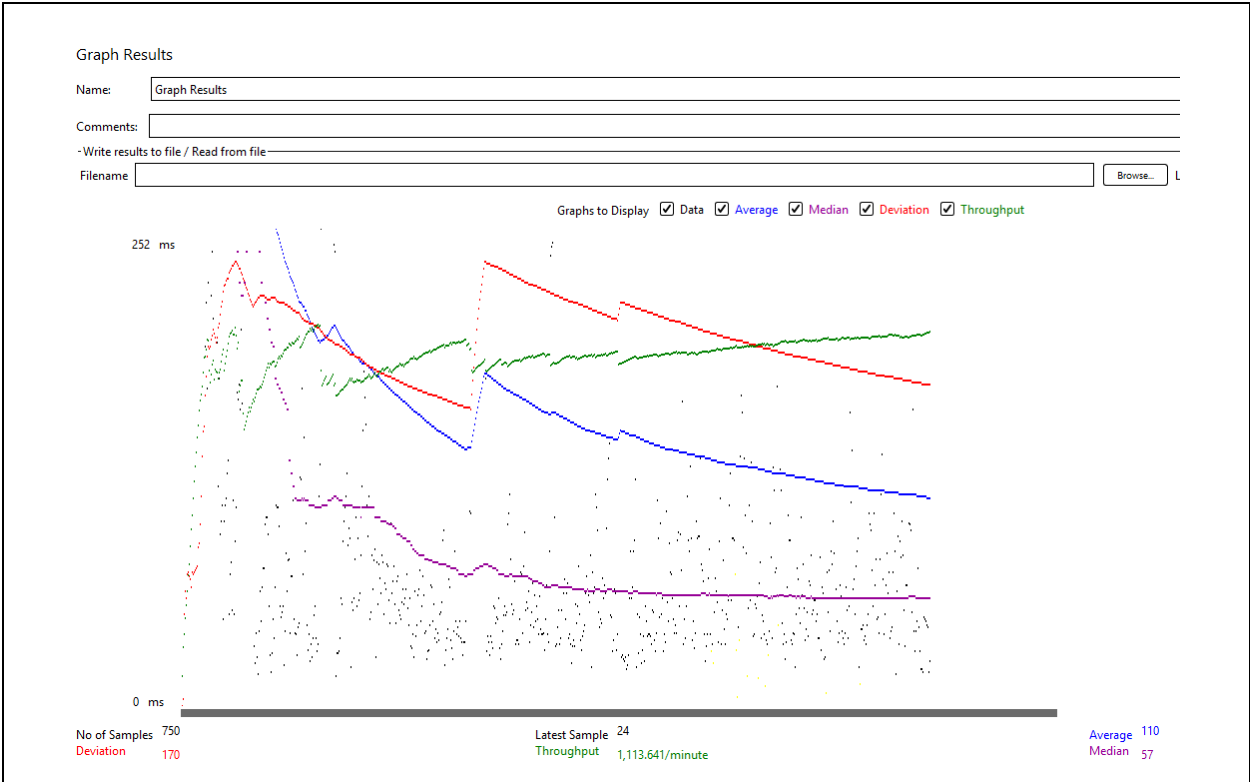
Name:

Comments:

-Write results to file / Read from file

Filename:   Log/Display Only: ☐ Errors ☐ Successes

| Label        | # Samples | Average | Min | Max  | Std. Dev. | Error % | Throughput | Received KB/sec | Sent KB/sec | Avg. Bytes |
|--------------|-----------|---------|-----|------|-----------|---------|------------|-----------------|-------------|------------|
| HTTP Request | 750       | 110     | 5   | 1044 | 170.26    | 1.73%   | 18.6/sec   | 8.68            | 578.76      | 478.8      |
| TOTAL        | 750       | 110     | 5   | 1044 | 170.26    | 1.73%   | 18.6/sec   | 8.68            | 578.76      | 478.8      |



## 5. 300人3回

Summary Report

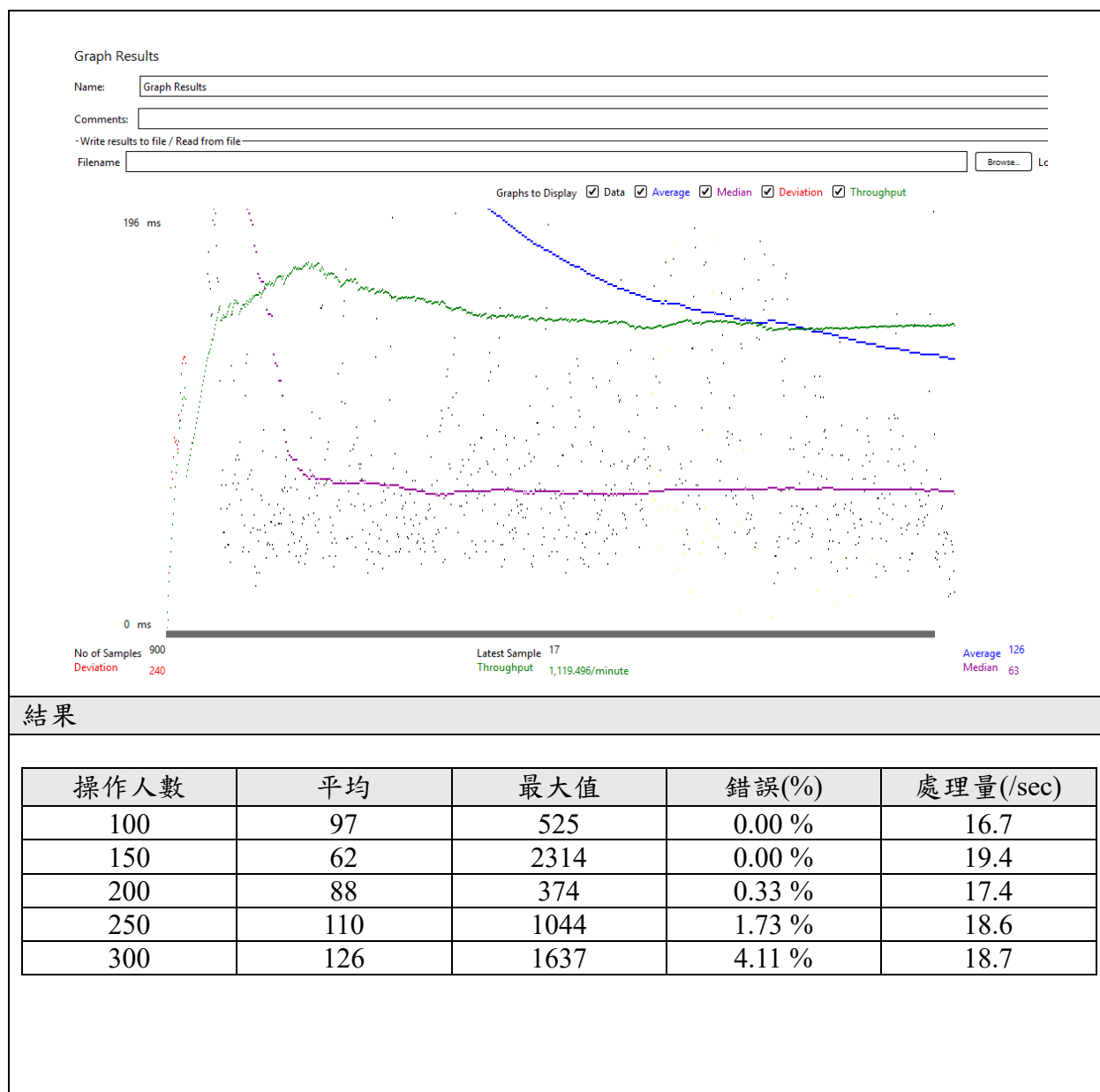
Name:

Comments:

- Write results to file / Read from file

Filename   ☐ Log/Display Only: ☐ Errors ☐ Successes

| Label        | # Samples | Average | Min | Max  | Std. Dev. | Error % | Throughput | Received KB/sec | Sent KB/sec | Avg. Bytes |
|--------------|-----------|---------|-----|------|-----------|---------|------------|-----------------|-------------|------------|
| HTTP Request | 900       | 126     | 5   | 1637 | 240.81    | 4.11%   | 18.7/sec   | 8.67            | 581.68      | 475.7      |
| TOTAL        | 900       | 126     | 5   | 1637 | 240.81    | 4.11%   | 18.7/sec   | 8.67            | 581.68      | 475.7      |



### Burst Mode 測試結果：

在錯誤出現（500 Error）之前，平均回應時間 (Average Response Time) 已經呈現線性甚至指數級的飆升。這代表伺服器並不是「突然」壞掉的，而是伺服器「處理不完，被塞爆了」=> 伺服器瓶頸。